



**Politécnico
de Viseu**

Escola Superior
de Tecnologia
e Gestão de Viseu

Vision4Farms Mobile

Diogo Paulo Ventura Rodrigues Augusto Caçador

Trabalho de Projeto

Licenciatura em

Engenharia Informática

junho de 2026



**Politécnico
de Viseu**

Escola Superior
de Tecnologia
e Gestão de Viseu



Vision4Farms Mobile

Diogo Paulo Ventura Rodrigues Augusto Caçador

Realizado na empresa Visionary Devotion Unipessoal Lda

Relatório de Projeto

Licenciatura em

Engenharia Informática

Trabalho efetuado sob a orientação de

Paulo Tomé (ESTGV)

Rafael Marques (Visionary Devotion Unipessoal Lda)

junho de 2026

Dedicatória

Aos meus pais, pelo apoio incondicional, pelos sacrifícios realizados e pela confiança que sempre depositaram em mim ao longo do meu percurso acadêmico.

Esta etapa é também resultado do vosso esforço e dedicação.

Agradecimentos

O presente relatório do projeto representa a conclusão de uma etapa importante do meu percurso académico, cuja concretização só foi possível com o apoio e contributo de várias pessoas e entidades, às quais expresso o meu sincero agradecimento.

Ao Professor Paulo Tomé, orientador pela instituição, pelo acompanhamento ao longo do projeto, pela disponibilidade demonstrada, pelas sugestões apresentadas e pela orientação prestada, que contribuíram de forma significativa para a qualidade do trabalho desenvolvido.

Ao Rafael Marques, orientador pela empresa Visionary Devotion Unipessoal Lda, pela oportunidade de integrar um projeto inovador na área da tecnologia agrícola, pela confiança depositada e por todo o apoio, acompanhamento e partilha de conhecimento ao longo das quinze semanas de projeto.

À Visionary Devotion Unipessoal Lda, pela integração num contexto profissional desafiante e pela possibilidade de participar no desenvolvimento de uma aplicação móvel de gestão agrícola, experiência que constituiu uma importante oportunidade de aprendizagem e valorização pessoal e profissional.

Ao corpo docente da Licenciatura em Engenharia Informática do Instituto Politécnico de Viseu, pelos conhecimentos transmitidos ao longo da formação, os quais se revelaram fundamentais para o desenvolvimento das competências aplicadas neste projeto.

Aos meus colegas, pela partilha de experiências, pelo espírito de entreajuda e pelo companheirismo demonstrado ao longo do percurso académico.

Por fim, à minha família, pelo apoio incondicional, pela paciência e pela motivação constante ao longo desta etapa.

Resumo

O presente relatório descreve o desenvolvimento do projeto Vision4Farms Mobile, uma aplicação móvel de apoio à gestão agrícola realizada na empresa Visionary Devotion Unipessoal Lda. O trabalho foi desenvolvido no âmbito da unidade curricular de Projeto da Licenciatura em Engenharia Informática e teve a duração de quinze semanas.

O projeto incidiu sobre duas componentes principais: a aplicação móvel e a API de suporte. No servidor, foi desenvolvida uma API RESTful com Django e Django REST Framework, assente numa base de dados MySQL. Esta API suporta autenticação por JSON Web Token (JWT), gestão de utilizadores, explorações agrícolas, terrenos, culturas, atividades, observações de campo, agenda, rega, notificações *in-app* e notificações *push* com Firebase Cloud Messaging.

Na componente *mobile*, foi desenvolvida uma aplicação em Flutter para Android e iOS, com validação realizada em ambiente Android. A aplicação inclui autenticação, seleção de exploração, *dashboard*, mapa interativo, gestão de terrenos e culturas, registo de atividades, observações e colheitas, agenda, rega, associação de sensores IoT, consulta de leituras em tempo real, alertas e decisões de apoio à decisão, geração de documentos PDF, notificações *push* e suporte *offline* permitindo consultar dados em cache, registar operações pendentes e sincronizá-las quando a ligação é restabelecida.

O presente projeto permitiu aplicar conhecimentos adquiridos ao longo da licenciatura, nomeadamente em desenvolvimento *mobile*, APIs REST, autenticação, modelação de bases de dados relacionais e integração entre sistemas. Os resultados obtidos constituem uma base funcional sólida para a continuidade e evolução futura do projeto Vision4Farms Mobile.

PALAVRAS-CHAVE: gestão agrícola; aplicação móvel; Flutter; Django REST Framework; MySQL; suporte offline.

Abstract

This report describes the work developed during the project work of the Bachelor's degree in Computer Engineering, carried out at Visionary Devotion Unipessoal Lda, in the context of the Vision4Farms Mobile project. The project lasted fifteen weeks and aimed to develop a mobile application to support agricultural farm management, along with the respective server-side component.

The project focused on two main components: the mobile application and the supporting API. On the server side, a RESTful API was developed using Django and Django REST Framework, backed by a MySQL database. This API supports JSON Web Token (JWT) authentication, user management, agricultural farms, lands, crops, activities, field observations, agenda, irrigation, in-app notifications and push notifications using Firebase Cloud Messaging.

On the mobile side, a Flutter application was developed for Android and iOS, with validation carried out in an Android environment. The application includes authentication, farm selection, a dashboard, an interactive map, land and crop management, activity and observation registration, agenda, irrigation, IoT sensor association, real-time sensor readings, push notifications and offline support, allowing data to be consulted from cache, pending operations to be stored and later synchronised when connectivity is restored.

The project allowed the application of knowledge acquired throughout the degree, particularly in mobile development, REST APIs, authentication, relational database modelling and system integration. The results obtained provide a solid functional foundation for the continued development and future evolution of the Vision4Farms Mobile project.

KEYWORDS: agricultural management; mobile application; Flutter; Django REST Framework; MySQL; offline support.

Índice

ÍNDICE DE FIGURAS	XIII
ÍNDICE DE TABELAS	XV
LISTA DE SIGLAS / ABREVIATURAS.....	XVI
GLOSSÁRIO.....	XVIII
1 INTRODUÇÃO.....	1
1.1 ENQUADRAMENTO E MOTIVAÇÃO.....	1
1.2 ENTIDADE DE ACOLHIMENTO	3
1.3 OBJETIVOS E RESULTADOS ESPERADOS	4
1.4 PLANO DE TRABALHO	6
1.5 ESTRUTURA DO RELATÓRIO.....	8
2 ESTADO DA ARTE E ANÁLISE DE SOLUÇÕES DIGITAIS	10
2.1 TRANSFORMAÇÃO DIGITAL NO SETOR AGRÍCOLA	10
2.2 AGRICULTURA DE PRECISÃO E GESTÃO AGRÍCOLA DIGITAL	11
2.3 APLICAÇÕES MÓVEIS NO APOIO À GESTÃO DE EXPLORAÇÕES AGRÍCOLAS	13
2.4 MONITORIZAÇÃO AGRÍCOLA, SENSORES E REGA INTELIGENTE	14
2.5 ANÁLISE DE SOLUÇÕES EXISTENTES NO MERCADO.....	15
2.6 COMPARAÇÃO FUNCIONAL ENTRE SOLUÇÕES ANALISADAS	17
2.7 POSICIONAMENTO DO VISION4FARMS MOBILE.....	21
3 METODOLOGIAS, TECNOLOGIAS E FERRAMENTAS UTILIZADAS	23
3.1 METODOLOGIA E ORGANIZAÇÃO DO DESENVOLVIMENTO.....	24
3.2 DESENVOLVIMENTO DA APLICAÇÃO MÓVEL.....	25
3.3 DESENVOLVIMENTO DA API E BACKEND.....	28
3.4 BASE DE DADOS E MODELAÇÃO DA INFORMAÇÃO.....	30
3.5 MAPAS, GEORREFERENCIAÇÃO E INTEGRAÇÃO VISUAL.....	31
3.6 FERRAMENTAS DE DESENVOLVIMENTO, TESTE E DISPONIBILIZAÇÃO	32

3.7	JUSTIFICAÇÃO GLOBAL DAS OPÇÕES TECNOLÓGICAS	34
4	ANÁLISE E ARQUITETURA DA SOLUÇÃO	37
4.1	VISÃO GERAL DA SOLUÇÃO	37
4.2	REQUISITOS FUNCIONAIS.....	38
4.3	REQUISITOS NÃO FUNCIONAIS	39
4.4	ARQUITETURA GERAL DO SISTEMA	40
4.5	ARQUITETURA DA APLICAÇÃO MÓVEL	42
4.6	ARQUITETURA DA API.....	44
4.7	MODELO DE DADOS	45
4.8	COMUNICAÇÃO ENTRE APLICAÇÃO MÓVEL E BACKEND	47
4.9	SEGURANÇA, AUTENTICAÇÃO E CONTROLO DE ACESSO	48
4.10	SUORTE OFFLINE E SINCRONIZAÇÃO DE DADOS	50
5	ATIVIDADES DESENVOLVIDAS	51
5.1	FASE 1 — ANÁLISE E PREPARAÇÃO	51
5.2	FASE 2 — BASE DE DADOS, BACKEND E AUTENTICAÇÃO	52
5.3	FASE 3 — ESTRUTURA BASE DA APLICAÇÃO MÓVEL	54
5.4	FASE 4 — DASHBOARD E MAPA INTERATIVO.....	57
5.5	FASE 5 — TERRENOS, CULTURAS, ATIVIDADES E OBSERVAÇÕES.....	60
5.6	FASE 6 — REGA, AGENDA, NOTIFICAÇÕES E SUORTE OFFLINE	65
5.7	TESTES, AJUSTES FINAIS E PREPARAÇÃO DA APK	71
6	RESULTADOS OBTIDOS.....	73
6.1	FUNCIONALIDADES IMPLEMENTADAS.....	73
6.2	RESULTADOS OBTIDOS NO BACKEND.....	74
6.3	RESULTADOS OBTIDOS NA APLICAÇÃO MÓVEL	75
6.4	RESULTADOS OBTIDOS NA BASE DE DADOS.....	77
6.5	RESULTADOS OBTIDOS NO SUORTE OFFLINE E SINCRONIZAÇÃO	78
6.6	VALIDAÇÃO DOS OBJETIVOS DEFINIDOS	79
6.7	LIMITAÇÕES IDENTIFICADAS	80
7	CONCLUSÃO	82
7.1	BALANÇO GERAL DO PROJETO	82
7.2	PRINCIPAIS RESULTADOS E CONTRIBUIÇÕES	83
7.3	COMPETÊNCIAS DESENVOLVIDAS	84
7.4	DIFICULDADES ENCONTRADAS	85
7.5	TRABALHO FUTURO.....	86

REFERÊNCIAS BIBLIOGRÁFICAS	89
ANEXO A – PLANEAMENTO DE ATIVIDADES	92
ANEXO B – MODELO DE DADOS	94
ANEXO C – ENDPOINTS DA API	96
ANEXO D – CAPTURAS DE ECRÃ DA APLICAÇÃO MÓVEL	102
D.1 AUTENTICAÇÃO E ACESSO.....	102
D.2 GESTÃO DE EXPLORAÇÕES	104
D.3 DASHBOARD, PERFIL E DEFINIÇÕES	105
D.4 MAPA E TERRENOS	108
D.5 ATIVIDADES, SENSORES, REGA E CULTURAS	112
D.6 AGENDA E NOTIFICAÇÕES.....	115
ANEXO E – MANUAL DE INSTALAÇÃO E EXECUÇÃO	117

Índice de Figuras

FIGURA 1.1 - CRONOGRAMA GERAL DO DESENVOLVIMENTO DO PROJETO	8
FIGURA 4.1 - ARQUITETURA GERAL DO SISTEMA	42
FIGURA 4.2 - DIAGRAMA DO MODELO DE DADOS.....	47
FIGURA 5.1 - FLUXO INICIAL DA APLICAÇÃO MÓVEL	56
FIGURA 5.2 - DASHBOARD E MENU DE AÇÕES.....	58
FIGURA 5.3 - MAPA INTERATIVO E PRÉ-VISUALIZAÇÃO DE TERRENO	59
FIGURA 5.4 - CRIAÇÃO DE TERRENO E DESENHO DE POLÍGONO.....	61
FIGURA 5.5 - DETALHE DO TERRENO ORGANIZADO POR SEPARADORES	62
FIGURA 5.6 - GESTÃO DE ATIVIDADES E OBSERVAÇÕES DE CAMPO	65
FIGURA 5.7 - MÓDULOS DE SENSORES, REGA E AGENDA.....	68
FIGURA 5.8 - NOTIFICAÇÕES IN-APP E NOTIFICAÇÕES PUSH	69
FIGURA 5.9 - SUPORTE OFFLINE E SINCRONIZAÇÃO DE DADOS.....	70
FIGURA D.1 - ECRÃ DE AUTENTICAÇÃO	102
FIGURA D.2 - FORMULÁRIO DE REGISTO COM CÓDIGO DE CONVITE.....	103
FIGURA D.3 - ECRÃ DE RECUPERAÇÃO DE PALAVRA-PASSE	103
FIGURA D.4 - FORMULÁRIO DE CRIAÇÃO DE EXPLORAÇÃO	104
FIGURA D.5 - LISTAGEM DE EXPLORAÇÕES DO UTILIZADOR.....	104
FIGURA D.6 - ECRÃ DE ENTRADA POR CÓDIGO DE CONVITE	105
FIGURA D.7 - DASHBOARD COM ÁREA CULTIVADA, METEOROLOGIA E SENSORES	105
FIGURA D.8 - DASHBOARD COM ATIVIDADES, AGENDA, REGA E ANÁLISES	106
FIGURA D.9 - ECRÃ DE PERFIL COM OPÇÕES DE GESTÃO.....	106
FIGURA D.10 - FORMULÁRIO DE EDIÇÃO DE PERFIL.....	107
FIGURA D.11 - CONFIGURAÇÃO DE PREFERÊNCIAS DE NOTIFICAÇÕES	107
FIGURA D.12 - MAPA INTERATIVO COM MARCADORES DE TERRENOS E LEGENDA DE ESTADO	108
FIGURA D.13 - <i>BOTTOM SHEET</i> DE PRÉ-VISUALIZAÇÃO DO TERRENO NO MAPA.....	108
FIGURA D.14 - LISTAGEM DOS TERRENOS DA EXPLORAÇÃO	109
FIGURA D.15 - ECRÃ DE DETALHE DO TERRENO	109
FIGURA D.16 - DADOS DE SENSORES COM GRÁFICOS DE HISTÓRICO.....	110
FIGURA D.17 - PRÓXIMAS REGAS E HISTÓRICO DE REGA DO TERRENO	110
FIGURA D.18 - ATIVIDADES ASSOCIADAS AO TERRENO	111

FIGURA D.19 - OBSERVAÇÕES DE CAMPO DO TERRENO	111
FIGURA D.20 - ANÁLISE DE SOLO COM VALORES LABORATORIAIS.....	112
FIGURA D.21 - LISTAGEM DE ATIVIDADES AGRÍCOLAS COM INDICADOR DE ATRASOS E FILTRO POR PRIORIDADE	112
FIGURA D.22 - FORMULÁRIO DE CRIAÇÃO DE ATIVIDADE AGRÍCOLA COM SELEÇÃO DE TIPO E PRIORIDADE.....	113
FIGURA D.23 - MÓDULO DE SENSORES, NÓS, ALERTAS E DECISÕES DE APOIO À DECISÃO	113
FIGURA D.24 - GESTÃO DE FONTES DE ÁGUA DA EXPLORAÇÃO.....	114
FIGURA D.25 - LISTAGEM DE CULTURAS COM MÉTRICAS RESUMIDAS	114
FIGURA D.26 - DETALHE DE CULTURA COM INFORMAÇÃO AGRONÓMICA E VARIEDADES	115
FIGURA D.27 - MÓDULO DE AGENDA COM CALENDÁRIO E FILTRO POR TERRENO	115
FIGURA D.28 - EVENTOS PENDENTES E EM ATRASO	116
FIGURA D.29 - ECRÃ DE NOTIFICAÇÕES COM ALERTAS DE ATIVIDADES E OBSERVAÇÕES	116

Índice de Tabelas

TABELA 1.1 - PLANEAMENTO GERAL DO PROJETO.....	7
TABELA 2.1 - COMPARAÇÃO FUNCIONAL ENTRE SOLUÇÕES DE GESTÃO AGRÍCOLA.....	19
TABELA A.1 - PLANEAMENTO DE ATIVIDADES DO PROJETO	92
TABELA B.1 - TABELAS DA BASE DE DADOS.....	94
TABELA C.1 - PRINCIPAIS <i>ENDPOINTS</i> DA API	96

Lista de siglas / abreviaturas

API - Application Programming Interface

APK - Android Package Kit

BD - Base de Dados

DRF - Django REST Framework

FCM - Firebase Cloud Messaging

GPS - Global Positioning System

HTTP - Hypertext Transfer Protocol

HTTPS - Hypertext Transfer Protocol Secure

IDE - Integrated Development Environment

IoT - Internet of Things

iOS - iPhone Operating System

JSON - JavaScript Object Notation

JWT - JSON Web Token

ORM - Object-Relational Mapping

REST - Representational State Transfer

SDK - Software Development Kit

SMTP - Simple Mail Transfer Protocol

SQL - Structured Query Language

SQLite - Base de dados relacional embutida

TDM - Tecnologias e Design de Multimédia

UI - User Interface

URL - Uniform Resource Locator

UX - User Experience

UUID - Universally Unique Identifier

Glossário

Agricultura de precisão — Conjunto de práticas que utilizam tecnologias digitais, sensores, dados geográficos e sistemas de informação para apoiar a gestão agrícola de forma mais eficiente e ajustada às necessidades reais da exploração.

Aplicação móvel — Software desenvolvido para utilização em dispositivos móveis, como smartphones ou tablets.

API RESTful — Interface de comunicação entre sistemas baseada nos princípios REST, permitindo que a aplicação móvel comunique com o servidor através de pedidos HTTP.

Autenticação — Processo utilizado para confirmar a identidade de um utilizador antes de permitir o acesso ao sistema.

Backend — Componente de servidor responsável pela lógica de negócio, gestão de dados, autenticação e disponibilização de informação à aplicação móvel.

Base de dados relacional — Sistema de armazenamento de dados organizado em tabelas relacionadas entre si através de chaves primárias e estrangeiras.

Cache — Armazenamento temporário de dados com o objetivo de permitir acesso mais rápido ou consulta posterior, mesmo em situações de ausência de ligação à internet.

Dashboard — Ecrã de resumo que apresenta os principais indicadores e informações de uma exploração agrícola.

Deploy — Processo de disponibilização de uma aplicação ou serviço num ambiente onde possa ser utilizado ou testado.

Endpoint — URL específica de uma API que permite aceder a determinada funcionalidade ou recurso do sistema.

Exploração — No contexto do Vision4Farms, unidade organizacional de topo que representa uma exploração agrícola. Toda a informação operacional, como terrenos, culturas, atividades, rega e notificações, está organizada por exploração.

Fallback — Mecanismo alternativo utilizado quando a fonte principal de dados não está disponível. No contexto da aplicação, permite recorrer a dados meteorológicos externos quando não existem leituras de sensores disponíveis.

Frontend — Componente da aplicação com a qual o utilizador interage diretamente, incluindo ecrãs, botões, formulários e navegação.

Georreferenciação — Associação de informação a coordenadas geográficas, permitindo representar elementos como terrenos ou explorações num mapa.

Hash — Resultado de uma função criptográfica aplicada a um valor, como uma palavra-passe, permitindo armazenar ou comparar informação de forma mais segura sem guardar o valor original em texto claro.

JSON Web Token (JWT) — Mecanismo de autenticação baseado na emissão de *tokens* digitais assinados, que identificam o utilizador autenticado e são enviados em cada pedido à API para validar o acesso.

Keychain — Mecanismo seguro de armazenamento utilizado em dispositivos iOS para guardar informação sensível, como *tokens* de autenticação.

Keystore — Mecanismo seguro de armazenamento utilizado em dispositivos Android para guardar informação sensível, como *tokens* de autenticação.

Modelo de dados — Estrutura que define as entidades, atributos e relações existentes numa base de dados.

Notificações push — Mensagens enviadas para o dispositivo do utilizador através de um serviço externo, como o Firebase Cloud Messaging, podendo ser recebidas mesmo quando a aplicação não está aberta.

Offline — Estado em que a aplicação é utilizada sem ligação ativa à internet, recorrendo a dados previamente guardados localmente.

Sincronização — Processo de atualização e troca de dados entre a aplicação móvel e o servidor, garantindo que ambos mantêm informação consistente.

Token — Valor digital utilizado para representar uma sessão, autorização ou pedido temporário. No projeto, é utilizado na autenticação JWT, recuperação de palavra-passe e notificações *push*.

Widget — Elemento base da interface em Flutter, utilizado para construir componentes visuais como textos, botões, formulários, listas e cartões.

1 Introdução

O presente relatório descreve o desenvolvimento do projeto Vision4Farms Mobile, uma aplicação móvel de apoio à gestão agrícola realizada na empresa Visionary Devotion Unipessoal Lda. O trabalho foi desenvolvido no âmbito da unidade curricular de Projeto da Licenciatura em Engenharia Informática.

A Engenharia Informática assume atualmente um papel relevante em diferentes setores de atividade, contribuindo para a criação de soluções capazes de melhorar processos, organizar informação, apoiar a tomada de decisão e aumentar a eficiência das organizações. No setor agrícola, esta importância torna-se particularmente evidente através da digitalização de processos associados à gestão de explorações, consulta de dados operacionais e acompanhamento de atividades no terreno.

1.1 Enquadramento e motivação

A transformação digital tem vindo a ganhar expressão no setor agrícola, criando novas oportunidades para modernizar práticas tradicionais e melhorar a gestão das explorações. A utilização de aplicações móveis, sistemas de informação, mapas digitais, bases de dados e mecanismos de sincronização permite aproximar a tecnologia das necessidades reais dos agricultores e dos responsáveis pela gestão agrícola.

No contexto de uma exploração agrícola, existem várias informações que necessitam de ser consultadas e atualizadas com frequência, como terrenos, culturas, atividades, observações de campo, regas planeadas, notificações e dados associados à gestão operacional. A existência de uma aplicação móvel permite que essa informação esteja disponível diretamente no terreno, facilitando o registo de ocorrências, o

acompanhamento das tarefas e a consulta de dados relevantes de forma rápida e organizada.

Neste contexto surge o projeto Vision4Farms Mobile, orientado para o desenvolvimento de uma aplicação móvel de apoio à gestão agrícola. A solução pretende disponibilizar ao utilizador um conjunto de funcionalidades úteis no dia a dia de uma exploração, permitindo a consulta de informação geral, a visualização de terrenos em mapa, a gestão de culturas, atividades, observações, rega, notificações e outros elementos relevantes para a organização da atividade agrícola.

A motivação para o desenvolvimento do Vision4Farms Mobile resulta, assim, da necessidade de criar uma solução digital capaz de apoiar a gestão agrícola em mobilidade. O projeto não se limita à construção de uma aplicação móvel isolada, mas envolve também o desenvolvimento de uma API de suporte, a adaptação da base de dados, a integração entre sistemas e a preparação de mecanismos que permitam o funcionamento em cenários de conectividade limitada.

Outro fator relevante prende-se com a necessidade de disponibilizar uma solução adaptada ao contexto real de utilização. Em ambiente agrícola, nem sempre existe ligação estável à internet, e a utilização da aplicação pode ocorrer diretamente no campo. Por esse motivo, o projeto contemplou também mecanismos de suporte *offline* e sincronização, permitindo a consulta de dados previamente carregados e o registo de operações pendentes para posterior envio ao servidor.

Desta forma, o projeto enquadra-se num contexto de desenvolvimento de software aplicado a um problema real, permitindo articular conhecimentos de programação, bases de dados, APIs, desenvolvimento *mobile*, autenticação, mapas, integração de sistemas e desenho de soluções orientadas para utilização prática no terreno.

1.2 Entidade de acolhimento

A Visionary Devotion Unipessoal Lda, que opera comercialmente sob a designação Vision4You, é uma empresa tecnológica focada na implementação de soluções digitais adaptadas a cada organização, promovendo eficiência, crescimento sustentável e sucesso através de novos modelos de negócio digitais. A empresa ambiciona criar uma melhor sociedade através da tecnologia digital, encorajando redes colaborativas de inovação, criatividade e talento.

As principais áreas de atuação da empresa organizam-se em três domínios: inovação de negócios, com apoio na definição de estratégias de tecnologias de informação e na implementação de recursos digitais; inovação tecnológica, com foco na aplicação estratégica de inteligência artificial, na tomada de decisão baseada em dados, na otimização de processos e na convergência entre automação inteligente e fator humano; e inovação para a sustentabilidade, com participação em projetos de inovação social e colaborativa orientados para o impacto positivo no negócio, nas pessoas e no planeta.

O projeto foi realizado na empresa Visionary Devotion Unipessoal Lda (Vision4You, 2023), entidade responsável pelo projeto Vision4Farms. A empresa proporcionou um contexto de trabalho real, no qual foi possível contactar com requisitos concretos, necessidades funcionais bem definidas e decisões técnicas associadas ao desenvolvimento de uma solução digital dirigida ao setor agrícola.

No âmbito do projeto, foram disponibilizados elementos de base para o desenvolvimento do projeto, nomeadamente requisitos funcionais, documentação técnica, estrutura da base de dados e referências visuais para a aplicação móvel. Parte da componente visual teve como referência um protótipo desenvolvido em Figma por um colega da área de Tecnologias e Design de Multimédia (TDM), no âmbito de um projeto relacionado. Esse protótipo serviu como base de inspiração para alguns layouts, fluxos de navegação e organização visual da aplicação.

No entanto, a implementação final exigiu a adaptação, recriação e complementação desses layouts em Flutter, tendo em conta os requisitos funcionais definidos, os

dados reais disponibilizados pela API, os estados da interface e as necessidades específicas da aplicação. Assim, o trabalho desenvolvido no presente projeto centrou-se no desenvolvimento técnico da aplicação móvel em Flutter, no desenvolvimento da API e da componente de servidor em Django REST Framework, na integração com a base de dados MySQL e na ligação entre a aplicação móvel e o *backend*.

A realização do projeto nesta entidade revelou-se importante do ponto de vista académico e profissional, uma vez que permitiu aplicar conhecimentos adquiridos ao longo da licenciatura num contexto prático, com objetivos concretos e próximos da realidade do mercado de trabalho. Simultaneamente, possibilitou o desenvolvimento de competências relacionadas com autonomia, organização, responsabilidade, capacidade de adaptação e resolução de problemas técnicos.

1.3 Objetivos e resultados esperados

O principal objetivo do projeto consistiu em contribuir para o desenvolvimento do projeto Vision4Farms Mobile, através da análise, planeamento e implementação de uma aplicação móvel de apoio à gestão agrícola, acompanhada pela respetiva API e componente de servidor.

De forma mais específica, o projeto teve como objetivos:

- complementar a formação académica através do contacto com um contexto real de desenvolvimento de software;
- aplicar, em contexto prático, conhecimentos adquiridos ao longo da Licenciatura em Engenharia Informática;
- analisar os requisitos funcionais e técnicos do projeto;
- estudar e seleccionar tecnologias adequadas às necessidades da aplicação;
- adaptar a base de dados ao contexto da aplicação móvel;

- desenvolver a API RESTful responsável pela comunicação entre a aplicação móvel, a lógica de negócio e a base de dados;
- implementar mecanismos de autenticação, recuperação de palavra-passe e controlo de acesso;
- desenvolver a aplicação móvel em Flutter;
- integrar a aplicação móvel com a API;
- implementar funcionalidades associadas à gestão de explorações, terrenos, culturas, atividades, observações, rega, notificações e mapa;
- preparar mecanismos de suporte *offline* e sincronização de dados;
- disponibilizar a aplicação em formato APK para validação em ambiente Android;
- consolidar competências técnicas e profissionais em contexto real de trabalho.

Relativamente aos resultados esperados, pretendia-se obter uma base funcional do sistema Vision4Farms Mobile, composta pela aplicação móvel, pela API/*backend* e pela base de dados, suportada por uma arquitetura técnica adequada às necessidades do projeto. Entre os principais resultados previstos encontravam-se a implementação da autenticação, a seleção de exploração, a estrutura principal de navegação, o *dashboard*, o mapa interativo, a gestão de terrenos e culturas, o registo de atividades e observações, a agenda, as notificações, a rega e o suporte *offline*.

Para além dos resultados técnicos, esperava-se também o reforço de competências pessoais e profissionais, nomeadamente ao nível da organização do trabalho, análise de problemas, tomada de decisão, autonomia, comunicação com o orientador e adaptação a um contexto real de desenvolvimento de software.

1.4 Plano de trabalho

O projeto foi organizado ao longo de quinze semanas, distribuídas por várias fases de trabalho. Esta organização permitiu uma evolução progressiva, começando pela análise inicial e preparação técnica, passando depois para o desenvolvimento da base funcional da aplicação e, por fim, para a implementação dos módulos complementares, testes, ajustes e documentação.

Numa primeira fase, o trabalho incidiu sobre a análise da documentação disponível, o estudo de soluções concorrentes, a análise de mercado, a escolha das tecnologias e a preparação do ambiente de desenvolvimento. Esta etapa foi essencial para compreender o problema, enquadrar a solução a desenvolver e definir uma base técnica consistente.

Na segunda fase, iniciou-se a adaptação da base de dados e o desenvolvimento da API/*backend*, com foco na autenticação e nos primeiros *endpoints*. Posteriormente, o desenvolvimento avançou para a estrutura base da aplicação móvel, integração com a API e implementação progressiva dos módulos de *dashboard*, mapa, terrenos, culturas, atividades, observações, rega, agenda, notificações e suporte *offline*.

A Tabela 1 apresenta, de forma resumida, a organização das diferentes fases do desenvolvimento do projeto.

Tabela 1.1 - Planeamento geral do projeto

FASE	SEMANAS	PRINCIPAIS ATIVIDADES
FASE 1	1 a 2	Análise da documentação, estudo de soluções concorrentes, análise de mercado, escolha das tecnologias e preparação do ambiente de desenvolvimento
FASE 2	3 a 5	Adaptação da base de dados, desenvolvimento da API/ <i>backend</i> , autenticação, recuperação de palavra-passe, sistema de convites e primeiros <i>endpoints</i>
FASE 3	6 a 7	Estrutura base da aplicação móvel, navegação, gestão de explorações, integração com a API e implementação dos fluxos iniciais
FASE 4	8 a 9	Desenvolvimento do <i>dashboard</i> e do mapa interativo com Mapbox, incluindo representação dos terrenos através de polígonos
FASE 5	10 a 12	Desenvolvimento dos módulos de terrenos, culturas, atividades, observações de campo, análises e criação de terrenos com polígonos
FASE 6	13 a 15	Desenvolvimento dos módulos de rega, agenda, sensores IoT, notificações, suporte <i>offline</i> , sincronização, testes, ajustes finais e preparação da APK

Este planeamento permitiu organizar o projeto de forma faseada e coerente, evitando concentrar todo o desenvolvimento numa única etapa. A divisão por fases facilitou a validação progressiva das funcionalidades, a correção de problemas técnicos e a integração gradual entre a aplicação móvel, a API e a base de dados.

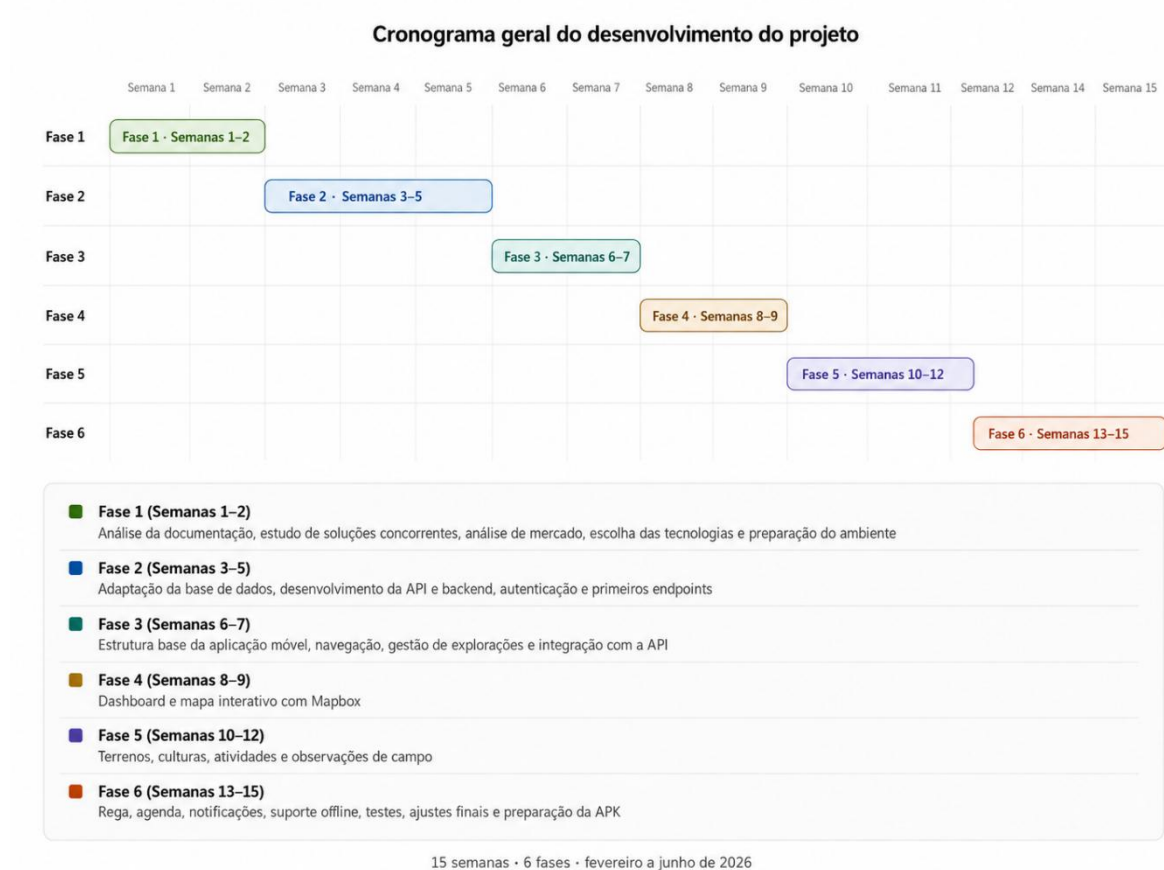


Figura 1.1 - Cronograma geral do desenvolvimento do projeto

1.5 Estrutura do relatório

O presente relatório encontra-se organizado em sete capítulos, estruturados de forma a enquadrar o projeto, apresentar a fundamentação do projeto, descrever as tecnologias utilizadas, explicar a arquitetura da solução, documentar o trabalho desenvolvido e refletir sobre os resultados obtidos.

No Capítulo 1 é apresentada a introdução ao relatório, incluindo o enquadramento e motivação do projeto, a entidade de acolhimento, os objetivos e resultados esperados, o plano de trabalho e a estrutura geral do documento.

No Capítulo 2 é desenvolvido o estado da arte e a análise de mercado, abordando a transformação digital no setor agrícola, a agricultura de precisão, as aplicações

móveis de apoio à gestão de explorações, a monitorização agrícola, os sensores, a rega inteligente e a comparação com soluções existentes no mercado.

No Capítulo 3 são apresentadas as metodologias, tecnologias e ferramentas utilizadas ao longo do projeto, incluindo as opções associadas ao desenvolvimento da aplicação móvel, à API, à base de dados, aos mapas, às ferramentas de teste, ao alojamento e à disponibilização da aplicação.

No Capítulo 4 é apresentada a análise e arquitetura da solução, descrevendo os requisitos funcionais e não funcionais, a arquitetura geral do sistema, a arquitetura da aplicação móvel, a arquitetura da API, o modelo de dados, a comunicação entre aplicação e *backend*, a segurança e o suporte *offline*.

No Capítulo 5 são descritas as atividades desenvolvidas durante o projeto, organizadas por fases de trabalho. Este capítulo apresenta o percurso de implementação realizado, desde a análise inicial até aos testes, ajustes finais e preparação da APK.

No Capítulo 6 são apresentados os resultados obtidos, com destaque para as funcionalidades implementadas, os resultados alcançados no *backend*, na aplicação móvel, na base de dados e no suporte *offline*, bem como a validação dos objetivos definidos e as limitações identificadas.

Por fim, no Capítulo 7 são apresentadas as conclusões do projeto, incluindo o balanço geral, os principais resultados e contribuições, as competências desenvolvidas, as dificuldades encontradas e o trabalho futuro.

2 Estado da Arte e Análise de Soluções Digitais

Neste capítulo é apresentado o enquadramento teórico e contextual do projeto Vision4Farms Mobile, com foco na transformação digital do setor agrícola, na agricultura de precisão, nas aplicações móveis de apoio à gestão de explorações e na utilização de tecnologias como sensores, mapas digitais e sistemas de rega inteligente.

Para além do enquadramento conceptual, é também apresentada uma análise de mercado a soluções existentes na área da gestão agrícola digital. Esta análise permitiu identificar funcionalidades relevantes, limitações das plataformas atuais e oportunidades de diferenciação para o projeto desenvolvido no âmbito do projeto.

2.1 Transformação digital no setor agrícola

A transformação digital tem vindo a assumir uma importância crescente no setor agrícola, contribuindo para a modernização de práticas tradicionais e para a melhoria da eficiência na gestão das explorações. A introdução de ferramentas digitais permite recolher, organizar e analisar informação de forma mais rápida, facilitando o acompanhamento das operações agrícolas e apoiando a tomada de decisão em contexto real (FAO, 2026; European Commission, 2025).

O setor agrícola é caracterizado pela existência de múltiplas variáveis que influenciam diretamente a produção, como as condições meteorológicas, o estado das culturas, a disponibilidade de água, a qualidade do solo, o planeamento de tarefas e a monitorização do terreno. Neste contexto, a digitalização representa uma

oportunidade para melhorar o controlo sobre a atividade agrícola, permitindo que os dados sejam registados, consultados e analisados de forma mais estruturada.

A utilização de soluções digitais no setor agrícola não se limita apenas à substituição de processos manuais por ferramentas tecnológicas. Trata-se também de uma mudança na forma como a informação é utilizada para apoiar a gestão diária das explorações. Através de aplicações móveis, plataformas web, sensores, sistemas de informação geográfica e bases de dados, torna-se possível centralizar informação operacional e reduzir a dispersão de dados entre documentos físicos, folhas de cálculo ou registos informais (European Commission, 2025).

Esta evolução tem sido acompanhada pelo aparecimento de plataformas especializadas em gestão agrícola, monitorização de culturas, planeamento de rega, registo de atividades, análise de dados e acompanhamento de indicadores de produção. No entanto, a diversidade de soluções existentes demonstra também que continuam a existir desafios ao nível da integração funcional, da simplicidade de utilização e da adaptação ao contexto real do agricultor no terreno.

Foi neste enquadramento que surgiu a necessidade de desenvolver o Vision4Farms Mobile, uma solução orientada para apoiar a gestão agrícola através de uma aplicação móvel integrada com uma API e uma base de dados. O objetivo do projeto passa por facilitar o acesso à informação da exploração e disponibilizar funcionalidades úteis para o acompanhamento das atividades agrícolas em mobilidade.

2.2 Agricultura de precisão e gestão agrícola digital

A agricultura de precisão está associada à utilização de tecnologias digitais para melhorar a gestão das explorações agrícolas, permitindo uma atuação mais informada, localizada e ajustada às necessidades reais de cada terreno ou cultura. Esta abordagem baseia-se na recolha e utilização de dados para apoiar decisões relacionadas com rega, fertilização, tratamentos, monitorização de pragas, produtividade e gestão de recursos (European Parliamentary Research Service, 2016).

Ao contrário de uma abordagem agrícola mais tradicional, onde muitas decisões são tomadas com base na experiência acumulada ou em observações gerais, a agricultura de precisão procura utilizar dados concretos para compreender melhor o estado da exploração. Estes dados podem ter origem em sensores, imagens de satélite, mapas digitais, registos de atividades, análises laboratoriais, observações de campo ou históricos de produção (European Parliamentary Research Service, 2016; Soussi et al., 2024).

No contexto da gestão agrícola digital, a informação deixa de estar isolada e passa a ser organizada em sistemas que permitem consultar, atualizar e relacionar diferentes elementos da exploração. Por exemplo, um terreno pode estar associado a culturas, atividades planeadas, observações de campo, regas realizadas, análises de solo e notificações. Esta relação entre dados permite uma leitura mais completa da realidade agrícola e facilita o acompanhamento operacional.

A gestão agrícola digital torna-se especialmente relevante quando permite ao utilizador aceder à informação em tempo útil. No caso de uma exploração agrícola, muitas decisões são tomadas diretamente no terreno, o que torna importante a existência de ferramentas móveis capazes de acompanhar o utilizador no local onde a atividade ocorre. Assim, a combinação entre agricultura de precisão e aplicações móveis contribui para uma gestão mais próxima da realidade diária da exploração.

O Vision4Farms Mobile enquadra-se nesta lógica, ao procurar reunir informação funcional, geográfica e operacional numa única aplicação móvel. A solução foi pensada para permitir a gestão de explorações, terrenos, culturas, atividades, observações, rega e notificações, organizando os dados em torno da exploração selecionada pelo utilizador.

2.3 Aplicações móveis no apoio à gestão de explorações agrícolas

O crescimento das aplicações móveis no setor agrícola resulta da necessidade de aceder à informação em mobilidade, diretamente no terreno, sem dependência exclusiva de plataformas de secretária ou de contextos administrativos. A utilização de dispositivos móveis permite consultar dados da exploração, confirmar tarefas, registar ocorrências, visualizar informação geográfica e acompanhar atividades no momento em que estas acontecem (FAO, 2026; European Commission, 2025).

Numa exploração agrícola, o utilizador pode necessitar de consultar rapidamente que atividades estão pendentes, que terrenos exigem atenção, que culturas estão ativas ou que regas estão planeadas. Se esta informação estiver acessível numa aplicação móvel, o processo de gestão torna-se mais direto e menos dependente de registos manuais ou de deslocações a um computador.

Apesar de existirem várias soluções digitais com aplicação móvel, nem todas foram concebidas com a mesma prioridade de utilização em contexto de campo. Em alguns casos, a aplicação móvel surge apenas como complemento de uma plataforma web, mantendo interfaces densas ou pouco ajustadas à utilização rápida em dispositivos móveis. Esta limitação pode dificultar a adoção por utilizadores que necessitam de simplicidade, rapidez e clareza durante o trabalho diário.

No caso do Vision4Farms Mobile, a componente móvel assume-se como elemento central da solução. A aplicação foi organizada em torno da exploração selecionada, disponibilizando áreas como *Dashboard*, Mapa, Atividades, Terrenos, Culturas e outras secções complementares. Esta estrutura permite que o utilizador aceda rapidamente aos principais módulos da aplicação e mantenha sempre o contexto da exploração em que está a trabalhar.

Outro aspeto importante nas aplicações móveis agrícolas é a possibilidade de funcionamento em ambientes com conectividade limitada. Em contexto rural ou em zonas de campo, a ligação à internet pode ser instável ou inexistente. Por esse

motivo, uma aplicação móvel de apoio à gestão agrícola deve considerar mecanismos de cache, leitura *offline* e sincronização posterior, garantindo que o utilizador consegue continuar a consultar ou registar informação mesmo quando a ligação não está disponível.

Deste modo, as aplicações móveis representam uma ferramenta relevante para aproximar os sistemas digitais da realidade operacional das explorações agrícolas. Quando bem desenhadas, podem reduzir a distância entre a gestão administrativa e o trabalho no terreno, tornando a informação mais acessível, atualizada e útil.

2.4 Monitorização agrícola, sensores e rega inteligente

A monitorização agrícola baseada em sensores constitui uma das vertentes mais relevantes da digitalização do setor. A possibilidade de recolher dados sobre variáveis como humidade, temperatura, condições do solo, disponibilidade de água e estado da rega permite obter uma visão mais precisa sobre a exploração e apoiar decisões mais informadas (Soussi et al., 2024).

Os sensores agrícolas podem contribuir para melhorar a eficiência na utilização de recursos, especialmente no caso da água. A rega é uma das operações mais importantes numa exploração agrícola, e a sua gestão inadequada pode provocar desperdício de recursos, aumento de custos ou impacto negativo no desenvolvimento das culturas. A integração de dados de sensores com sistemas de planeamento de rega permite acompanhar melhor as necessidades das culturas e tomar decisões mais ajustadas (European Parliamentary Research Service, 2016; Soussi et al., 2024).

A rega inteligente surge, neste contexto, como uma área de particular relevância. Através da combinação entre sensores, histórico de regas, previsões meteorológicas e necessidades das culturas, torna-se possível planear ou ajustar operações de rega de forma mais eficiente. Embora nem todas as soluções de gestão agrícola incluam

este tipo de funcionalidade, as plataformas mais avançadas tendem a integrar mecanismos de monitorização e apoio à decisão.

No conjunto das soluções analisadas no âmbito do projeto, verificou-se que a integração de sensores e de mecanismos de rega inteligente não está presente de forma uniforme. Algumas plataformas, como a Bee2Crop e a Wisecrop, apresentam maior maturidade neste domínio, com funcionalidades ligadas à rega, fertirrega, sensores e monitorização em tempo real. No entanto, estas soluções tendem a estar mais centradas nessa dimensão específica ou a enquadrá-la numa abordagem mais técnica.

No caso do Vision4Farms Mobile, a monitorização e a rega são encaradas como parte de um ecossistema mais abrangente de gestão agrícola. A informação relativa à rega, às observações, aos terrenos e às culturas é organizada no contexto da exploração, permitindo uma leitura integrada da atividade agrícola. Assim, a rega não surge apenas como uma funcionalidade isolada, mas como uma dimensão relacionada com o restante funcionamento da exploração.

Desta forma, a monitorização agrícola, os sensores e a rega inteligente assumem um papel importante no enquadramento do projeto, não apenas enquanto tecnologias específicas, mas enquanto componentes que podem melhorar o acompanhamento das explorações, apoiar o planeamento e reduzir a fragmentação da informação.

2.5 Análise de soluções existentes no mercado

Após o enquadramento teórico apresentado nas secções anteriores, foi realizada uma análise de mercado a várias soluções digitais existentes na área da gestão agrícola. Esta análise teve como objetivo compreender que funcionalidades já se encontram disponíveis, identificar limitações das plataformas atuais e perceber de que forma o Vision4Farms Mobile poderia diferenciar-se.

Foi realizada uma análise de mercado a várias soluções digitais existentes na área da gestão agrícola. O objetivo desta análise foi compreender que funcionalidades já se

encontram disponíveis, identificar limitações das plataformas atuais e perceber de que forma o Vision4Farms Mobile poderia diferenciar-se.

Entre as soluções analisadas encontram-se AgriWebb, Cropio, FarmLogs, Agrivi, Bee2Crop e Wisecrop. Estas aplicações foram selecionadas por representarem diferentes abordagens à gestão agrícola digital, incluindo gestão de terrenos, monitorização por satélite, registo de atividades, gestão agronómica, rega inteligente, sensores e plataformas modulares (AgriWebb, 2026; Cropio, 2026; FarmLogs, 2026; Agrivi, 2026; Bee2Crop, 2026; Wisecrop, 2026).

A AgriWebb apresenta-se como uma solução orientada sobretudo para a gestão pecuária e de terrenos, disponibilizando funcionalidades como registo de atividades, relatórios e alguma gestão financeira. Apesar disso, revela limitações no que diz respeito à integração com sensores IoT e mostra-se menos adequada a contextos de agricultura de precisão centrados em culturas, rega e monitorização agrícola integrada.

A Cropio destaca-se pela monitorização via satélite e pela análise do estado da vegetação, recorrendo a mapas e indicadores relacionados com a saúde das culturas. No entanto, apresenta uma abordagem mais técnica, com maior complexidade para utilizadores menos especializados e custos potencialmente mais elevados, o que pode dificultar a adoção por pequenos agricultores ou utilizadores que procuram uma solução simples para o dia a dia.

A FarmLogs disponibiliza funcionalidades de registo de atividades, histórico de aplicações agrícolas e previsão meteorológica. Contudo, encontra-se muito orientada ao mercado norte-americano, o que pode limitar a sua adequação a outros contextos, nomeadamente ao nível das unidades utilizadas, práticas agrícolas e enquadramento operacional.

A Agrivi assume-se como uma plataforma de gestão agrícola bastante completa, incluindo planeamento de culturas, gestão financeira, inventário e relatórios. No entanto, essa abrangência funcional traduz-se também numa interface mais densa e

complexa, podendo tornar-se menos prática para uma utilização rápida em contexto de campo.

No contexto português, a Bee2Crop evidencia-se na área da rega e fertirrega inteligente, com integração de sensores e programação automática. Ainda assim, encontra-se fortemente centrada nessa dimensão específica, não cobrindo de forma alargada outras necessidades de gestão agrícola, como atividades, observações de campo, análises laboratoriais ou gestão operacional completa.

A Wisecrop, também portuguesa, apresenta uma oferta modular mais abrangente, incluindo clima, rega, fertilização, fitossanidade, atividades de campo e monitorização. No entanto, surge mais orientada a técnicos, agrónomos e organizações agrícolas, podendo apresentar uma experiência menos simples para utilização direta e frequente por parte do agricultor no terreno.

A análise destas soluções permitiu concluir que o mercado dispõe de várias ferramentas capazes de responder a necessidades específicas da gestão agrícola. Contudo, também se verificou que muitas destas plataformas se encontram especializadas numa área concreta ou apresentam interfaces e fluxos pouco centrados na utilização móvel simples e direta.

2.6 Comparação funcional entre soluções analisadas

A comparação funcional entre as soluções analisadas permitiu identificar as principais áreas cobertas por cada plataforma e perceber em que pontos o Vision4Farms Mobile poderia apresentar uma proposta diferenciadora (AgriWebb, 2026; Cropio, 2026; FarmLogs, 2026; Agrivi, 2026; Bee2Crop, 2026; Wisecrop, 2026).

De forma geral, verificou-se que várias plataformas disponibilizam funcionalidades de gestão de terrenos e parcelas, planeamento de atividades, relatórios e acompanhamento de culturas. No entanto, nem todas integram mapas interativos com polígonos, sensores IoT, rega inteligente, registo de anomalias, análises

laboratoriais, histórico de regas, *upload* de imagens e suporte multi-exploração de forma integrada.

A análise realizada evidenciou que algumas soluções se destacam numa área específica. A Cropio apresenta maior foco na monitorização por satélite e análise de vegetação. A Bee2Crop destaca-se pela rega e fertirrega inteligente. A Agrivi apresenta uma cobertura funcional mais ampla, mas com maior complexidade de utilização. A Wisecrop aproxima-se mais de uma solução completa, mas com orientação mais técnica e modular.

Por outro lado, o Vision4Farms Mobile foi pensado para combinar várias destas dimensões numa aplicação móvel centrada no contexto da exploração. Entre as funcionalidades consideradas relevantes encontram-se a gestão de explorações, terrenos e culturas, a visualização de terrenos em mapa, o registo de atividades, o acompanhamento de observações de campo, a consulta de regas, a gestão de notificações, a utilização de dados geográficos e a existência de mecanismos de suporte *offline*.

A comparação funcional permitiu também identificar uma lacuna importante: muitas soluções disponíveis não apresentam uma experiência móvel suficientemente autónoma ou simples para utilização direta no terreno. Em algumas plataformas, a aplicação móvel funciona como extensão da plataforma web, enquanto noutras a especialização funcional limita a cobertura das necessidades diárias da exploração.

Assim, a comparação entre soluções ajudou a reforçar a orientação do Vision4Farms Mobile como uma aplicação móvel modular, evolutiva e centrada na utilização prática em contexto agrícola.

Tabela 2.1 - Comparação funcional entre soluções de gestão agrícola

FUNCIONA LIDADE ÁREA	AGRI / WEBB	CRO PIO	FARML OGS	AGR IVI	BEE2C ROP	WISEC ROP	VISION4F ARMS MOBILE
GESTÃO DE TERRENOS E PARCELAS	Sim	Sim	Sim	Sim	Parcial	Sim	Sim
MAPA INTERATIV O COM POLÍGONO S	Parcial	Sim	Parcial	Parci al	Não	Sim	Sim
SENSORES IOT EM TEMPO REAL	Não	Não	Não	Não	Sim	Sim	Parcial
REGA E FERTIRREG A INTELIGEN TE	Não	Não	Parcial	Parci al	Sim	Sim	Sim
AGENDA DE ATIVIDADE S	Sim	Não	Sim	Sim	Não	Sim	Sim

REGISTO DE ANOMALIAS NO CAMPO	Parcial	Não	Parcial	Parcial	Não	Parcial	Sim
ANÁLISES DE SOLO E FOLIARES	Não	Parcial	Não	Sim	Não	Sim	Sim
HISTÓRICO DE REGAS	Não	Não	Sim	Sim	Sim	Sim	Sim
SUPORTE MULTI-EXPLORAÇÃO	Sim	Sim	Não	Sim	Sim	Sim	Sim
UPLOAD DE IMAGENS DE CAMPO	Parcial	Não	Sim	Parcial	Não	Parcial	Sim
APLICAÇÃO MÓVEL AUTÓNOMA PARA O AGRICULTOR	Sim	Parcial	Sim	Parcial	Parcial	Não	Sim
DESIGN UI/UX DEDICADO	Não	Não	Não	Não	Não	Não	Sim

Legenda:

Sim — funcionalidade presente

Não – funcionalidade ausente

Parcial - funcionalidade existente de forma limitada, complementar ou não central na solução.

2.7 Posicionamento do Vision4Farms Mobile

A análise do estado da arte e das soluções existentes permite concluir que o setor agrícola já dispõe de várias plataformas digitais capazes de apoiar diferentes dimensões da gestão agrícola. No entanto, também evidencia que estas soluções apresentam frequentemente limitações ao nível da integração funcional, da simplicidade de utilização em contexto móvel e da adequação a uma experiência centrada no utilizador no terreno (FAO, 2026; European Commission, 2025).

Algumas plataformas destacam-se pela monitorização por satélite, outras pela rega inteligente, outras pela gestão agronómica completa. Contudo, o estudo realizado aponta para a existência de espaço para uma solução que reúna, de forma coerente, gestão de explorações, terrenos, culturas, atividades, observações de campo, rega, notificações, mapas digitais e suporte *offline* numa única aplicação móvel.

O Vision4Farms Mobile posiciona-se precisamente nesse espaço. A solução procura funcionar como uma aplicação móvel de apoio à gestão agrícola, organizada por exploração e orientada para a consulta e registo de informação operacional. Através de módulos como *Dashboard*, Mapa, Terrenos, Culturas, Atividades, Observações, Rega e Notificações, a aplicação pretende centralizar dados relevantes e facilitar a utilização em contexto real de trabalho.

Um dos fatores diferenciadores do projeto é a integração entre informação funcional e informação geográfica. A representação de terrenos em mapa, com suporte a coordenadas e polígonos, permite aproximar a aplicação da realidade física da exploração. Esta abordagem torna o mapa não apenas um elemento visual, mas uma ferramenta útil para consultar terrenos, identificar áreas e relacionar informação operacional com o espaço agrícola.

Outro aspeto relevante é o suporte *offline* e a sincronização de dados. Considerando que a aplicação pode ser utilizada em zonas com conectividade limitada, a possibilidade de consultar dados em cache e registar operações pendentes representa uma vantagem importante para a utilização em campo.

Desta forma, o Vision4Farms Mobile procura ocupar um espaço intermédio entre plataformas excessivamente técnicas, soluções demasiado especializadas numa única área e aplicações móveis pouco autónomas face à componente web. O seu posicionamento assenta na ideia de uma solução móvel, modular e evolutiva, centrada na utilização prática no terreno e capaz de articular informação funcional, geográfica e operacional num único ambiente digital.

3 Metodologias, Tecnologias e Ferramentas Utilizadas

Neste capítulo são apresentadas as metodologias adotadas, bem como as tecnologias e ferramentas utilizadas no desenvolvimento do projeto Vision4Farms Mobile. As opções tomadas resultaram da análise dos requisitos funcionais, das necessidades específicas do projeto, das características da base de dados existente e da necessidade de desenvolver uma solução móvel integrada com uma API e preparada para utilização em contexto agrícola.

O desenvolvimento do projeto envolveu várias componentes técnicas, nomeadamente a aplicação móvel, a API/*backend*, a base de dados, a integração com mapas, os mecanismos de autenticação, o suporte *offline*, a sincronização de dados e a disponibilização da aplicação em formato APK para testes. Por esse motivo, a escolha das tecnologias teve em conta não só a produtividade, mas também a manutenção, escalabilidade e adequação ao contexto de utilização da solução.

3.1 Metodologia e organização do desenvolvimento

O desenvolvimento do projeto foi organizado de forma iterativa e incremental. Não foi adotada formalmente uma metodologia ágil específica, como *Scrum*, mas o trabalho seguiu princípios próximos de abordagens ágeis, nomeadamente a divisão do projeto em fases, a implementação progressiva de funcionalidades, a revisão frequente do trabalho desenvolvido e a realização de ajustes à medida que os módulos eram concluídos. Esta forma de organização aproxima-se de princípios iterativos e incrementais presentes em metodologias ágeis, nas quais o desenvolvimento evolui por ciclos sucessivos de planeamento, implementação, validação e adaptação (Schwaber & Sutherland, 2020).

A organização do projeto foi estruturada em fases, começando pela análise da documentação, estudo de soluções concorrentes, escolha das tecnologias e preparação do ambiente de desenvolvimento. Após esta fase inicial, o trabalho avançou para a adaptação da base de dados, desenvolvimento da API/*backend*, implementação da autenticação, construção da estrutura base da aplicação móvel e posterior desenvolvimento dos módulos principais, como *dashboard*, mapa, terrenos, culturas, atividades, observações, rega, agenda, notificações e suporte *offline*.

Esta abordagem permitiu uma evolução gradual do projeto, reduzindo o risco de acumulação de problemas técnicos. Cada etapa foi construída sobre a anterior, permitindo validar progressivamente a comunicação entre a aplicação móvel, a API e a base de dados. A divisão por fases revelou-se particularmente importante devido à dimensão do projeto, que não consistiu apenas numa aplicação móvel isolada, mas sim num sistema composto por *frontend mobile*, *backend*, base de dados, autenticação, sincronização e disponibilização para teste em ambiente externo.

O desenvolvimento foi acompanhado pelo orientador da empresa, permitindo validar decisões técnicas, ajustar prioridades e confirmar se as funcionalidades implementadas estavam alinhadas com os objetivos definidos para o projeto. Esta comunicação foi importante em várias decisões, como a organização dos módulos, o

METODOLOGIAS, TECNOLOGIAS E FERRAMENTAS UTILIZADAS

sistema de autenticação, a gestão de explorações, o funcionamento por *roles*, o sistema de convites e a preparação da aplicação para utilização fora do ambiente local.

Ao nível da interface, o desenvolvimento da aplicação teve como referência inicial um protótipo desenvolvido em Figma por um colega da área de Tecnologias e Design de Multimédia (TDM), no âmbito de um projeto relacionado. O Figma é uma ferramenta de desenho e prototipagem colaborativa, utilizada para criar interfaces, organizar fluxos visuais e apoiar o desenvolvimento de produtos digitais (Figma, 2026). Esse protótipo serviu como base visual e estrutural para alguns *layouts*, fluxos de navegação e organização dos ecrãs. No entanto, a implementação final exigiu a adaptação, recriação e complementação desses *layouts* em Flutter, tendo em conta os requisitos funcionais, os dados reais provenientes da API, os estados de carregamento, erro e ausência de dados, bem como o suporte *offline*.

A gestão do trabalho foi realizada de forma autónoma, com recurso a organização por módulos e validação progressiva. O controlo de versões foi efetuado com Git, sistema de controlo de versões distribuído que permite acompanhar alterações ao código, manter histórico do desenvolvimento e recuperar versões anteriores quando necessário (Git, 2026). A utilização desta ferramenta permitiu acompanhar a evolução do código, manter histórico das alterações e reduzir o risco de perda de trabalho durante o desenvolvimento.

3.2 Desenvolvimento da aplicação móvel

A aplicação móvel foi desenvolvida com Flutter, utilizando a linguagem Dart. O Flutter foi escolhido por permitir o desenvolvimento de aplicações móveis para Android e iOS a partir de uma única base de código, evitando a necessidade de criar projetos separados para cada plataforma (Flutter, 2026). Esta característica foi particularmente relevante, uma vez que permitiu concentrar o esforço de desenvolvimento numa solução multiplataforma, mantendo consistência visual e funcional entre os diferentes ambientes.

O Flutter utiliza uma abordagem baseada em *widgets*, o que permitiu construir a interface da aplicação através da composição de componentes reutilizáveis. Esta característica revelou-se importante no Vision4Farms Mobile, uma vez que a aplicação inclui vários elementos recorrentes, como *cards*, listas, formulários, botões, *badges*, filtros, *bottom sheets*, *banners* de estado e ecrãs de detalhe. A utilização de *widgets* facilitou a organização da interface e permitiu adaptar os layouts inspirados no Figma a uma aplicação funcional integrada com dados reais.

A linguagem Dart foi utilizada como linguagem principal da aplicação móvel. A sua integração direta com o Flutter e o suporte a *null safety* contribuíram para um desenvolvimento mais seguro, sobretudo numa aplicação que consome dados de uma API e que precisa de lidar com situações como respostas incompletas, falhas de ligação, estados de carregamento, erros e dados em cache.

Para a gestão de estado foi utilizado o *package* Provider, que permitiu separar a lógica da aplicação da interface gráfica. Esta separação tornou o código mais organizado e facilitou a manutenção dos diferentes módulos. Foram criados *providers* associados a áreas centrais da aplicação, como autenticação, exploração selecionada, conectividade, sincronização e dados funcionais. Desta forma, os ecrãs puderam reagir a alterações de estado sem concentrar toda a lógica diretamente nos *widgets*.

A navegação foi implementada com o *package* GoRouter, permitindo definir uma estrutura de rotas organizada e centralizada. Esta solução foi utilizada para controlar os fluxos principais da aplicação, como *splash screen*, *login*, seleção de exploração e navegação dentro da área principal. O GoRouter permitiu também aplicar redirecionamentos com base no estado do utilizador, garantindo que utilizadores não autenticados fossem encaminhados para o *login* e que utilizadores autenticados sem exploração selecionada fossem encaminhados para o ecrã adequado.

A comunicação com a API foi realizada com o *package* Dio, utilizado como cliente HTTP (Dio, 2026). Esta biblioteca permitiu configurar pedidos à API, tratar respostas em formato JSON, gerir erros e utilizar interceptors. No projeto, foi implementado um interceptor responsável por adicionar automaticamente o *token* JWT ao cabeçalho

dos pedidos e por tratar situações em que a autenticação precisava de ser renovada. Esta abordagem contribuiu para centralizar a comunicação com o *backend* e evitar repetição de lógica nos vários ecrãs da aplicação.

Para o armazenamento seguro de dados sensíveis, como *tokens* de autenticação, foi utilizado o *package* Flutter Secure Storage (Flutter Secure Storage, 2026). Esta tecnologia permite guardar informação em mecanismos seguros do dispositivo, evitando o armazenamento de credenciais em locais não protegidos. A sua utilização foi essencial para manter a sessão do utilizador de forma mais segura e permitir que a aplicação verificasse a existência de autenticação válida ao iniciar.

Uma das componentes relevantes da aplicação foi o suporte offline. Para responder ao contexto agrícola, onde a conectividade pode ser limitada ou instável, foram implementados mecanismos de cache, leitura *offline*, armazenamento local e fila de operações pendentes. A aplicação recorreu a uma base de dados local SQLite, permitindo guardar dados previamente obtidos da API e disponibilizá-los ao utilizador mesmo sem ligação à internet. Para além disso, foram criados mecanismos de sincronização, permitindo que determinadas operações realizadas *offline* ficassem registadas para posterior envio ao servidor quando a conectividade fosse restabelecida.

A aplicação incluiu ainda elementos visuais relacionados com o estado de conectividade, como um *banner offline*, permitindo informar o utilizador quando se encontra sem ligação e quando está a consultar dados locais. Esta funcionalidade é importante no contexto de utilização em campo, uma vez que evita que o utilizador interprete dados em cache como informação necessariamente atualizada em tempo real.

Para a integração com sensores IoT e notificações *push*, foram também utilizados serviços associados ao Firebase, nomeadamente Firebase Cloud Messaging e Cloud Firestore (Firebase, 2026a; Firebase, 2026c). A aplicação passou a recorrer a duas instâncias Firebase: uma instância principal, associada ao projeto Vision4Farms, utilizada para Firebase Cloud Messaging, e uma instância secundária, associada ao

projeto externo dos sensores, utilizada para leitura de dados em tempo real no Firestore. Esta separação permitiu manter as notificações sob controlo do projeto Vision4Farms, ao mesmo tempo que a aplicação consegue consultar leituras publicadas pelos sensores físicos numa infraestrutura externa.

3.3 Desenvolvimento da API e backend

A componente de servidor foi desenvolvida com Django e Django REST Framework. O Django foi escolhido por ser um *framework* web maduro, baseado em Python, que permite desenvolver aplicações de forma estruturada e produtiva. A sua organização por aplicações, o suporte a modelos, *views*, configuração centralizada e integração com bases de dados relacionais tornaram-no adequado ao desenvolvimento do *backend* do Vision4Farms Mobile (Django Software Foundation, 2026; Django REST Framework, 2026).

O Django REST Framework foi utilizado para construir a API RESTful responsável pela comunicação entre a aplicação móvel e o servidor. Esta API disponibiliza *endpoints* em formato JSON, permitindo que a aplicação Flutter consulte, crie, atualize e sincronize informação relacionada com utilizadores, explorações, terrenos, culturas, atividades, observações, rega, notificações, *dashboard* e outros módulos do sistema.

A API foi organizada de forma modular, através de várias aplicações Django independentes, cada uma associada a um domínio funcional. Esta separação permitiu manter o código mais organizado e facilitar a evolução do projeto. Entre os módulos desenvolvidos encontram-se áreas como autenticação, explorações, terrenos, culturas, atividades, *dashboard*, mapa, agenda, notificações e sincronização. Esta organização modular revelou-se importante para evitar a concentração de toda a lógica de negócio num único ficheiro ou aplicação.

Foram utilizados *serializers* para converter os dados dos modelos em estruturas JSON adequadas ao consumo pela aplicação móvel. As *views* foram responsáveis por processar os pedidos HTTP, aplicar validações, consultar a base de dados e devolver

respostas estruturadas à aplicação. Esta abordagem permitiu criar *endpoints* específicos para cada funcionalidade, como *login*, recuperação de palavra-passe, listagem de explorações, dados do *dashboard*, dados do mapa, terrenos, culturas, atividades, observações e notificações.

A autenticação foi implementada com JSON Web Token (JWT), permitindo um modelo de autenticação adequado a aplicações móveis. Após o *login*, o utilizador recebe *tokens* que são utilizados pela aplicação para autenticar pedidos seguintes. Este modelo evita a dependência de sessões tradicionais e permite que a aplicação comunique com o *backend* de forma *stateless*. Foi também implementada lógica para associar o utilizador autenticado ao respetivo perfil e às explorações a que tem acesso.

O *backend* incluiu ainda funcionalidades associadas à recuperação de palavra-passe por email. Neste fluxo, o utilizador solicita a recuperação, o servidor gera um *token* temporário, guarda a informação necessária na base de dados e envia um email com instruções para redefinição da palavra-passe. Este mecanismo permitiu criar um fluxo mais completo de autenticação e suporte ao utilizador.

Outro aspeto relevante foi o sistema de convites e controlo de acesso. O registo e o acesso a explorações foram pensados de forma controlada, permitindo que utilizadores fossem associados a explorações mediante convite ou autorização. Esta lógica revelou-se importante porque a aplicação trabalha com dados organizados por exploração agrícola, sendo necessário garantir que cada utilizador apenas acede às explorações às quais está associado.

No *backend*, foi também integrada a comunicação com Firebase Cloud Messaging através do Firebase Admin SDK (Firebase, 2026b; Firebase, 2026c). Esta integração permitiu enviar notificações *push* para os dispositivos dos utilizadores, com base nos *tokens* FCM registados no sistema. Foram ainda criados *endpoints* para registo e remoção de *tokens* dos dispositivos, permitindo associar cada dispositivo ao respetivo utilizador autenticado.

A API/*backend* foi, portanto, uma componente central do projeto, não apenas como camada de comunicação, mas também como responsável pela lógica de negócio, validação de dados, autenticação, permissões e ligação à base de dados.

3.4 Base de dados e modelação da informação

A base de dados utilizada no projeto foi MySQL, uma base de dados relacional adequada ao tipo de informação gerida pelo Vision4Farms Mobile (MySQL, 2026a). O projeto envolve várias entidades relacionadas entre si, como utilizadores, perfis, explorações, terrenos, culturas, variedades, atividades, observações, regas, agenda e notificações. Por esse motivo, uma base de dados relacional revelou-se adequada para representar as relações entre os diferentes elementos do sistema.

A gestão e consulta da base de dados durante o desenvolvimento foram realizadas com MySQL Workbench (MySQL, 2026b). Esta ferramenta permitiu visualizar tabelas, analisar relações, validar dados inseridos pela API e confirmar o comportamento das operações realizadas pela aplicação móvel. A utilização do MySQL Workbench foi importante durante a adaptação da base de dados, sobretudo para verificar se os dados estavam a ser corretamente persistidos e relacionados.

A base de dados existente foi adaptada ao contexto da aplicação móvel. Para além das entidades associadas a utilizadores, empresas, explorações, terrenos e culturas, foram consideradas funcionalidades específicas da *app*, como atividades, observações de campo, rega planeada, agenda e notificações. Esta adaptação permitiu aproximar o modelo de dados das necessidades reais da aplicação móvel e dos fluxos definidos para o utilizador.

No *backend* Django, foram utilizados modelos que representam as tabelas da base de dados. Em alguns casos, a estrutura da base de dados já existia ou foi definida externamente ao ciclo normal de migrações do Django, pelo que os modelos foram configurados de forma a respeitar a estrutura existente. Esta decisão permitiu utilizar o ORM do Django para consultar e manipular dados, mantendo compatibilidade com o esquema MySQL utilizado no projeto.

A modelação da informação teve como preocupação central manter os dados organizados por exploração. Esta organização é essencial no contexto do Vision4Farms Mobile, uma vez que o utilizador pode estar associado a diferentes explorações e cada exploração pode conter os seus próprios terrenos, culturas, atividades, observações, regas e notificações. Esta separação permite garantir que os dados apresentados na aplicação correspondem ao contexto selecionado pelo utilizador.

A estrutura relacional também facilitou a ligação entre módulos. Um terreno pode estar associado a uma exploração, uma cultura pode estar associada a um terreno, uma atividade pode estar associada a um terreno ou cultura, uma observação pode originar uma atividade, e uma notificação pode estar relacionada com uma operação específica. Estas relações permitiram construir uma aplicação em que a informação não surge isolada, mas integrada no contexto operacional da exploração agrícola.

Para o suporte *offline*, foi também utilizada uma base de dados local SQLite na aplicação móvel. Esta base local permitiu guardar dados em cache e manter uma fila de operações pendentes, contribuindo para que a aplicação pudesse funcionar em cenários de conectividade limitada. Assim, o projeto utilizou MySQL como base de dados principal no servidor e SQLite como apoio local na aplicação móvel (SQLite, 2026).

3.5 Mapas, georreferenciação e integração visual

O módulo de mapa foi desenvolvido com recurso ao Mapbox, uma plataforma de mapas e localização com SDK para Flutter (Mapbox, 2026). A escolha do Mapbox esteve relacionada com a importância do mapa no contexto do Vision4Farms Mobile, uma vez que a visualização geográfica dos terrenos agrícolas constitui uma funcionalidade central para aproximar a aplicação da realidade física da exploração.

O mapa permite representar os terrenos da exploração através de informação geográfica, nomeadamente coordenadas GPS e polígonos em formato GeoJSON. Esta representação possibilita ao utilizador visualizar parcelas agrícolas diretamente

sobre o mapa, facilitando a identificação espacial dos terrenos e a relação entre a informação operacional e a localização física.

A integração com Mapbox permitiu utilizar mapas base, incluindo visualização em modo satélite, e sobrepor informação específica da exploração. Os polígonos dos terrenos podem representar a forma aproximada das parcelas, tornando a visualização mais útil do que simples marcadores pontuais. Esta abordagem é especialmente relevante em contexto agrícola, onde os terrenos possuem formas e dimensões variáveis.

Para além da representação dos terrenos, o mapa foi pensado como uma ferramenta de consulta operacional. Foram considerados filtros, *badges* e *bottom sheets* para melhorar a interação com a informação. Os filtros permitem restringir a visualização de acordo com determinados critérios, os *badges* permitem indicar informação resumida associada aos terrenos, e os *bottom sheets* apresentam detalhes rápidos sem obrigar o utilizador a sair imediatamente do mapa.

A componente visual da aplicação teve como referência inicial o protótipo em Figma, mas a implementação em Flutter exigiu adaptação aos dados reais da aplicação e aos comportamentos funcionais dos módulos. No caso do mapa, isto significou transformar uma proposta visual em interação funcional, com carregamento de dados da API, representação geográfica, tratamento de estados e navegação para ecrãs de detalhe.

Desta forma, a utilização do Mapbox e da georreferenciação contribuiu para tornar o Vision4Farms Mobile mais adequado ao contexto agrícola, onde a localização dos terrenos, a sua forma e a associação entre espaço físico e informação operacional são elementos relevantes para a gestão diária.

3.6 Ferramentas de desenvolvimento, teste e disponibilização

O desenvolvimento do projeto recorreu a várias ferramentas de apoio, utilizadas na programação, teste, gestão da base de dados, controlo de versões e disponibilização da aplicação.

O Visual Studio Code foi utilizado como ambiente principal de desenvolvimento, tanto na componente Flutter como na componente Django (Visual Studio Code, 2026). Através das extensões adequadas, foi possível trabalhar com Dart, Flutter e Python no mesmo ambiente, facilitando a navegação pelo código e a edição dos diferentes módulos do projeto.

O Android Studio foi utilizado para configurar o ambiente de desenvolvimento *mobile*, gerir o SDK Android e executar a aplicação em emuladores (Android Developers, 2026). A validação da aplicação foi realizada em ambiente Android, incluindo testes através de emulador e preparação da aplicação em formato APK.

Para o teste manual dos *endpoints* da API foi utilizado o Thunder Client no Visual Studio Code (Thunder Client, 2026). Esta ferramenta permitiu enviar pedidos HTTP para a API, validar respostas JSON, testar autenticação, verificar *endpoints* e confirmar o comportamento do *backend* antes da integração ou validação na aplicação móvel. A utilização desta ferramenta foi importante para reduzir erros durante a comunicação entre a *app* e o servidor.

A gestão da base de dados foi realizada com MySQL Workbench. Esta ferramenta permitiu consultar tabelas, verificar dados, analisar relações e validar alterações resultantes das operações feitas pela API. O MySQL Workbench foi particularmente útil durante a adaptação da base de dados e durante os testes de integração entre *backend* e base de dados.

O Git foi utilizado para controlo de versões, permitindo manter histórico das alterações realizadas no código e acompanhar a evolução do projeto ao longo das várias fases de desenvolvimento (Git, 2026). Esta ferramenta contribuiu para maior segurança no processo de desenvolvimento, permitindo recuperar versões anteriores e organizar melhor o progresso do trabalho.

O Figma foi utilizado como referência visual para a implementação da interface (Figma, 2026). Embora o protótipo inicial tenha sido desenvolvido por um colega da área de TDM, este serviu como base para alguns layouts e fluxos da aplicação. Durante a implementação, esses layouts foram adaptados e complementados em Flutter, de acordo com as necessidades funcionais, os dados provenientes da API e os estados reais da aplicação.

O Railway foi utilizado para disponibilizar a API/*backend* fora do ambiente local. Esta disponibilização foi importante porque a aplicação em formato APK não podia depender de um servidor local executado no computador de desenvolvimento. Ao alojar a API no Railway, tornou-se possível testar a comunicação da aplicação com um *backend* acessível online, aproximando o ambiente de teste de um cenário real de utilização (Railway, 2026).

A disponibilização da API no Railway exigiu também a configuração de variáveis de ambiente e a adaptação das configurações do projeto para funcionamento fora do ambiente local. Esta etapa foi relevante para validar a aplicação em formato APK, garantindo que os pedidos realizados pela *app* eram enviados para uma API acessível externamente.

O Firebase foi utilizado em duas vertentes distintas: Firebase Cloud Messaging, para envio de notificações *push*, e Firebase Firestore, como fonte externa de dados em tempo real provenientes dos sensores IoT (Firebase, 2026a; Firebase, 2026c).

3.7 Justificação global das opções tecnológicas

As tecnologias escolhidas para o desenvolvimento do Vision4Farms Mobile foram selecionadas tendo em conta os requisitos do projeto, a duração do projeto, a necessidade de construir uma aplicação móvel funcional, a existência de uma base de dados relacional e a importância de disponibilizar a solução para testes fora do ambiente local.

A escolha do Flutter justificou-se pela possibilidade de desenvolver uma aplicação para Android e iOS com uma única base de código. Esta opção permitiu reduzir o

esforço de desenvolvimento e garantir consistência visual entre plataformas. Além disso, a arquitetura baseada em *widgets* revelou-se adequada à construção de uma interface composta por múltiplos ecrãs, *cards*, listas, formulários, filtros e componentes reutilizáveis.

A utilização de Provider, GoRouter, Dio e Flutter Secure Storage complementou a escolha do Flutter, permitindo estruturar a gestão de estado, a navegação, a comunicação com a API e o armazenamento seguro de *tokens*. Estas ferramentas contribuíram para uma aplicação mais organizada, com melhor separação de responsabilidades e maior facilidade de manutenção.

A escolha do Django e do Django REST Framework justificou-se pela necessidade de desenvolver uma API RESTful de forma estruturada e produtiva. O Django permitiu organizar o *backend* em módulos funcionais, enquanto o Django REST Framework facilitou a criação de *serializers*, *views* e *endpoints* em formato JSON. Esta combinação revelou-se adequada para suportar a comunicação entre a aplicação móvel e a base de dados.

A escolha do MySQL esteve relacionada com a natureza relacional dos dados do projeto. O Vision4Farms Mobile trabalha com entidades fortemente relacionadas entre si, como utilizadores, explorações, terrenos, culturas, atividades, observações, regas e notificações. Por esse motivo, uma base de dados relacional permitiu representar melhor as ligações entre os diferentes elementos do sistema. A utilização do MySQL Workbench facilitou ainda a administração e validação dos dados durante o desenvolvimento.

O Mapbox foi escolhido pela importância da componente geográfica no projeto. Como o mapa representa uma funcionalidade central da aplicação, era necessário utilizar uma tecnologia capaz de suportar visualização geográfica, mapas base, coordenadas e polígonos. Esta escolha permitiu representar terrenos agrícolas de forma mais próxima da realidade física da exploração.

O Railway foi utilizado para alojar a API e permitir que a aplicação em formato APK comunicasse com um *backend* acessível online. Esta opção foi importante para

ultrapassar a limitação do ambiente local, tornando possível testar a aplicação num contexto mais próximo de utilização real.

Por fim, a inclusão de mecanismos *offline* e de sincronização justificou-se pelo contexto agrícola da aplicação. Como a utilização no terreno pode ocorrer em zonas com ligação limitada ou instável, a existência de cache local, armazenamento de operações pendentes e sincronização posterior contribuiu para tornar a aplicação mais adequada ao seu contexto de utilização.

Em conjunto, estas tecnologias formaram uma *stack* coerente para o desenvolvimento do projeto, permitindo construir uma solução composta por aplicação móvel, API/*backend*, base de dados, mapas, autenticação, suporte *offline* e disponibilização externa para testes.

4 Análise e Arquitetura da Solução

Neste capítulo é apresentada a análise e arquitetura da solução desenvolvida no âmbito do projeto Vision4Farms Mobile, descrevendo os requisitos identificados, a organização dos sistemas, os mecanismos de comunicação, autenticação e suporte *offline*.

A solução é composta por três componentes principais: uma aplicação móvel em Flutter, uma API em Django REST Framework e uma base de dados MySQL. Para suporte *offline*, a aplicação dispõe ainda de uma base de dados local SQLite.

4.1 Visão geral da solução

O Vision4Farms Mobile é uma aplicação móvel de apoio à gestão agrícola, organizada em torno do conceito de exploração. Após autenticação, o utilizador seleciona uma exploração agrícola e passa a aceder aos módulos principais da aplicação, como *Dashboard*, Mapa, Atividades, Terrenos, Culturas e Mais.

A solução tem como objetivo centralizar informação relevante para a gestão diária de uma exploração agrícola. Entre os dados tratados encontram-se terrenos, culturas, atividades, observações de campo, regas, notificações, dados geográficos e informação associada ao utilizador. Esta organização permite que o agricultor, colaborador ou gestor consulte rapidamente o estado da exploração e registe informação diretamente no contexto em que está a trabalhar.

A aplicação móvel comunica com a API através de pedidos HTTP, recebendo e enviando dados em formato JSON. A API é responsável por validar pedidos, aplicar regras de negócio, garantir permissões de acesso e interagir com a base de dados

MySQL. Em paralelo, a aplicação móvel mantém dados locais para suportar cenários de utilização com ligação limitada ou inexistente.

Desta forma, a solução não foi concebida apenas como uma aplicação de consulta de dados, mas como um sistema integrado, composto por *frontend mobile*, *backend*, base de dados, autenticação, mapas, sincronização e suporte *offline*.

4.2 Requisitos funcionais

Os requisitos funcionais representam as funcionalidades que o sistema deve disponibilizar ao utilizador. No caso do Vision4Farms Mobile, estes requisitos foram definidos com base na documentação do projeto e nas necessidades associadas à gestão agrícola.

Entre os principais requisitos funcionais identificados destacam-se:

- autenticação de utilizadores;
- recuperação de palavra-passe;
- seleção de exploração agrícola;
- gestão de acesso por convite;
- visualização de *dashboard* com dados agregados;
- consulta de terrenos associados à exploração;
- visualização de terrenos em mapa;
- representação geográfica de terrenos através de coordenadas e polígonos;
- gestão de culturas;
- registo e consulta de atividades agrícolas;
- registo de observações de campo;
- consulta de informação de rega;
- gestão de agenda e notificações;
- suporte *offline* com cache e operações pendentes;

- sincronização de dados com o servidor quando a ligação é restabelecida;
- associação de sensores IoT a terrenos;
- consulta de leituras de sensores em tempo real;
- *fallback* meteorológico para terrenos sem sensores;
- receção de notificações *push*;
- deteção e notificação de alertas de pragas.

Estes requisitos refletem os módulos previstos para a aplicação, incluindo *Dashboard*, Mapa, Terrenos, Culturas, Atividades, Rega, Observações, Agenda e Notificações. Foram também considerados estados da interface como carregamento, erro, ausência de dados e ausência de ligação, importantes para garantir uma experiência de utilização mais robusta.

4.3 Requisitos não funcionais

Para além das funcionalidades principais, foram considerados requisitos não funcionais relacionados com segurança, usabilidade, desempenho, manutenção, portabilidade e disponibilidade da aplicação.

Ao nível da segurança, a solução deveria garantir que apenas utilizadores autenticados acediam aos dados da aplicação. Além disso, cada utilizador deveria visualizar apenas as explorações às quais estivesse associado, respeitando o papel definido no sistema.

Ao nível da usabilidade, a aplicação deveria apresentar uma navegação simples e adequada ao contexto móvel. Como a utilização pode ocorrer no terreno, a interface deveria privilegiar acesso rápido à informação, *cards* de resumo, filtros, listas claras e navegação por módulos.

Ao nível da manutenção, a solução foi organizada de forma modular, separando responsabilidades entre aplicação móvel, API/*backend* e base de dados. Esta separação facilita futuras alterações, correções e evolução da aplicação.

Também foi considerada a portabilidade da solução, uma vez que a aplicação móvel foi desenvolvida em Flutter, permitindo a sua evolução para Android e iOS a partir da mesma base de código. Esta característica reduz a duplicação de esforço no desenvolvimento e facilita a manutenção futura da aplicação em diferentes plataformas.

Por fim, foi considerado o funcionamento em contextos de conectividade limitada. Por se tratar de uma aplicação agrícola, a solução deveria permitir acesso a dados previamente carregados e registo de operações pendentes, reduzindo a dependência de ligação constante à internet.

4.4 Arquitetura geral do sistema

A arquitetura geral do sistema segue uma estrutura cliente-servidor, complementada por integrações externas necessárias ao funcionamento dos sensores IoT, das notificações *push* e do suporte *offline*. A aplicação móvel funciona como cliente, sendo responsável pela interface com o utilizador, navegação, apresentação de dados e registo de operações. A API/*backend* funciona como camada intermédia, processando pedidos, aplicando lógica de negócio e comunicando com a base de dados MySQL.

De forma simplificada, a arquitetura principal pode ser representada da seguinte forma:

Aplicação Flutter → API Django REST Framework → Base de dados MySQL

Para o suporte *offline*, existe ainda uma camada local na aplicação móvel:

Aplicação Flutter → Base de dados local SQLite → Sincronização posterior com API

Com a integração dos sensores IoT, a arquitetura passou também a incluir uma fonte externa de dados em tempo real. As leituras dos sensores são publicadas numa instância Firebase Firestore pertencente à equipa responsável pelo hardware dos sensores. A aplicação Vision4Farms consulta essa fonte externa para obter leituras como temperatura, humidade do ar, humidade do solo e pressão atmosférica. No

entanto, o *backend* da Vision4Farms mantém no MySQL o mapeamento entre cada nó sensor físico e o terreno onde se encontra instalado.

Assim, a componente de sensores pode ser representada da seguinte forma:

Aplicação Flutter → Firebase Firestore externo → Leituras dos sensores
Aplicação Flutter → API Django REST Framework → MySQL → Associações entre sensores e terrenos

Para terrenos sem sensores associados, a aplicação pode recorrer a dados meteorológicos externos como mecanismo de fallback, permitindo continuar a apresentar informação útil ao utilizador mesmo quando não existem leituras físicas disponíveis.

A arquitetura passou ainda a incluir Firebase Cloud Messaging para o envio de notificações *push*. Neste caso, a aplicação móvel regista o *token* FCM do dispositivo no *backend*, e o *backend* utiliza Firebase Cloud Messaging para enviar notificações *push* associadas a atividades, agenda, observações ou alertas de sensores.

Esta componente pode ser representada da seguinte forma:

Aplicação Flutter → API Django REST Framework → MySQL → Registo do *token* FCM
API Django REST Framework → Firebase Cloud Messaging → Dispositivo do utilizador

Esta arquitetura permite separar responsabilidades entre a aplicação móvel, a API, a base de dados principal, a base de dados local, os serviços Firebase e as fontes externas de dados. A aplicação móvel concentra-se na experiência do utilizador e na apresentação integrada da informação, a API centraliza a lógica de negócio e as permissões, o MySQL mantém a persistência relacional, o SQLite suporta a utilização *offline*, o Firebase Firestore externo fornece leituras em tempo real dos sensores e o Firebase Cloud Messaging permite a entrega de notificações *push*.

Desta forma, a solução mantém uma organização modular e evolutiva, permitindo integrar diferentes fontes de dados sem comprometer a separação entre interface, lógica de negócio, persistência e serviços externos.

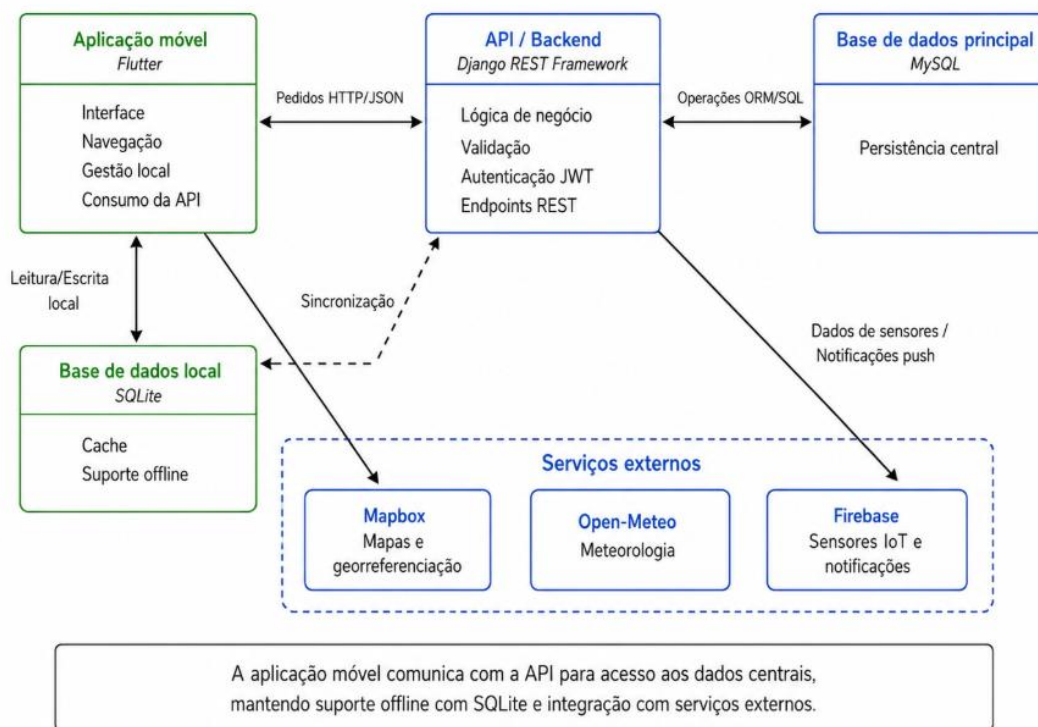


Figura 4.1 - Arquitetura geral do sistema

4.5 Arquitetura da aplicação móvel

A aplicação móvel foi desenvolvida em Flutter e organizada por módulos funcionais. A estrutura da aplicação foi pensada para acompanhar o fluxo principal do utilizador: autenticação, seleção de exploração e acesso à área principal da aplicação.

A navegação principal é baseada numa estrutura com separadores, permitindo aceder rapidamente aos módulos *Dashboard*, Mapa, Atividades, Terrenos, Culturas e Mais. Esta organização facilita a utilização da aplicação em contexto móvel e permite manter o utilizador dentro do contexto da exploração selecionada.

A gestão de estado foi organizada através de *providers*, permitindo separar a lógica da interface. Foram utilizados *providers* como *AuthProvider*, responsável pela autenticação e sessão do utilizador; *FarmProvider*, responsável pela exploração selecionada e dados associados; *ConnectivityProvider*, responsável pelo estado de ligação; e *SyncProvider*, associado ao processo de sincronização *offline*. Com a integração dos sensores, foi também utilizado o *SensorProvider*, responsável por

manter em memória as associações e leituras dos sensores por exploração e por terreno. Esta organização permitiu centralizar estados importantes e evitar que a lógica ficasse dispersa pelos ecrãs.

A aplicação recorreu ainda a serviços específicos para separar responsabilidades técnicas, como *ApiService*, responsável pela comunicação com a API; *LocalDatabase*, associado à persistência local; *SyncService*, responsável pela sincronização; e classes de apoio como *OfflineMutation* e *OfflineRead*, utilizadas para representar operações pendentes e leituras em contexto *offline*.

Com a evolução da aplicação, foram também acrescentados serviços específicos para sensores e notificações. O *FirebaseSensorService* ficou responsável por consultar os dados dos sensores no *Firebase Firestore* externo, enquanto o *SensorAssociationService* comunica com a API para gerir associações entre sensores e terrenos. O *SensorDataResolverService* agrega leituras por terreno e calcula médias, permitindo apresentar informação resumida no *dashboard*, nas listas e no detalhe dos terrenos.

Para o sistema de notificações *push*, foi criado o *FcmService*, responsável por pedir permissões ao utilizador, obter o *token* FCM do dispositivo, registá-lo no *backend* e tratar mensagens recebidas com a aplicação aberta, em segundo plano ou encerrada. Foi ainda implementado o *FirebaseAlertService*, responsável por subscrever alertas de sensores em tempo real no *Firestore* externo e filtrar apenas os alertas associados aos sensores da exploração selecionada.

Foi ainda considerada a apresentação de decisões associadas aos sensores, obtidas a partir da coleção *decisoes* da instância *Firebase Firestore* externa. Estas decisões são apresentadas na aplicação como recomendações de apoio à decisão, contendo informação sobre a anomalia detetada, o nó sensor associado, a ação sugerida, o resultado, o grau de confiança e os valores que originaram a recomendação.

A aplicação foi também preparada para lidar com diferentes estados de utilização, como carregamento, erro, ausência de dados e ausência de ligação. Esta preocupação foi importante para melhorar a experiência de utilização e tornar a *app* mais robusta

em contexto real, especialmente em cenários de utilização no terreno, com conectividade limitada e dados provenientes de diferentes fontes.

4.6 Arquitetura da API

A API foi desenvolvida com Django REST Framework e organizada por módulos funcionais. Cada módulo ficou responsável por um conjunto específico de *endpoints*, modelos, *serializers*, *views* e regras de negócio.

A API foi organizada em aplicações Django independentes, cada uma associada a um domínio funcional. Entre as principais *apps* encontram-se authentication, farms, lands, crops, activities, dashboard, map, yields_app, agenda, notifications_app, water e sensors. Esta divisão modular permitiu separar responsabilidades, facilitar a manutenção e tornar mais clara a evolução futura de cada componente.

A API disponibiliza *endpoints* para autenticação, gestão de utilizadores, seleção de explorações, dashboard, terrenos, culturas, atividades, mapa, agenda, notificações, rega, sensores e outros módulos. As respostas são enviadas em formato JSON, permitindo que a aplicação móvel trate os dados de forma simples e estruturada.

Com a integração dos sensores IoT, foi criada a *app sensors*, responsável pela gestão das associações entre nós sensores físicos e terrenos agrícolas. Esta *app* não armazena diretamente as leituras dos sensores, uma vez que esses dados são publicados numa instância Firebase Firestore externa. O *backend* guarda apenas o mapeamento entre cada sensor e o terreno onde se encontra instalado, através da tabela *sensor_node_associations*.

Foram criados *endpoints* autenticados para listar associações de sensores por exploração, criar ou atualizar associações em massa, alterar associações existentes e remover associações através de eliminação lógica. A leitura das associações é

permitida aos membros da exploração, enquanto as operações de criação, alteração e remoção ficam restritas a utilizadores com papel de Gestor.

A componente de notificações foi também expandida para suportar notificações *push* com Firebase Cloud Messaging. Para isso, foram criados endpoints que permitem à aplicação móvel registar e remover *tokens* FCM associados aos dispositivos dos utilizadores. Estes *tokens* são guardados no *backend* e utilizados para enviar notificações *push* quando são criadas novas notificações no sistema.

Foram utilizados *serializers* para converter os dados dos modelos em respostas adequadas ao consumo pela aplicação móvel. As *views* ficaram responsáveis por processar os pedidos HTTP, aplicar validações, consultar a base de dados, verificar permissões e devolver respostas estruturadas.

A camada de API é também responsável por validar o acesso aos dados. Isto é especialmente importante porque a informação está organizada por exploração, e um utilizador não deve aceder a dados de explorações às quais não está associado. Esta validação aplica-se também às associações de sensores, garantindo que os sensores apresentados e configurados pertencem ao contexto da exploração selecionada.

4.7 Modelo de dados

O modelo de dados foi estruturado em torno das principais entidades do domínio agrícola. Entre as entidades mais relevantes encontram-se utilizadores, perfis, empresas, explorações, terrenos, culturas, variedades, atividades, observações, regas, agenda, notificações, sensores e *tokens* de dispositivos.

A exploração agrícola funciona como contexto central da aplicação. A partir dela organizam-se terrenos, culturas, atividades, observações, regas, notificações e associações de sensores. Esta estrutura permite separar dados por exploração e apresentar ao utilizador apenas a informação relacionada com o contexto selecionado.

No âmbito do trabalho de projeto, foram criadas novas tabelas para suportar funcionalidades específicas da aplicação móvel, nomeadamente *observations*,

activities, *activities_images*, *agenda*, *notifications* e *water_irrigation_planned*. Estas tabelas permitiram suportar observações de campo, atividades agrícolas, imagens associadas, eventos de agenda, notificações e regas planeadas.

Com a evolução do projeto, foram ainda criadas as tabelas *sensor_node_associations* e *fcm_tokens*. A tabela *sensor_node_associations* guarda o mapeamento entre os nós sensores físicos e os terrenos agrícolas onde estes se encontram instalados. Esta tabela não armazena as leituras dos sensores, uma vez que esses dados são obtidos a partir de uma instância Firebase Firestore externa. A tabela *fcm_tokens* armazena os *tokens* Firebase Cloud Messaging associados aos dispositivos dos utilizadores, permitindo o envio de notificações *push*.

A estrutura das tabelas inclui campos de rastreabilidade, como *created_at*, *created_by*, *updated_at* e *updated_by*, permitindo acompanhar a criação e alteração dos registos. Foram também considerados campos de eliminação lógica, como *deleted_at* e *deleted_by*, permitindo marcar registos como removidos sem os eliminar fisicamente da base de dados.

A estrutura relacional facilita a ligação entre módulos. Um terreno pode estar associado a uma exploração, uma cultura pode estar associada a um terreno, uma atividade pode estar associada a um terreno ou cultura, uma observação pode originar uma atividade, uma notificação pode estar relacionada com uma operação específica e um sensor pode estar associado a um terreno. Estas relações permitem construir uma aplicação em que a informação não surge isolada, mas integrada no contexto operacional da exploração agrícola.

Para o suporte *offline*, foi também utilizada uma base de dados local SQLite na aplicação móvel. Esta base local permite guardar dados em cache e manter uma fila de operações pendentes, contribuindo para que a aplicação possa funcionar em cenários de conectividade limitada.

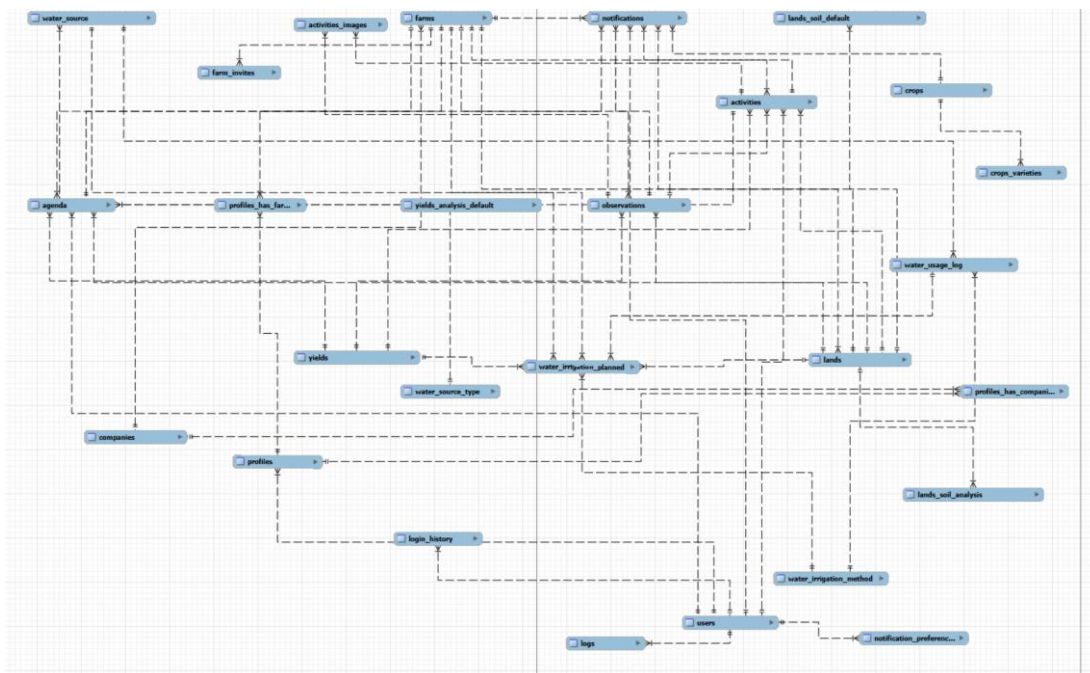


Figura 4.2 - Diagrama do modelo de dados

4.8 Comunicação entre aplicação móvel e backend

A comunicação entre a aplicação móvel e o *backend* é realizada através de pedidos HTTP para a API. A aplicação envia pedidos para *endpoints* específicos e recebe respostas em formato JSON.

O cliente HTTP utilizado na aplicação móvel centraliza a comunicação com o servidor, permitindo configurar cabeçalhos, tratar erros e adicionar automaticamente o *token* de autenticação aos pedidos. No caso de uma resposta 401, associada à expiração do *token*, o Dio tenta renovar o *token* de forma transparente e repetir o pedido original, evitando que o utilizador tenha de iniciar sessão novamente sempre que a renovação for bem-sucedida.

A estrutura de comunicação permite que módulos como *dashboard*, mapa, terrenos, culturas, atividades, notificações, rega e sensores obtenham dados do *backend* de forma organizada. Alguns *endpoints* foram pensados para reduzir o número de chamadas necessárias à API. O *dashboard*, por exemplo, agrega numa única resposta vários dados relevantes da exploração, como terrenos, atividades, culturas, regas, observações e notificações.

Com a integração dos sensores IoT, a comunicação da aplicação passou também a envolver uma fonte externa de dados em tempo real. As leituras dos sensores são obtidas diretamente a partir de uma instância Firebase Firestore externa, pertencente à equipa responsável pelo hardware dos sensores. A API/*backend* da Vision4Farms não armazena essas leituras, mantendo apenas o mapeamento entre sensores físicos e terrenos agrícolas.

Desta forma, a aplicação móvel comunica com o *backend* para obter dados relacionais, permissões, associações de sensores e notificações, e comunica com o Firebase Firestore externo para obter as leituras dos sensores em tempo real. Quando um terreno não possui sensores associados, a aplicação pode recorrer a dados meteorológicos externos como *fallback*, permitindo continuar a apresentar informação útil ao utilizador.

A comunicação com Firebase Cloud Messaging também faz parte da arquitetura. A aplicação móvel obtém o *token* FCM do dispositivo e envia-o para o *backend*, que o associa ao utilizador autenticado. Posteriormente, quando são criadas notificações no sistema, o *backend* utiliza esse *token* para enviar notificações *push* para o dispositivo do utilizador.

Esta comunicação é fundamental para manter a aplicação móvel integrada com a base de dados principal, com os serviços externos e com os mecanismos de notificação, garantindo que os dados apresentados correspondem à exploração selecionada, às permissões do utilizador e às fontes de informação disponíveis.

4.9 Segurança, autenticação e controlo de acesso

A segurança da solução assenta principalmente na autenticação por JSON Web Token (JWT) e no controlo de acesso por exploração. Após o *login*, o utilizador recebe *tokens* que permitem autenticar os pedidos seguintes à API.

Sempre que a aplicação realiza um pedido protegido, o *token* é enviado no cabeçalho da requisição. A API valida esse *token* e identifica o utilizador associado. Este

mecanismo permite proteger os endpoints e impedir o acesso a dados por utilizadores não autenticados.

Na aplicação móvel, os *tokens* são guardados com recurso ao Flutter Secure Storage, utilizando os mecanismos seguros do próprio dispositivo, nomeadamente *Keychain* em iOS e *Keystore* em Android. Esta abordagem evita que os *tokens* fiquem armazenados em locais não protegidos.

O controlo de acesso é particularmente importante porque a aplicação suporta múltiplas explorações. Um utilizador apenas deve visualizar e manipular dados das explorações a que está associado. Além disso, o sistema considera diferentes papéis, como gestor, colaborador ou consultor, permitindo ajustar permissões consoante o perfil do utilizador.

O sistema de convites também contribui para o controlo de acesso, uma vez que limita a entrada de novos utilizadores e a associação a explorações. Desta forma, o acesso à informação agrícola é condicionado por autenticação e autorização.

No caso dos sensores, o acesso às associações respeita igualmente o contexto da exploração. Todos os membros de uma exploração podem consultar os sensores associados, mas apenas utilizadores com papel de Gestor podem criar, alterar ou remover associações entre sensores físicos e terrenos. Esta regra evita que utilizadores sem permissões administrativas modifiquem a configuração dos sensores da exploração.

Relativamente às notificações *push*, os *tokens* FCM são registados no *backend* associados ao utilizador autenticado. Desta forma, o envio de notificações é direcionado apenas para os dispositivos pertencentes ao utilizador correto. Em caso de *logout*, o *token* pode ser removido, reduzindo o risco de envio de notificações para dispositivos que já não devem estar associados à sessão do utilizador.

Foi também implementado um mecanismo de verificação do estado da aplicação durante o arranque. Este mecanismo permite ao *backend* indicar se a aplicação deve apresentar um ecrã de manutenção ou exigir atualização obrigatória quando a versão

instalada deixa de ser suportada. Esta abordagem contribui para maior controle operacional da aplicação em ambientes de teste ou produção.

No fluxo de recuperação de palavra-passe, os *tokens* gerados têm validade limitada de duas horas e são invalidados após utilização bem-sucedida. Esta medida reduz o risco de reutilização indevida do *token* e reforça a segurança do processo de redefinição de palavra-passe.

4.10 Suporte offline e sincronização de dados

O suporte *offline* foi considerado uma componente importante da arquitetura, devido ao contexto de utilização da aplicação. Em ambiente agrícola, a ligação à internet pode ser instável ou inexistente, especialmente quando a aplicação é usada diretamente no terreno.

Para responder a esse problema, a aplicação móvel utiliza armazenamento local para guardar dados previamente obtidos da API. Assim, mesmo sem ligação, o utilizador pode consultar informação já carregada, como dados de explorações, terrenos, culturas, atividades, notificações e, quando aplicável, dados relacionados com sensores já consultados.

Além da leitura *offline*, foi considerada uma fila de operações pendentes. Quando o utilizador realiza determinadas ações sem ligação, essas operações são registadas localmente para sincronização posterior. Quando a conectividade é restabelecida, a aplicação tenta enviar essas operações para a API, atualizando a base de dados principal.

As operações realizadas em modo *offline* são registadas localmente com identificadores únicos, recorrendo a UUID para garantir idempotência e evitar duplicações durante a sincronização. Quando a ligação é restabelecida, a aplicação tenta reenviar essas operações para a API.

O processo de sincronização considera também mecanismos de tentativa e erro. Cada operação pendente pode ser repetida até três tentativas. Erros de rede ou respostas 5xx são tratados como falhas temporárias, incrementando o número de tentativas.

Por outro lado, erros 4xx são interpretados como falhas definitivas, uma vez que normalmente indicam problemas de validação, permissões ou dados inválidos. Esta distinção permite tornar a sincronização mais controlada e previsível.

A aplicação inclui também indicação visual do estado *offline*, permitindo informar o utilizador quando está a consultar dados locais. Esta transparência é importante para evitar confusão entre dados atualizados em tempo real e dados guardados em cache.

No caso dos sensores IoT, as leituras em tempo real dependem da disponibilidade da ligação à instância Firebase Firestore externa. Quando não existem sensores associados ou quando não é possível obter leituras, a aplicação pode recorrer a dados meteorológicos externos como *fallback*. Esta abordagem permite manter a apresentação de informação útil ao utilizador, mesmo quando não existem dados físicos disponíveis para determinado terreno.

Desta forma, o suporte *offline* e a sincronização tornam a solução mais adequada ao contexto agrícola, reduzindo a dependência de conectividade constante e permitindo maior continuidade de utilização no terreno.

5 Atividades Desenvolvidas

Neste capítulo são descritas as atividades desenvolvidas ao longo do projeto, organizadas de acordo com as fases de trabalho definidas no plano inicial. A apresentação por fases permite compreender a evolução progressiva do projeto, desde a análise e preparação até à implementação dos módulos principais, suporte *offline*, testes e preparação da aplicação em formato APK.

5.1 Fase 1 — Análise e preparação

A primeira fase foi dedicada à análise do projeto, à compreensão dos requisitos e à preparação do ambiente de desenvolvimento. Nesta etapa foi analisada a documentação disponibilizada pela empresa, incluindo os requisitos funcionais, a estrutura da base de dados, a documentação técnica existente e as referências visuais da aplicação móvel.

Foi também realizada uma análise de mercado a soluções digitais de gestão agrícola, com o objetivo de compreender funcionalidades existentes, identificar limitações e perceber oportunidades de diferenciação para o Vision4Farms Mobile. Entre as soluções analisadas encontram-se plataformas como AgriWebb, Cropio, FarmLogs, Agrivi, Bee2Crop e Wisecrop.

Paralelamente, foram estudadas e selecionadas as tecnologias a utilizar no projeto. Para a aplicação móvel foi escolhido Flutter, devido à sua capacidade de desenvolvimento multiplataforma. Para a componente de servidor foi utilizado Django REST Framework, pela sua adequação à construção de APIs REST. A base de dados principal foi mantida em MySQL, tendo sido posteriormente adaptada às necessidades da aplicação móvel.

Nesta fase foram ainda configurados os ambientes de desenvolvimento, incluindo Visual Studio Code, Android Studio, Flutter SDK, ambiente Python/Django e ligação à base de dados MySQL. Esta preparação foi essencial para garantir uma base técnica estável antes do início da implementação.

5.2 Fase 2 — Base de dados, backend e autenticação

A segunda fase centrou-se na adaptação da base de dados, no desenvolvimento inicial da API/*backend* e na implementação dos mecanismos de autenticação.

Ao nível da base de dados, foi analisada a estrutura existente e foram identificadas as necessidades de adaptação para suportar os módulos previstos para a aplicação móvel. No âmbito do trabalho de projeto, foram criadas seis novas tabelas principais: *observations*, *activities*, *activities_images*, *agenda*, *notifications* e *water_irrigation_planned*. Estas tabelas permitiram suportar funcionalidades como

observações de campo, atividades agrícolas, imagens associadas, eventos de agenda, notificações e regas planeadas.

Com a evolução do projeto, foram também criadas tabelas adicionais para suportar a integração com sensores IoT e notificações *push*. A tabela *sensor_node_associations* foi criada para guardar o mapeamento entre nós sensores físicos e terrenos agrícolas. Já a tabela *fcm_tokens* foi criada para armazenar os *tokens* Firebase Cloud Messaging associados aos dispositivos dos utilizadores, permitindo o envio de notificações *push*.

No *backend*, os modelos Django foram configurados com *managed=False*, uma vez que a estrutura da base de dados foi definida diretamente em MySQL e não gerida automaticamente pelas migrações do Django. Esta decisão permitiu utilizar o ORM do Django para interagir com as tabelas, mantendo compatibilidade com a estrutura existente da base de dados.

Foi criado e organizado o projeto Django com Django REST Framework. A API foi estruturada por módulos funcionais, permitindo separar responsabilidades e facilitar a manutenção. Foram desenvolvidos *endpoints* iniciais para autenticação, consulta de utilizador autenticado, gestão de explorações e comunicação com a aplicação móvel.

O sistema de autenticação foi implementado com recurso a JSON Web Token (JWT). Este mecanismo permite que, após o *login*, a aplicação móvel receba tokens de autenticação e os utilize nos pedidos seguintes à API. A verificação da palavra-passe foi realizada através do algoritmo *pbkdf2_sha256*, permitindo comparar a palavra-passe introduzida pelo utilizador com o *hash* armazenado na base de dados.

Também foi desenvolvido o fluxo de recuperação de palavra-passe por email. Para este processo, foi utilizada a função *secrets.token_urlsafe*, responsável por gerar *tokens* seguros e temporários para validação do pedido de recuperação. O *token* gerado é associado ao utilizador e possui validade limitada, permitindo posteriormente a redefinição da palavra-passe.

Nesta fase foi ainda trabalhado o sistema de convites e controlo de acesso. O objetivo foi garantir que o acesso à aplicação e às explorações agrícolas fosse controlado,

permitindo associar utilizadores a explorações específicas e definir permissões de acordo com o papel atribuído.

O sistema de convites foi desenvolvido de forma a permitir que utilizadores com permissões adequadas pudessem gerar códigos de acesso associados a diferentes perfis, nomeadamente Colaborador, Consultor ou Gestor. Este mecanismo permitiu controlar a entrada de novos utilizadores nas explorações, garantindo que cada pessoa fica associada a uma exploração concreta e com um papel definido. Esta funcionalidade contribuiu para reforçar o controlo de acesso e a organização da informação por exploração.

Posteriormente, a API foi expandida com a *app sensors*, responsável pela gestão das associações entre sensores físicos e terrenos. Foram criados *endpoints* autenticados para listar associações por exploração, criar ou atualizar associações em massa, alterar associações existentes e remover associações através de eliminação lógica. A leitura das associações ficou disponível para os membros da exploração, enquanto as operações de escrita ficaram restritas a utilizadores com papel de Gestor.

A componente de notificações também foi expandida com integração do Firebase Admin SDK, permitindo ao *backend* enviar notificações *push* através do Firebase Cloud Messaging. Para isso, foram criados *endpoints* para registo e remoção de *tokens* FCM dos dispositivos, permitindo associar cada dispositivo ao utilizador autenticado. Esta integração preparou o sistema para enviar notificações automáticas associadas a atividades, observações, agenda e alertas provenientes dos sensores.

5.3 Fase 3 — Estrutura base da aplicação móvel

A terceira fase foi dedicada à construção da estrutura base da aplicação móvel em Flutter. O trabalho começou pela criação da organização principal do projeto, definição de rotas, estrutura de ecrãs e implementação dos primeiros fluxos funcionais.

Foram desenvolvidos os ecrãs associados à autenticação, incluindo *login*, recuperação de palavra-passe e verificação do estado de sessão. A aplicação passou a

comunicar com a API através de um serviço centralizado, responsável por enviar pedidos HTTP, tratar respostas e incluir automaticamente o *token* JWT nos pedidos autenticados.

No fluxo de recuperação de palavra-passe, foi também configurado o suporte a *deep links*. O email enviado ao utilizador contém um link que redireciona para a aplicação móvel, passando o *token* de recuperação como parâmetro. Desta forma, o utilizador consegue abrir diretamente o ecrã de redefinição de palavra-passe na *app*, com o *token* necessário ao processo.

Foi também implementada a lógica de seleção de exploração. Após autenticação, o utilizador acede à lista de explorações às quais está associado e seleciona o contexto em que pretende trabalhar. Esta escolha passa a condicionar os dados apresentados nos restantes módulos da aplicação.

A navegação principal foi organizada com uma estrutura de separadores, permitindo acesso às áreas *Dashboard*, Mapa, Atividades, Terrenos, Culturas e Mais. Esta estrutura permitiu preparar a aplicação para o desenvolvimento progressivo dos módulos funcionais seguintes.

O fluxo inicial da aplicação foi organizado de forma a conduzir o utilizador desde a autenticação até à seleção da exploração agrícola. A partir do ecrã de *login*, o utilizador pode iniciar sessão, criar conta ou recuperar a palavra-passe. Após a autenticação, é apresentada a listagem de explorações associadas ao utilizador, permitindo selecionar uma exploração existente, criar uma exploração ou juntar-se a uma exploração através de um código de convite, tal como demonstra a Figura 5.1.

Esta lógica foi importante porque toda a aplicação funciona em torno da exploração selecionada. Assim, os dados apresentados nos módulos seguintes, como *dashboard*, mapa, terrenos, culturas, atividades, rega e notificações, passam a estar condicionados pelo contexto agrícola escolhido pelo utilizador presente na Figura 5.1.

No *AuthProvider*, foi utilizada a *flag isNewRegistration* para distinguir situações de registo recente de situações de *login* normal. Esta distinção foi importante no

redirecionamento central do *GoRouter*, permitindo encaminhar o utilizador para o fluxo correto consoante o estado da sessão e o contexto de entrada na aplicação.

Nesta fase foram ainda implementados *providers* e serviços essenciais, nomeadamente para autenticação, gestão da exploração selecionada, comunicação com a API e armazenamento seguro de *tokens*. A estrutura criada nesta etapa serviu como base para a integração dos módulos funcionais desenvolvidos nas fases seguintes.

Foi também implementado o ecrã de perfil, permitindo ao utilizador consultar e editar os seus dados pessoais, alterar a palavra-passe, configurar preferências de notificações, gerir convites da exploração, quando tem permissões para tal, e terminar sessão. Esta área permitiu concentrar funcionalidades de gestão pessoal e administrativa sem sobrecarregar os módulos principais da aplicação.



Figura 5.1 - Fluxo inicial da aplicação móvel

5.4 Fase 4 — Dashboard e mapa interativo

Na quarta fase foram desenvolvidos dois módulos centrais da aplicação: o *dashboard* e o mapa interativo.

O *dashboard* foi concebido como ecrã principal após a seleção de uma exploração. O objetivo deste módulo foi apresentar ao utilizador uma visão resumida do estado da exploração, reunindo informação relevante numa única área. Para isso, foi desenvolvido um *endpoint* no *backend* capaz de agregar, numa única resposta, vários dados relevantes da exploração, nomeadamente terrenos, área total, culturas ativas, atividades pendentes e atrasadas, próximas regas, observações recentes e notificações não lidas. Esta abordagem reduziu a necessidade de múltiplas chamadas à API e tornou o carregamento do *dashboard* mais eficiente.

Com a evolução do projeto, o *dashboard* foi também enriquecido com informação associada a sensores IoT e dados meteorológicos. Quando existem sensores associados a terrenos da exploração, a aplicação apresenta médias agregadas por terreno, permitindo consultar valores como temperatura, humidade do ar e humidade do solo. Quando não existem sensores associados ou não estão disponíveis leituras recentes, a aplicação pode recorrer à API Open-Meteo como mecanismo de *fallback*, garantindo que o utilizador continua a ter informação meteorológica útil.

Na aplicação móvel, estes dados foram apresentados através de *cards* e listas resumidas, permitindo ao utilizador consultar rapidamente os principais indicadores da exploração. Este módulo contribuiu para melhorar a usabilidade da aplicação, funcionando como ponto de entrada para os restantes módulos.

O *dashboard* foi organizado em cartões interativos, permitindo apresentar informação resumida e, ao mesmo tempo, servir como ponto de acesso rápido a módulos específicos. A parte superior do ecrã apresenta informação como área cultivada, número de terrenos, previsão meteorológica e dados associados a sensores.

A parte inferior reúne informação operacional, como atividades pendentes e atrasadas, eventos de agenda, regas, análises e outros indicadores relevantes para o acompanhamento da exploração.

Foi ainda implementado um botão de ação flutuante, permitindo criar rapidamente novos elementos a partir da área principal da aplicação. Através deste menu, o utilizador pode aceder à criação de eventos, observações, atividades, terrenos, culturas ou explorações, reduzindo o número de passos necessários para iniciar uma nova operação, conforme ilustrado na Figura 5.2.

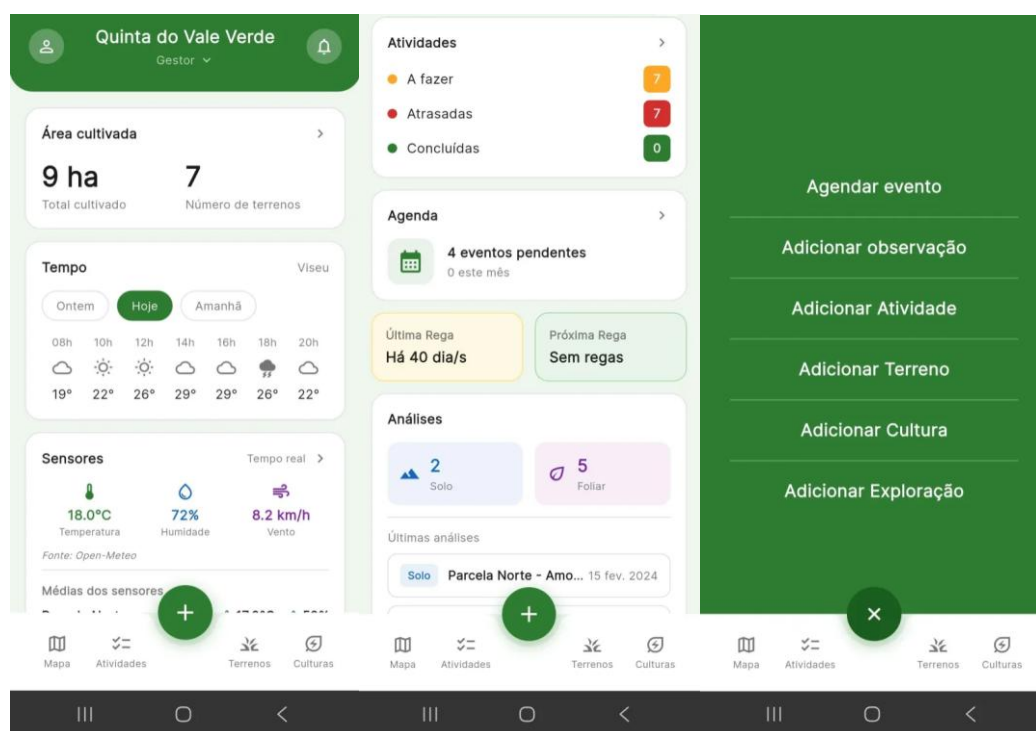


Figura 5.2 - Dashboard e menu de ações

O módulo de mapa foi desenvolvido com recurso ao Mapbox. Esta funcionalidade permite visualizar terrenos agrícolas sobre uma base cartográfica, com suporte a coordenadas GPS e polígonos. O mapa foi pensado para facilitar a identificação visual das parcelas e relacionar a informação operacional com a localização física dos terrenos.

Foram também implementados elementos de interação no mapa, como filtros, marcadores, *badges* de informação e *bottom sheets* com dados resumidos do terreno

selecionado. Ao tocar num terreno, é apresentado um *bottom sheet* com uma pré-visualização, incluindo informação resumida e opção de navegação para o detalhe. Foi também prevista a possibilidade de criar uma atividade diretamente associada ao terreno selecionado, tornando o mapa uma ferramenta operacional e não apenas visual.

O mapa passou, assim, a funcionar como uma área de consulta e interação com os terrenos da exploração. A representação dos polígonos permite identificar visualmente as parcelas, enquanto a legenda de estado ajuda a distinguir terrenos normais de terrenos com atividades em atraso. A utilização de *bottom sheets* permite apresentar informação contextual sem obrigar o utilizador a abandonar imediatamente o mapa, mantendo uma navegação mais fluida, presente na Figura 5.3.

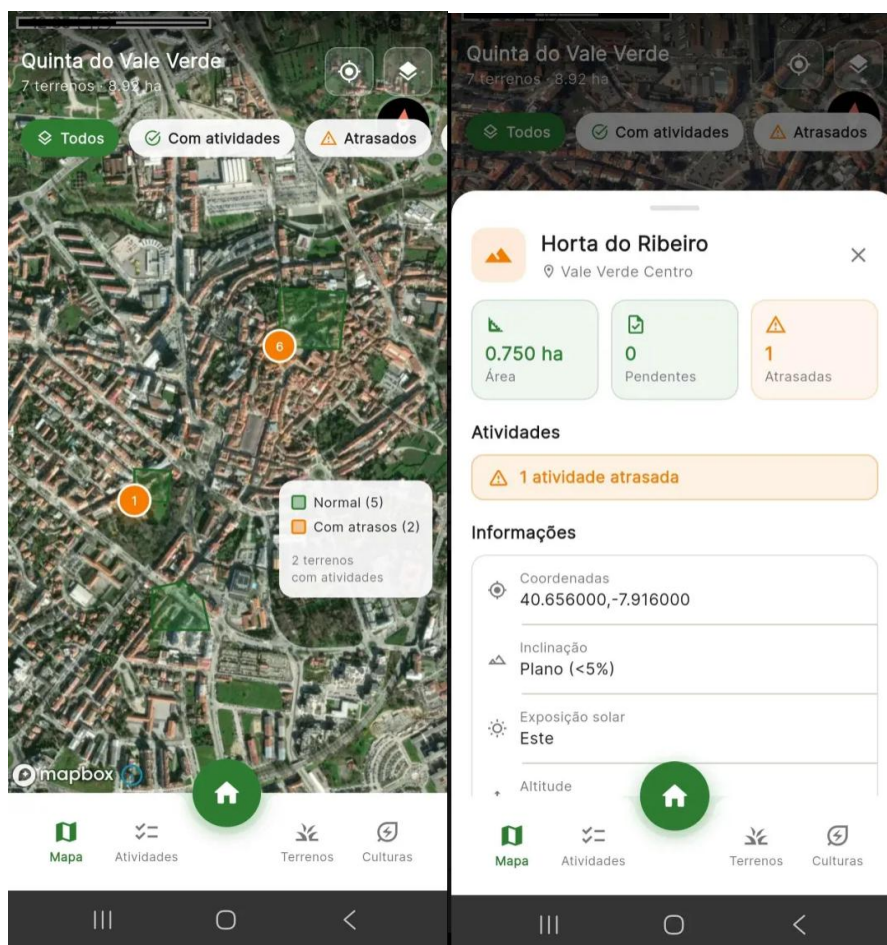


Figura 5.3 - Mapa interativo e pré-visualização de terreno

5.5 Fase 5 — Terrenos, culturas, atividades e observações

A quinta fase concentrou-se no desenvolvimento dos módulos de terrenos, culturas, atividades e observações de campo.

No módulo de terrenos, foi implementada a listagem dos terrenos associados à exploração selecionada e o respetivo ecrã de detalhe. O detalhe do terreno foi desenvolvido como um *hub* central da aplicação, organizado com *tabs* internas. A partir deste ecrã, o utilizador pode consultar diferentes dimensões associadas ao terreno, como informação geral, culturas, atividades, observações, regas, análises e dados de monitorização. Esta organização facilita a navegação e concentra a informação operacional do terreno num único ponto.

A criação de terrenos foi implementada através de um formulário com os dados principais do terreno, incluindo nome, localização e coordenadas GPS. Para além do preenchimento manual de informação, foi também integrada a possibilidade de desenhar o polígono do terreno diretamente sobre o mapa. Neste fluxo, o utilizador marca os vértices da parcela sobre a imagem de satélite, sendo possível visualizar a forma do terreno e calcular a respetiva área.

Esta funcionalidade tornou o módulo de terrenos mais completo, uma vez que permite associar informação textual e geográfica ao mesmo registo. A representação por polígonos é particularmente relevante no contexto agrícola, dado que os terrenos não correspondem apenas a pontos no mapa, mas sim a áreas físicas com formas e dimensões próprias, a Figura 5.4 corrobora o explicado.

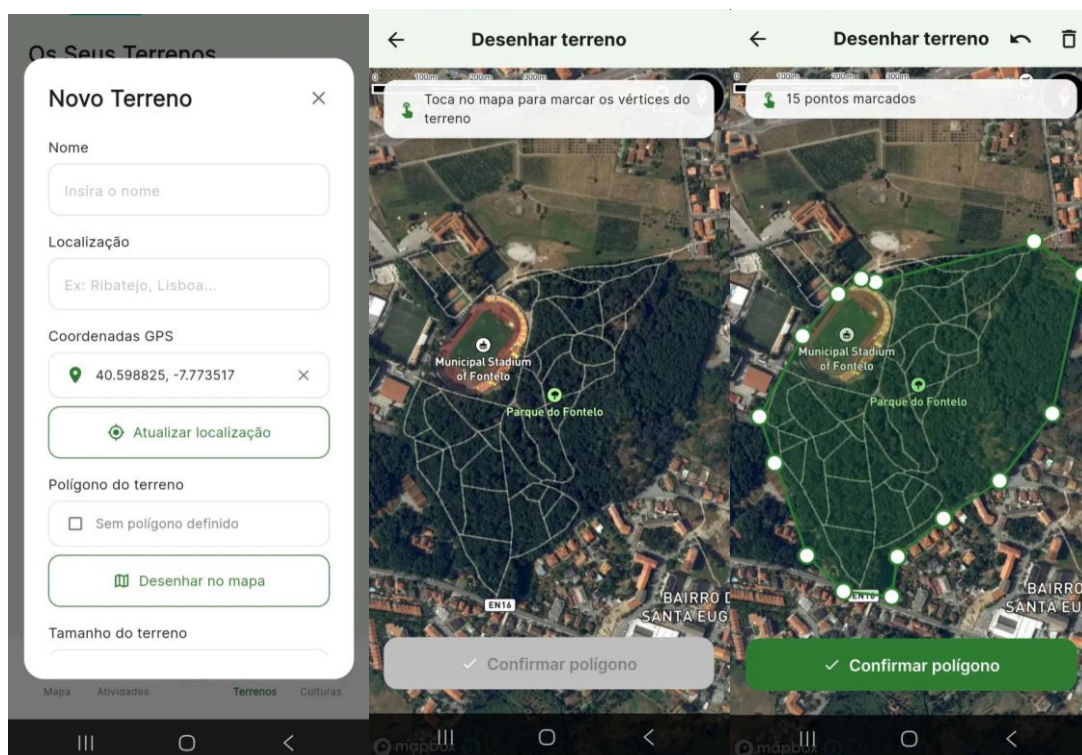


Figura 5.4 - Criação de terreno e desenho de polígono

Com a integração dos sensores IoT, o detalhe do terreno passou também a apresentar informação relativa ao estado atual do terreno, com base nas últimas leituras disponíveis. Entre os dados apresentados encontram-se temperatura, humidade do ar, humidade do solo e pressão atmosférica. Para além do estado atual, foi incluída uma componente de histórico, com gráficos separados por métrica, permitindo acompanhar a evolução dos valores ao longo do tempo.

A organização do detalhe do terreno por separadores internos permitiu concentrar num único ecrã diferentes dimensões da informação agrícola. Desta forma, o utilizador pode alternar entre a visão geral do terreno, os dados dos sensores, a informação de rega, as atividades, as observações e as análises associadas. Esta abordagem evita a dispersão da informação por múltiplos ecrãs independentes e facilita a consulta contextual dos dados presente na Figura 5.5.

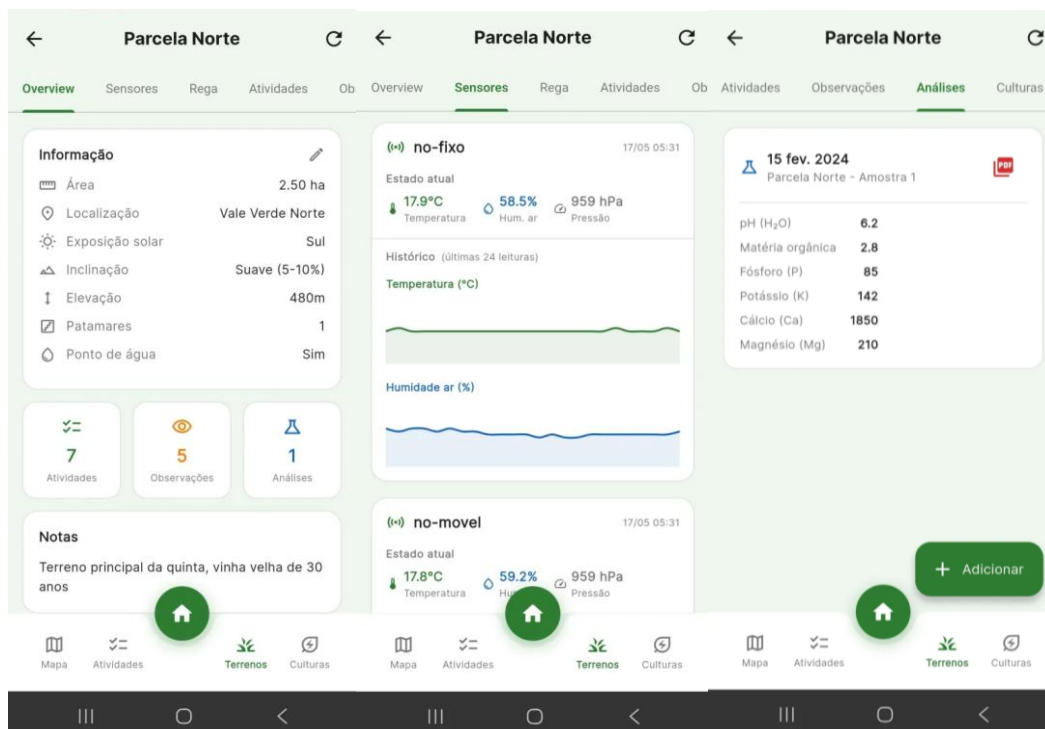


Figura 5.5 - Detalhe do terreno organizado por separadores

No módulo de culturas, foi desenvolvida a listagem de culturas associadas à exploração e aos terrenos. Este módulo permite organizar a informação agrícola por cultura, variedade e terreno associado. A integração com a API permitiu obter dados reais da base de dados e apresentá-los de forma estruturada na aplicação móvel.

A listagem de culturas foi estruturada de forma a apresentar informação resumida sobre cada cultura, incluindo dados como área associada, terrenos, atividades, observações e regas. O detalhe da cultura permite consultar informação agronómica mais completa, como tipo de cultura, ciclo produtivo, épocas de plantação e colheita, necessidades hídricas e variedades associadas.

No módulo de culturas foi também implementado o registo de colheitas associadas a cada cultura. Para além do separador de colheitas presente no detalhe da cultura, que apresenta um resumo das colheitas registadas, foi desenvolvido um ecrã dedicado à consulta e criação de colheitas por cultura.

Cada registo de colheita inclui informação como a data, a quantidade produzida, a unidade de medida e custos associados, nomeadamente custo geral, custo de mão-

de-obra e custo de máquinas. Estes dados são enviados para a API e persistidos na tabela `yields_harvests`, ficando associados à cultura e ao terreno correspondentes.

A gestão das colheitas segue o mesmo modelo das restantes operações da aplicação, suportando listagem, consulta de detalhe, criação, atualização e eliminação lógica. À semelhança das análises, é também possível gerar um documento PDF com o registo de uma colheita. Esta funcionalidade complementa o módulo de culturas, permitindo acompanhar não apenas o estado agronómico da cultura, mas também os resultados produtivos obtidos.

Foi ainda implementada a geração de documentos PDF associados às análises de solo, às análises foliares e aos registos de colheita. Estes documentos são gerados no *backend* com recurso à biblioteca ReportLab, em formato A4 e com uma identidade visual alinhada com a aplicação, nomeadamente através da utilização da cor verde institucional.

Durante a geração dos documentos, foi necessário aplicar tratamento aos dados para garantir a correta apresentação no PDF. Alguns caracteres Unicode sem representação direta na fonte utilizada, como índices subscritos, foram convertidos para a sua forma ASCII equivalente. Adicionalmente, os textos foram escapados para evitar problemas de formatação durante a geração do documento. Este tratamento foi introduzido após erros identificados nos testes de geração dos PDFs.

Do lado da aplicação móvel, os documentos são descarregados de forma autenticada, sendo o *token* JWT enviado no cabeçalho do pedido. O conteúdo recebido em formato de *bytes* é guardado localmente e aberto através do *package OpenFilex*, recorrendo ao mecanismo *FileProvider* do Android. Esta abordagem foi adotada para ultrapassar as restrições introduzidas a partir do Android 7, que impedem a abertura direta de ficheiros através de URI do tipo `file://`.

A possibilidade de gerar e abrir PDFs torna a aplicação mais completa, uma vez que permite ao utilizador consultar ou partilhar documentos associados a análises e colheitas fora do ecrã da aplicação. A existência desta funcionalidade é visível, por

exemplo, na Figura D.20, onde é apresentado o acesso ao documento PDF associado a uma análise de solo.

O módulo de atividades permitiu registrar e consultar tarefas agrícolas, como tratamentos, fertilizações, podas, regas, colheitas, inspeções ou outras operações. As atividades foram estruturadas com tipos concretos, como *treatment*, *fertilization*, *pruning*, *irrigation*, *harvest*, *inspection* e *other*. Esta classificação permite organizar melhor as tarefas agrícolas e facilita a filtragem e interpretação das atividades associadas a cada exploração, terreno ou cultura.

As atividades incluem informação como tipo, estado, prioridade, data prevista e associação a terreno ou cultura. Esta funcionalidade é importante para apoiar o planeamento e acompanhamento do trabalho diário na exploração.

A listagem de atividades foi desenvolvida com mecanismos de filtragem e indicadores visuais de prioridade, permitindo ao utilizador identificar rapidamente tarefas pendentes, concluídas ou em atraso. Esta funcionalidade torna-se relevante no contexto da exploração agrícola, onde diferentes operações podem estar associadas a terrenos, culturas e datas específicas, corroborado na Figura .

Foi também desenvolvido o módulo de observações de campo, permitindo registrar ocorrências diretamente no terreno. Para além do texto descritivo e da associação a uma exploração, terreno ou cultura, foram considerados campos específicos como estado fenológico, número de armadilha, quantidade detetada e tipo de agente identificado, podendo este corresponder a praga, fungo, vírus, bactéria ou outro. Esta estrutura permite registar informação agronómica mais detalhada e associá-la posteriormente a atividades de resposta.

O formulário de observações foi pensado para apoiar o registo de ocorrências no terreno de forma mais detalhada, como é demonstrado na Figura . Para além da descrição textual, o utilizador pode associar informação complementar, como fotografia, localização GPS, tipo de problema, quantidade observada e estado fenológico. Esta informação permite criar um histórico mais rico das ocorrências registadas na exploração.

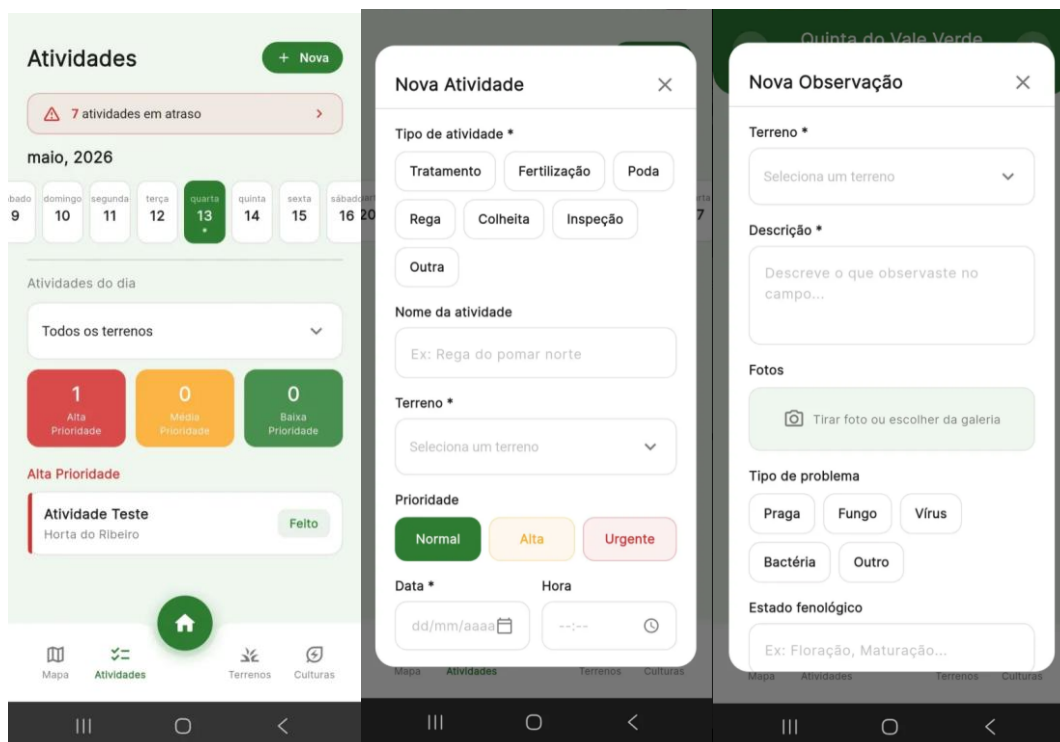


Figura 5.6 - Gestão de atividades e observações de campo

5.6 Fase 6 — Rega, agenda, notificações e suporte offline

A sexta fase integrou módulos complementares e funcionalidades de suporte ao funcionamento da aplicação, incluindo sensores IoT, rega, agenda, notificações e suporte *offline*.

No módulo de rega, foram consideradas informações relacionadas com regas realizadas e regas planeadas. Esta funcionalidade permite ao utilizador consultar informação relevante sobre a utilização de água na exploração e acompanhar operações associadas aos terrenos ou culturas, Figura 5.7 .

O módulo de rega foi estruturado de forma a permitir a consulta de regas por terreno, distinguindo entre regas planeadas e regas executadas. Esta organização permite acompanhar o histórico de utilização da água e consultar operações previstas, contribuindo para uma gestão mais organizada dos recursos hídricos da exploração.

Foi também considerada a gestão de fontes de água, permitindo associar à exploração diferentes tipos de origem, como furos, reservatórios ou linhas de água.

Foi também implementado o sistema de associação e leitura de sensores IoT. A aplicação passou a permitir que um utilizador com papel de Gestor associe nós sensores físicos a terrenos da exploração através de um fluxo em três passos: pesquisa do *gateway*, atribuição dos nós encontrados aos terrenos e gravação das associações no *backend*. As leituras dos sensores são obtidas a partir de uma instância Firebase Firestore externa, enquanto o *backend* Vision4Farms guarda apenas o mapeamento entre sensores e terrenos.

O módulo de sensores permite consultar os nós associados à exploração e visualizar dados de monitorização agrícola, como temperatura, humidade do ar e humidade do solo, Figura 5.7. A aplicação passou também a considerar alertas provenientes dos sensores, permitindo apresentar ocorrências relevantes ao utilizador, como possíveis situações de praga ou condições anómalas detetadas.

Para além da consulta dos nós sensores e dos alertas associados, o módulo de sensores foi organizado com um separador adicional dedicado ao apoio à decisão. Assim, o ecrã de sensores passou a incluir três áreas principais: nós, alertas e decisões. O separador de decisões apresenta recomendações geradas a partir da análise dos dados dos sensores, permitindo que a aplicação não se limite à apresentação de leituras em bruto.

As decisões são obtidas em tempo real a partir da coleção *decisoes* da instância Firebase Firestore externa utilizada pela componente de sensores. A aplicação apresenta as trinta decisões mais recentes, ordenadas por data de criação. Cada registo inclui informação sobre o tipo de anomalia detetada, o nó sensor associado, a ação sugerida, o resultado da decisão, o grau de confiança expresso em percentagem e os valores dos sensores que estiveram na origem da recomendação.

Esta funcionalidade aproxima a aplicação do conceito de agricultura de precisão, uma vez que transforma dados de monitorização em informação mais interpretável para o utilizador. À semelhança das leituras e dos alertas dos sensores, estas decisões são

produzidas por uma infraestrutura externa, ficando a aplicação Vision4Farms Mobile responsável pela sua consulta, organização e apresentação ao utilizador.

Na aplicação móvel, foram criados serviços específicos para suportar esta integração. O `FirebaseSensorService` ficou responsável pela consulta das leituras no Firestore externo, o `SensorAssociationService` pela comunicação com a API para gerir associações, o `SensorDataResolverService` pela agregação de leituras por terreno e o `SensorProvider` pela gestão do estado dos sensores na aplicação. Esta organização permitiu separar a lógica de sensores da restante lógica da aplicação.

O módulo de agenda foi desenvolvido para organizar eventos e tarefas por data. Esta funcionalidade complementa o módulo de atividades, permitindo planejar ações futuras e acompanhar eventos associados à exploração agrícola, Figura 5.7.

A agenda foi organizada com uma vista de calendário e filtros por terreno, permitindo consultar eventos de forma mais contextualizada. A criação de eventos inclui a seleção do tipo de evento, como tarefa, visita, reunião, lembrete ou outro, facilitando o planeamento de atividades futuras associadas à exploração.

As notificações foram implementadas em duas camadas: notificações *in-app*, persistidas na base de dados e visíveis no ecrã de notificações, e notificações *push*, enviadas através de Firebase Cloud Messaging. No *backend*, foram criados *endpoints* para registo e remoção de *tokens* FCM dos dispositivos dos utilizadores. Foi também implementado um mecanismo baseado em *signals* do Django, permitindo disparar automaticamente uma notificação *push* sempre que uma nova notificação é criada na base de dados.

Na aplicação móvel, foi criado o `FcmService`, responsável por pedir permissões ao utilizador, obter o *token* FCM, registá-lo no *backend* após *login* e removê-lo no *logout*. Este serviço trata mensagens recebidas com a aplicação aberta, em segundo plano ou terminada, permitindo ainda navegação contextual quando o utilizador toca numa notificação.

Foi também implementado o tratamento de alertas de sensores em tempo real. O `FirebaseAlertService` subscreve a coleção de alertas no Firestore externo e filtra

apenas os alertas associados aos sensores da exploração selecionada. Quando é detetado um alerta relevante, como uma possível praga, a aplicação cria uma notificação persistente no *backend*, que por sua vez pode originar uma notificação *push* para o utilizador.

Para além das notificações *push*, foi mantido um ecrã de notificações dentro da aplicação, permitindo consultar alertas e avisos associados à exploração. O utilizador pode acompanhar notificações relativas a atividades, observações, agenda ou sensores, mantendo um histórico acessível mesmo depois de a notificação inicial ter sido recebida, Figura 5.8.

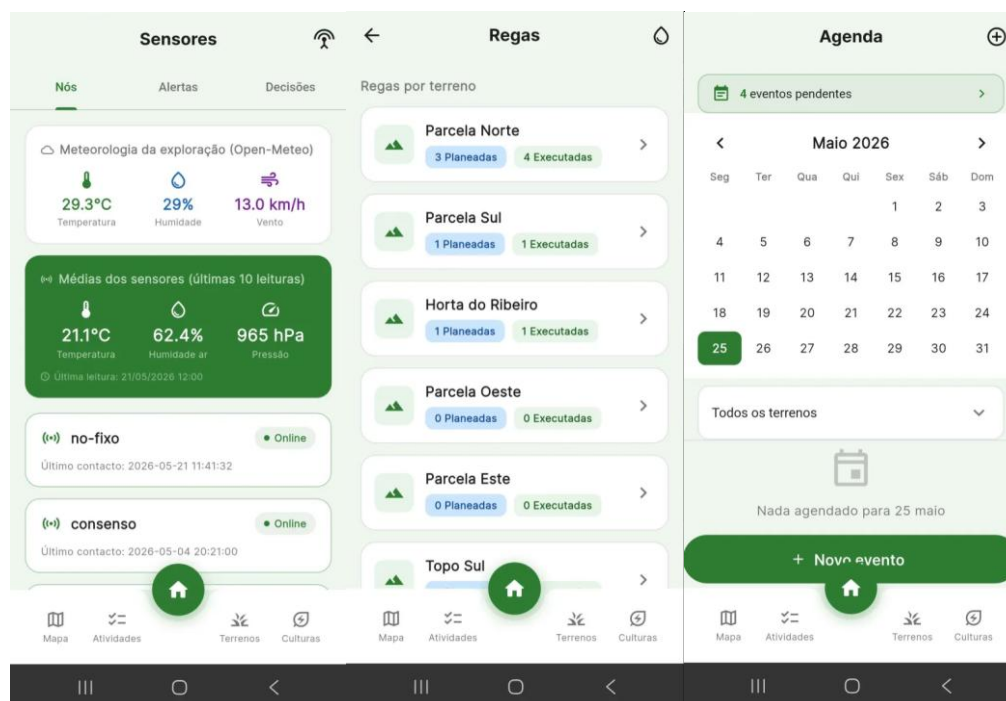


Figura 5.7 - Módulos de sensores, rega e agenda

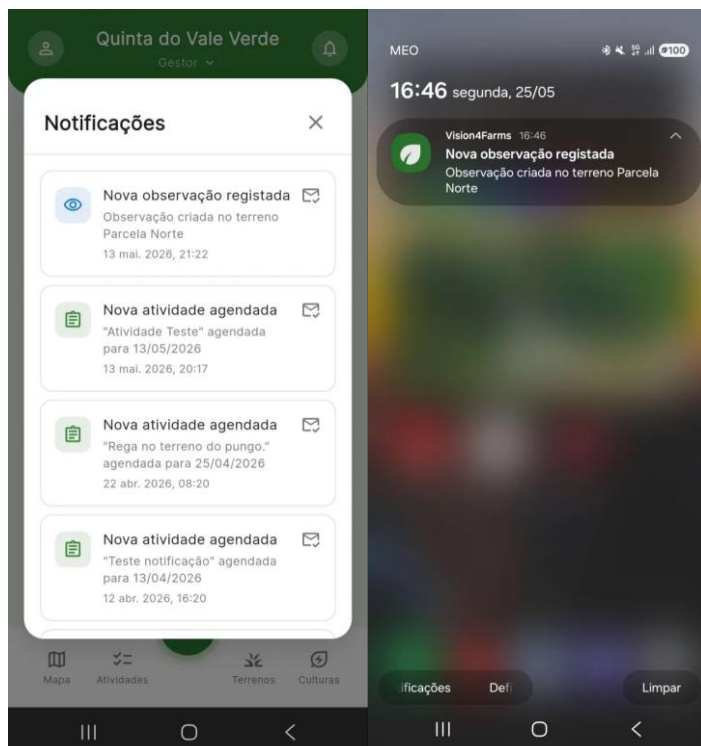


Figura 5.8 - Notificações in-app e notificações push

Nesta fase foi também desenvolvido o suporte *offline*, uma das componentes relevantes da aplicação. Foram criados mecanismos de cache local, leitura de dados sem ligação e registo de operações pendentes. Para isso, foi utilizada uma base de dados SQLite na aplicação móvel, permitindo guardar dados obtidos previamente da API e disponibilizá-los quando não existe ligação à internet.

As operações realizadas em modo offline são registadas numa estrutura local, incluindo a tabela *local_writes*, com recurso a UUID para garantir idempotência e evitar duplicações durante a sincronização. Foi também utilizada uma *sync_queue*, responsável por armazenar operações pendentes até que a ligação seja restabelecida.

O processo de sincronização inclui lógica de repetição até três tentativas. Erros de rede ou respostas 5xx são tratados como falhas temporárias, incrementando o número de tentativas. Por outro lado, erros 4xx são tratados como falhas definitivas, por normalmente indicarem problemas de validação, permissões ou dados inválidos. Esta distinção permite tornar o processo de sincronização mais controlado e previsível.

A aplicação passou ainda a apresentar indicação visual do estado offline, informando o utilizador quando está sem ligação e quando os dados apresentados podem corresponder a informação guardada localmente. Esta funcionalidade é particularmente relevante no contexto agrícola, onde a utilização da aplicação pode ocorrer em zonas com conectividade limitada.

O comportamento offline foi também refletido visualmente na interface através de *banners* informativos, Figura 5.9. Quando a aplicação se encontra sem ligação, o utilizador é informado de que os dados apresentados correspondem a informação guardada localmente. Quando a conectividade é restabelecida, a aplicação apresenta indicação de sincronização, comunicando que as operações pendentes estão a ser enviadas para o servidor, Figura 5.9. Esta transparência ajuda o utilizador a compreender o estado real da aplicação e a diferença entre dados atualizados e dados em cache.

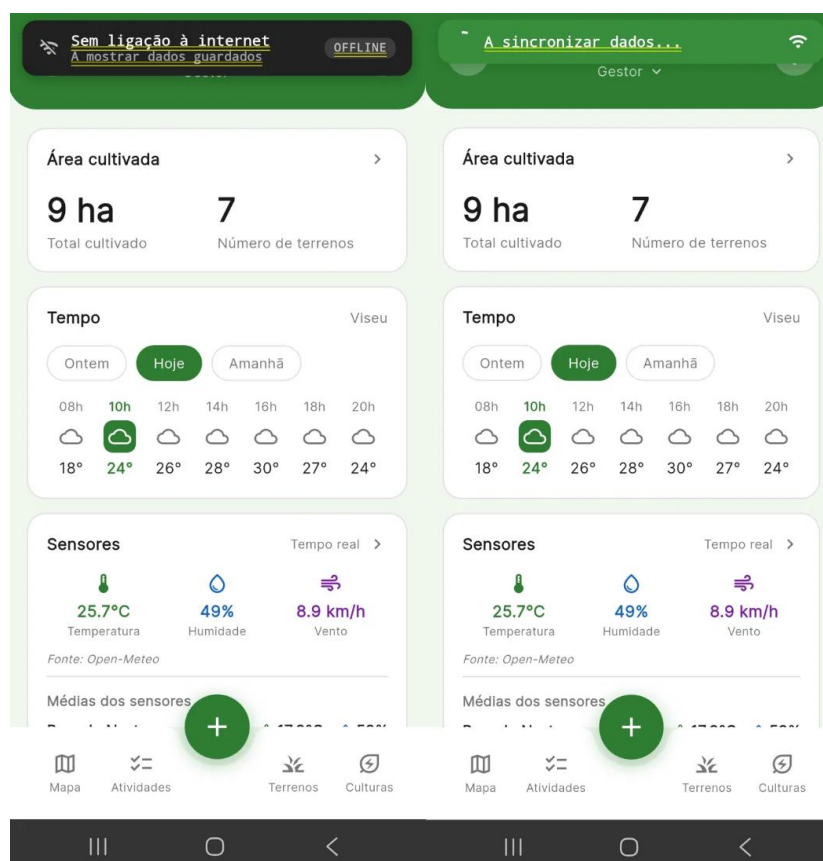


Figura 5.9 - Suporte offline e sincronização de dados

5.7 Testes, ajustes finais e preparação da APK

A fase final foi dedicada à realização de testes, correção de problemas, ajustes visuais e preparação da aplicação para validação em formato APK.

Antes da preparação final da APK, foi também implementado um mecanismo de verificação do estado da aplicação no arranque. Durante o ecrã de *splash*, a aplicação consulta o *endpoint /status* da API, que devolve informação sobre o estado do sistema, nomeadamente se o servidor se encontra em manutenção, qual a versão mínima da aplicação suportada e o endereço da loja de aplicações.

Com base nesta resposta, a aplicação pode apresentar ecrãs especiais ao utilizador. No modo de manutenção, é apresentada uma mensagem a indicar que o serviço se encontra temporariamente indisponível. No modo de atualização forçada, ativado quando a versão instalada é inferior à versão mínima exigida, o utilizador é encaminhado para atualizar a aplicação antes de poder continuar. A comparação de versões é realizada segmento a segmento, considerando o formato *major.minor.patch*.

Caso a verificação de estado falhe, por exemplo devido a uma indisponibilidade temporária do servidor, a aplicação prossegue normalmente, evitando bloquear o acesso do utilizador por uma falha momentânea de comunicação. O ecrã disponibiliza ainda uma opção para repetir a verificação, permitindo ao utilizador tentar novamente quando necessário.

Foram realizados testes manuais aos *endpoints* da API utilizando o Thunder Client, integrado no Visual Studio Code. Esta ferramenta permitiu validar respostas, autenticação, permissões e estrutura dos dados devolvidos antes da integração completa com a aplicação móvel.

Na aplicação Flutter, foram testados os principais fluxos de utilização, incluindo *login*, seleção de exploração, navegação entre separadores, carregamento do *dashboard*, consulta de mapa, listagem de terrenos, culturas, atividades, observações, notificações e funcionamento *offline*. Foram também verificados estados de carregamento, ausência de dados, erros e perda de ligação.

Foram também testados fluxos específicos associados à criação de terrenos, desenho de polígonos no mapa, criação de culturas, registo de atividades, registo de observações, consulta de sensores, visualização de alertas, utilização da agenda, receção de notificações e sincronização após recuperação da ligação. Estes testes permitiram identificar pequenos problemas de navegação, apresentação de dados e comportamento da interface, que foram corrigidos durante os ajustes finais.

Foram realizados ajustes de interface com base no comportamento real da aplicação, adaptando layouts, espaçamentos, mensagens e componentes visuais. A interface teve como referência inicial os layouts em Figma, mas foi ajustada ao longo da implementação para responder aos dados reais, limitações técnicas e necessidades funcionais do projeto.

Para permitir a utilização da aplicação fora do ambiente de desenvolvimento, foi preparada uma versão em formato APK. Como a aplicação precisava de comunicar com uma API acessível online, o *backend* foi disponibilizado no Railway. Esta etapa permitiu validar a comunicação entre a APK e a API alojada fora do ambiente local, aproximando o teste de um cenário real de utilização.

No final desta fase, o projeto apresentava uma base funcional composta por aplicação móvel, API/*backend*, base de dados, módulos principais, suporte *offline* e disponibilização externa para testes.

6 Resultados Obtidos

Neste capítulo são apresentados os principais resultados obtidos no desenvolvimento do projeto Vision4Farms Mobile. A análise encontra-se organizada por áreas, distinguindo as funcionalidades implementadas, os resultados alcançados no *backend*, na aplicação móvel, na base de dados e no suporte *offline*, bem como a validação dos objetivos definidos e as limitações identificadas.

6.1 Funcionalidades implementadas

O projeto resultou no desenvolvimento de uma base funcional do sistema Vision4Farms Mobile, composta por uma aplicação móvel, uma API/backend e uma base de dados adaptada ao contexto da gestão agrícola.

Entre as principais funcionalidades implementadas destacam-se os fluxos de autenticação, recuperação de palavra-passe, seleção de exploração agrícola, *dashboard*, mapa interativo, consulta de terrenos, culturas, atividades, observações de campo, análises, colheitas, rega, agenda, notificações, suporte *offline*, associação de sensores IoT a terrenos, alertas e decisões de apoio à decisão baseadas em dados dos sensores, geração de documentos PDF e notificações *push* com Firebase Cloud Messaging.

A aplicação foi organizada em torno do conceito de exploração, permitindo que o utilizador trabalhe sempre dentro de um contexto agrícola específico. Esta organização garante que os dados apresentados pertencem à exploração selecionada e facilita a consulta da informação.

O *dashboard* reúne, num único ecrã, informação resumida sobre a exploração, incluindo terrenos, área total, culturas ativas, atividades pendentes e atrasadas,

próximas regas, observações recentes, notificações não lidas e dados associados a sensores ou informação meteorológica externa. Quando existem sensores associados a terrenos, a aplicação apresenta médias agregadas por terreno; quando não existem sensores disponíveis, pode recorrer a dados meteorológicos externos como *fallback*.

O mapa interativo permite representar terrenos agrícolas através de coordenadas e polígonos, incluindo interação por *bottom sheet* e possibilidade de criação de atividades associadas ao terreno selecionado. Foram também implementados módulos de gestão de terrenos, culturas, atividades e observações de campo, permitindo estruturar informação relevante sobre a exploração e apoiar o registo de tarefas e ocorrências no terreno.

Com a integração dos sensores IoT, a aplicação passou ainda a permitir associar nós sensores físicos a terrenos, consultar leituras em tempo real, visualizar valores atuais no detalhe do terreno e consultar histórico através de gráficos. Foram também implementadas notificações *push* e alertas associados a eventos relevantes, incluindo alertas provenientes dos sensores.

6.2 Resultados obtidos no backend

No *backend*, foi desenvolvida uma API RESTful com Django REST Framework, responsável por servir a aplicação móvel e intermediar a comunicação com a base de dados MySQL.

A API foi organizada por módulos funcionais, permitindo separar responsabilidades e facilitar a manutenção. Foram desenvolvidas *apps* associadas a autenticação, explorações, terrenos, culturas, atividades, *dashboard*, mapa, agenda, notificações, rega, sensores e outros domínios do sistema.

Um dos resultados mais importantes foi a implementação do sistema de autenticação com JSON Web Token (JWT). Este sistema permitiu proteger os *endpoints* da API e garantir que apenas utilizadores autenticados podiam aceder aos dados. A verificação de palavra-passe foi realizada com `pbkdf2_sha256`, e o processo de recuperação de palavra-passe utilizou *tokens* temporários gerados com `secrets.token_urlsaf`.

Foi também implementado o controlo de acesso por exploração, garantindo que cada utilizador apenas acede às explorações às quais está associado. Outro resultado relevante foi a criação de *endpoints* agregadores, como o *endpoint* do *dashboard*, que devolve numa única resposta os principais dados necessários ao ecrã inicial da aplicação.

Com a evolução do projeto, o *backend* passou também a suportar a gestão de associações entre sensores e terrenos, através da *app sensors*. Esta *app* permite listar associações por exploração, criar ou atualizar associações em massa, alterar associações existentes e remover associações através de eliminação lógica. A leitura das associações ficou disponível para membros da exploração, enquanto as operações de escrita ficaram restritas a utilizadores com papel de Gestor.

A componente de notificações foi também expandida com integração do Firebase Admin SDK e Firebase Cloud Messaging. Foram criados *endpoints* para registo e remoção de *tokens* FCM dos dispositivos dos utilizadores, permitindo que o *backend* envie notificações *push* para os dispositivos associados a cada utilizador autenticado.

Foi ainda implementado um mecanismo baseado em *signals* do Django, permitindo disparar automaticamente uma notificação *push* sempre que uma nova notificação é criada no sistema. Esta abordagem centraliza o envio de notificações e evita a duplicação de lógica em diferentes pontos do *backend*.

O *backend* foi disponibilizado no Railway, permitindo que a aplicação em formato APK comunicasse com uma API acessível online, fora do ambiente local de desenvolvimento

Passou ainda a suportar a geração de documentos PDF para análises e colheitas, permitindo disponibilizar ficheiros estruturados e descarregáveis pela aplicação móvel. Foi também criado o *endpoint* */status*, utilizado pela aplicação durante o arranque para verificar o estado do sistema, a versão mínima suportada e a eventual necessidade de apresentar modo de manutenção ou atualização obrigatória.

6.3 Resultados obtidos na aplicação móvel

Na aplicação móvel, foi desenvolvida uma solução em Flutter, com estrutura preparada para Android e iOS, embora a validação tenha sido realizada em ambiente Android.

A aplicação inclui ecrãs de autenticação, seleção de exploração, *dashboard*, mapa, terrenos, culturas, atividades, observações, agenda, notificações, rega, sensores e áreas complementares. A navegação foi organizada através do *GoRouter*, permitindo controlar redirecionamentos consoante o estado de autenticação e a existência de exploração selecionada.

A gestão de estado foi estruturada através de *providers*, como *AuthProvider*, *FarmProvider*, *ConnectivityProvider*, *SyncProvider* e *SensorProvider*. A comunicação com a API foi centralizada no *ApiService*, utilizando Dio para pedidos HTTP, envio de *tokens* de autenticação e tratamento de situações como a expiração do *token*.

Com a integração dos sensores, foram implementados serviços específicos para leitura de dados no Firebase Firestore externo, associação de sensores a terrenos e agregação de leituras por terreno. A aplicação passou a permitir a associação de nós sensores físicos a terrenos da exploração, através de um fluxo orientado por *gateway*, seleção de terreno e gravação das associações no *backend*.

Os dados dos sensores passaram a ser apresentados em diferentes pontos da aplicação, nomeadamente no *dashboard*, na listagem de terrenos e no detalhe do terreno. Neste último caso, foram incluídos valores atuais e histórico de métricas como temperatura, humidade do ar, humidade do solo e pressão atmosférica.

Para além da consulta de leituras e alertas, a aplicação passou também a apresentar decisões de apoio à decisão provenientes da infraestrutura externa de sensores. Estas decisões permitem disponibilizar recomendações estruturadas ao utilizador, incluindo anomalia detetada, ação sugerida, grau de confiança e valores que originaram a recomendação.

Foram ainda implementadas funcionalidades complementares associadas à geração e abertura de documentos PDF, nomeadamente para análises de solo, análises foliares e registos de colheita. Esta funcionalidade exigiu integração entre *backend* e

aplicação móvel, permitindo descarregar documentos autenticados, guardar o ficheiro localmente e abri-lo no dispositivo Android.

A aplicação passou também a incluir um mecanismo de verificação de estado no arranque, permitindo apresentar ecrãs de manutenção ou atualização obrigatória quando definido pela API. Esta funcionalidade contribui para maior controlo da aplicação em ambiente de teste ou produção.

Foi também implementado o suporte a notificações *push* através do FcmService, responsável por pedir permissões, obter o *token* FCM, registá-lo no *backend* e tratar mensagens recebidas. Além disso, foi implementado o FirebaseAlertService, responsável por acompanhar alertas em tempo real no Firestore externo e filtrar apenas os alertas associados à exploração selecionada.

A interface foi desenvolvida com base em referências visuais em Figma, mas adaptada às necessidades funcionais da aplicação, aos dados reais da API e aos diferentes estados da interface, como carregamento, erro, ausência de dados, ausência de ligação e dados provenientes de sensores.

A preparação da aplicação em formato APK permitiu testar a solução fora do ambiente de desenvolvimento, aproximando a validação de um cenário real de utilização.

6.4 Resultados obtidos na base de dados

Ao nível da base de dados, foi utilizada uma estrutura MySQL adaptada ao contexto do Vision4Farms Mobile. O modelo de dados foi organizado em torno de entidades como utilizadores, perfis, empresas, explorações, terrenos, culturas, atividades, observações, agenda, notificações, rega, sensores e *tokens* de dispositivos.

No âmbito do trabalho de projeto, foram inicialmente criadas seis tabelas principais para suportar funcionalidades específicas da aplicação móvel: *observations*, *activities*, *activities_images*, *agenda*, *notifications* e *water_irrigation_planned*.

Estas tabelas permitiram suportar observações de campo, atividades agrícolas, imagens associadas, eventos de agenda, notificações do sistema e regas planeadas. A sua criação foi fundamental para tornar a aplicação mais completa e ajustada às necessidades da gestão agrícola.

Com a evolução do projeto, foram ainda acrescentadas tabelas de suporte à integração com sensores IoT e notificações *push*, nomeadamente *sensor_node_associations* e *fcm_tokens*. A tabela *sensor_node_associations* guarda o mapeamento entre nós sensores físicos e terrenos agrícolas, enquanto a tabela *fcm_tokens* armazena os tokens Firebase Cloud Messaging associados aos dispositivos dos utilizadores.

Os modelos Django foram configurados com *managed=False*, permitindo que o *backend* interagisse com a base de dados sem assumir a gestão automática da sua estrutura através das migrações do Django. A estrutura relacional permitiu ainda associar corretamente terrenos, culturas, atividades, observações, regas, notificações e sensores à respetiva exploração.

6.5 Resultados obtidos no suporte offline e sincronização

O suporte *offline* foi uma das componentes relevantes do projeto, tendo em conta o contexto agrícola em que a aplicação pode ser utilizada. Em zonas rurais ou no terreno, a conectividade pode ser instável, pelo que a aplicação foi preparada para lidar com ausência temporária de ligação.

Foi implementada uma base de dados local SQLite para permitir o armazenamento de dados em cache. Esta estrutura possibilita a consulta de informação previamente carregada, mesmo quando a aplicação não tem ligação ativa à internet.

Foi também criada uma lógica de registo de operações pendentes, recorrendo a estruturas como *local_writes* e *sync_queue*. As operações realizadas em modo *offline*

são guardadas localmente e identificadas com UUID, permitindo garantir idempotência e evitar duplicações no momento da sincronização.

O processo de sincronização foi concebido para reenviar operações pendentes quando a ligação é restabelecida. Foram consideradas até três tentativas de reenvio, distinguindo entre erros temporários, como falhas de rede ou respostas 5xx, e erros definitivos, como respostas 4xx associadas a validação, permissões ou dados inválidos.

A aplicação inclui ainda indicação visual do estado *offline*, permitindo informar o utilizador quando está sem ligação e quando os dados apresentados podem corresponder a informação guardada localmente.

6.6 Validação dos objetivos definidos

Os objetivos definidos para o projeto foram alcançados. O projeto resultou numa solução funcional composta por aplicação móvel, *API/backend*, base de dados, mecanismos de autenticação, módulos agrícolas principais e suporte *offline*.

O objetivo de desenvolver uma aplicação móvel de apoio à gestão agrícola foi cumprido através da implementação dos principais módulos previstos, nomeadamente *dashboard*, mapa, terrenos, culturas, atividades, observações, agenda, notificações e rega.

O objetivo de desenvolver uma API de suporte também foi alcançado, através da criação de um *backend* em Django REST Framework, organizado por módulos e integrado com a base de dados MySQL.

O objetivo de adaptar a base de dados ao contexto da aplicação móvel foi concretizado através da criação de novas tabelas e da integração dos modelos Django com a estrutura relacional do sistema.

O objetivo de implementar autenticação e controlo de acesso foi cumprido através do sistema JWT, recuperação de palavra-passe, gestão de utilizadores e associação de utilizadores a explorações.

O objetivo de preparar a aplicação para contextos de conectividade limitada foi abordado através da implementação de cache local, fila de sincronização, operações pendentes e indicação visual de estado *offline*.

Para além dos objetivos técnicos, permitiu também aplicar conhecimentos adquiridos ao longo da licenciatura em contexto real, nomeadamente em desenvolvimento *mobile*, APIs REST, bases de dados relacionais, autenticação, integração de sistemas, testes e disponibilização de aplicações.

6.7 Limitações identificadas

Apesar dos resultados obtidos, foram identificadas algumas limitações naturais no contexto de um projeto com duração limitada.

Uma das principais limitações prende-se com a validação da aplicação apenas em ambiente Android. Embora Flutter permita desenvolvimento para Android e iOS, os testes realizados incidiram sobre Android, nomeadamente através de emulador e APK.

Embora a integração com sensores IoT tenha sido implementada, esta depende de uma infraestrutura Firebase externa, pertencente à equipa responsável pelo hardware dos sensores. Assim, a aplicação Vision4Farms não controla diretamente a forma como as leituras são publicadas no Firestore, ficando dependente da disponibilidade, estrutura e consistência dessa fonte externa.

O sistema de notificações *push* foi implementado, mas poderá ser aprofundado com mecanismos de preferência por utilizador, maior parametrização dos alertas e configuração de lembretes automáticos em ambiente de produção. Esta evolução permitiria tornar as notificações mais ajustadas às necessidades de cada utilizador e exploração.

O suporte *offline* foi implementado como componente relevante da aplicação, mas poderá ser expandido e refinado. Futuramente, poderá ser necessário alargar a

sincronização a mais tipos de operações, melhorar a resolução de conflitos e reforçar os mecanismos de auditoria de sincronização.

Também existem limitações associadas ao tempo disponível para testes. Foram realizados testes manuais aos principais fluxos e *endpoints*, mas não foi possível implementar uma bateria completa de testes automatizados unitários, de integração e de interface.

Por fim, algumas funcionalidades poderão beneficiar de maior maturidade em versões futuras, nomeadamente relatórios avançados, análise histórica mais aprofundada dos dados dos sensores, maior parametrização das notificações, integração com mais tipos de sensores e melhorias na componente de visualização geográfica.

7 Conclusão

Neste capítulo é apresentada a conclusão do relatório, reunindo o balanço geral do projeto, os principais resultados alcançados, as competências desenvolvidas, as dificuldades encontradas e as possibilidades de trabalho futuro para o projeto Vision4Farms Mobile.

7.1 Balanço geral do projeto

O trabalho de projeto realizado na empresa Visionary Devotion Unipessoal Lda permitiu aplicar, em contexto real, conhecimentos adquiridos ao longo da Licenciatura em Engenharia Informática. O trabalho desenvolvido incidiu sobre o projeto Vision4Farms Mobile, uma solução móvel de apoio à gestão agrícola, composta por aplicação Flutter, API/*backend* em Django REST Framework e base de dados MySQL.

Ao longo das quinze semanas, foi possível acompanhar várias fases do desenvolvimento de software, desde a análise inicial e escolha de tecnologias até à implementação, integração, testes e preparação da aplicação em formato APK. Esta experiência permitiu compreender melhor o ciclo de desenvolvimento de uma solução digital, bem como a importância de decisões relacionadas com arquitetura, organização de código, comunicação entre sistemas e adaptação a requisitos concretos.

O balanço geral é positivo, uma vez que os objetivos definidos foram alcançados e resultaram numa base funcional sólida para a continuidade do projeto. Para além dos resultados técnicos, contribuiu também para o desenvolvimento de autonomia,

sentido de responsabilidade, capacidade de resolução de problemas e adaptação a um contexto profissional.

7.2 Principais resultados e contribuições

O principal resultado foi o desenvolvimento de uma base funcional do sistema Vision4Farms Mobile. Esta base inclui uma aplicação móvel, uma API/*backend* e uma base de dados adaptada às necessidades da gestão agrícola.

Na aplicação móvel, foram implementados os principais fluxos de utilização, incluindo autenticação, recuperação de palavra-passe, seleção de exploração, *dashboard*, mapa interativo, consulta de terrenos e culturas, atividades, observações, rega, agenda, notificações, suporte *offline*, sensores IoT e notificações *push*. A aplicação foi desenvolvida em Flutter, com estrutura preparada para Android e iOS, tendo sido validada em ambiente Android.

No *backend*, foi desenvolvida uma API RESTful em Django REST Framework, responsável pela comunicação entre a aplicação móvel e a base de dados. Esta API inclui mecanismos de autenticação, recuperação de palavra-passe, controlo de acesso por exploração, *endpoints* agregadores e suporte aos principais módulos funcionais da aplicação.

Ao nível da base de dados, foram criadas novas tabelas para suportar funcionalidades específicas da aplicação móvel, como observações de campo, atividades, imagens, agenda, notificações, regas planeadas, associações de sensores e *tokens* FCM dos dispositivos. Esta adaptação permitiu aproximar o modelo de dados das necessidades reais da aplicação.

Uma contribuição relevante foi a integração de sensores IoT através de uma arquitetura híbrida, em que as leituras são obtidas a partir de uma instância Firebase Firestore externa e o *backend* Vision4Farms mantém o mapeamento entre sensores e terrenos. Esta abordagem permitiu apresentar dados de monitorização nos terrenos sem duplicar a responsabilidade de armazenamento das leituras.

Outra contribuição importante foi a implementação de notificações *push* com Firebase Cloud Messaging, permitindo alertar o utilizador sobre eventos relevantes mesmo quando a aplicação não está aberta. A solução mantém também notificações *in-app* persistentes, garantindo que o utilizador consegue consultar o histórico de alertas dentro da aplicação.

Por fim, destaca-se a implementação de mecanismos de suporte *offline* e sincronização. Esta componente torna a aplicação mais adequada ao contexto agrícola, onde a conectividade pode ser limitada ou instável.

7.3 Competências desenvolvidas

O projeto permitiu desenvolver e consolidar várias competências técnicas e profissionais. Ao nível técnico, foram aprofundados conhecimentos em desenvolvimento *mobile* com Flutter, construção de APIs REST com Django REST Framework, modelação e integração com bases de dados MySQL, autenticação com JWT, comunicação HTTP, gestão de estado, navegação, mapas digitais, suporte *offline* e sincronização.

A integração entre aplicação móvel, *backend* e base de dados foi uma das áreas mais relevantes, uma vez que exigiu compreender o funcionamento completo do sistema e garantir que os dados circulavam corretamente entre as várias camadas da solução.

Foram também desenvolvidas competências relacionadas com integração de serviços externos, nomeadamente Firebase Firestore, Firebase Cloud Messaging e Firebase Admin SDK. Estas integrações permitiram trabalhar com dados de sensores IoT, notificações *push*, *tokens* de dispositivos e alertas em tempo real.

A componente de sensores permitiu aprofundar competências na integração de dados provenientes de uma fonte externa, na associação entre dados físicos e entidades da aplicação, e na apresentação de informação de monitorização em

contexto agrícola. Já a componente de notificações permitiu compreender melhor o funcionamento de permissões, *tokens*, mensagens em *foreground/background* e navegação contextual a partir de notificações.

Foram ainda desenvolvidas competências relacionadas com teste e validação, nomeadamente através da utilização do Thunder Client para testar *endpoints*, do Android Studio para validação da aplicação em ambiente Android e do Railway para disponibilização da API fora do ambiente local.

Ao nível profissional, contribuiu para reforçar competências de organização, autonomia, gestão de tempo, análise de problemas e tomada de decisão técnica. A necessidade de adaptar o trabalho aos requisitos do projeto e ao acompanhamento do orientador permitiu também desenvolver maior capacidade de comunicação e de resposta a alterações durante o desenvolvimento.

7.4 Dificuldades encontradas

Durante o projeto foram encontradas algumas dificuldades técnicas e organizacionais. Uma das principais dificuldades esteve relacionada com a dimensão do projeto, que envolveu várias componentes interdependentes: aplicação móvel, API/*backend*, base de dados, autenticação, mapas, *offline*, sensores, notificações e disponibilização externa. A necessidade de integrar todas estas partes exigiu uma organização cuidadosa e validação constante.

Outra dificuldade esteve relacionada com a adaptação da base de dados ao contexto da aplicação móvel. Como o projeto envolvia várias entidades relacionadas, foi necessário garantir que a estrutura de dados suportava corretamente os módulos previstos e que a API conseguia consultar e devolver informação adequada à aplicação.

A implementação do suporte *offline* também representou um desafio, uma vez que implicou lidar com cache local, operações pendentes, sincronização, conectividade e distinção entre erros temporários e definitivos. Esta funcionalidade exigiu uma

abordagem mais cuidada, por estar diretamente relacionada com a consistência dos dados.

A integração do mapa interativo com dados geográficos foi igualmente desafiante, sobretudo pela necessidade de representar terrenos através de coordenadas e polígonos e de relacionar essa informação com dados vindos da API.

Com a evolução do projeto, a integração dos sensores IoT trouxe também dificuldades adicionais. As leituras dos sensores eram provenientes de uma instância Firebase Firestore externa, pertencente à equipa responsável pelo hardware, o que exigiu uma separação clara entre a origem dos dados em tempo real e o *backend* Vision4Farms, responsável apenas pelo mapeamento entre sensores e terrenos.

A configuração de duas instâncias Firebase na aplicação também exigiu cuidado, uma vez que a instância principal ficou associada às notificações *push* do projeto Vision4Farms, enquanto a instância secundária foi utilizada para consultar os dados dos sensores. Esta separação foi importante para evitar conflitos e garantir que cada serviço comunicava com o projeto Firebase correto.

A implementação das notificações *push* também representou um desafio técnico, nomeadamente ao nível da configuração do Firebase Cloud Messaging, gestão de *tokens* dos dispositivos, integração com o *backend* e tratamento das mensagens quando a aplicação se encontra aberta, em segundo plano ou encerrada.

Por fim, a disponibilização da aplicação em formato APK exigiu que o *backend* estivesse acessível fora do ambiente local, o que levou à utilização do Railway. Esta etapa permitiu ultrapassar a limitação do desenvolvimento local, mas exigiu ajustes na configuração da aplicação, da API e das variáveis de ambiente.

7.5 Trabalho futuro

Apesar dos objetivos definidos terem sido alcançados, o projeto Vision4Farms Mobile apresenta margem para evolução futura.

Uma das possibilidades passa pelo aprofundamento da componente de sensores, nomeadamente através da integração de mais tipos de sensores, da melhoria da análise histórica das leituras e da criação de indicadores mais avançados sobre temperatura, humidade, solo, água ou outros dados relevantes para a exploração agrícola.

Outra área de evolução consiste no reforço do suporte *offline* e da sincronização. Embora tenham sido implementados mecanismos de cache, fila de operações pendentes e sincronização, esta componente poderá ser expandida para abranger mais tipos de operações, melhorar a resolução de conflitos e reforçar os mecanismos de auditoria.

Também poderá ser desenvolvida uma camada mais avançada de análise de dados, com relatórios, gráficos, indicadores de produtividade, histórico de operações, evolução dos sensores e apoio à tomada de decisão. Esta evolução permitiria transformar a aplicação não apenas numa ferramenta de registo e consulta, mas também num instrumento de análise e planeamento.

A componente de notificações poderá igualmente ser aprofundada, através da criação de preferências por utilizador, maior parametrização dos alertas, configuração de lembretes automáticos e execução periódica de tarefas em ambiente de produção. Esta evolução permitiria adaptar melhor as notificações às necessidades de cada exploração e de cada perfil de utilizador.

Outra melhoria concreta passa pela implementação de um *endpoint* específico para o formulário de suporte existente na aplicação. Esta evolução permitiria que os pedidos enviados pelo utilizador fossem processados pelo *backend*, armazenados na base de dados ou encaminhados para a equipa responsável.

No futuro, seria também importante validar a aplicação em dispositivos iOS, realizar testes com utilizadores reais em contexto agrícola e implementar testes automatizados para aumentar a robustez do sistema. Estas melhorias permitiriam consolidar a solução e aproximá-la de uma versão mais madura para utilização em produção.

Referências Bibliográficas

AgriWebb. (2026). *Farm management software*. Consultado em 20 de fevereiro de 2026, de <https://www.agriwebb.com>

Agrivi. (2026). *Farm management software*. Consultado em 20 de fevereiro de 2026, de <https://www.agrivi.com>

Android Developers. (2026). *Android Studio*. Consultado em 20 de fevereiro de 2026, de <https://developer.android.com/studio>

Bee2Crop. (2026). *Bee2Crop*. Consultado em 20 de fevereiro de 2026, de <https://www.bee2crop.com>

Cropio. (2026). *Crop monitoring system*. Consultado em 20 de fevereiro de 2026, de <https://www.cropio.com>

Django REST Framework. (2026). *Django REST framework*. Consultado em 23 de fevereiro de 2026, de <https://www.django-rest-framework.org>

Django Software Foundation. (2026). *Django documentation*. Consultado em 27 de fevereiro de 2026, de <https://docs.djangoproject.com>

European Commission. (2025). *Digitalising the EU agricultural sector*. Consultado em 25 de fevereiro de 2026, de <https://digital-strategy.ec.europa.eu/en/policies/digitalisation-agriculture>

European Parliamentary Research Service. (2016). *Precision agriculture and the future of farming in Europe*. European Parliament. [https://www.europarl.europa.eu/RegData/etudes/STUD/2016/581892/EPRS_STU\(2016\)581892_EN.pdf](https://www.europarl.europa.eu/RegData/etudes/STUD/2016/581892/EPRS_STU(2016)581892_EN.pdf)

FAO. (2026). *Digital agriculture and AI innovation*. Food and Agriculture Organization of the United Nations. Consultado em 25 de fevereiro de 2026, de <https://www.fao.org/innovation/digital-agriculture-and-ai-innovation/en>

FarmLogs. (2026). *FarmLogs*. Consultado em 25 de fevereiro de 2026, de <https://farmlogs.com>

Figma. (2026). *Figma*. Consultado em 1 de março de 2026, de <https://www.figma.com>

Firebase. (2026a). *Cloud Firestore*. Consultado em 15 de maio de 2026, de <https://firebase.google.com/docs/firestore>

Firebase. (2026b). *Firebase Admin SDK*. Consultado em 17 de maio de 2026, de <https://firebase.google.com/docs/admin/setup>

Firebase. (2026c). *Firebase Cloud Messaging*. Consultado em 17 de maio de 2026, de <https://firebase.google.com/docs/cloud-messaging>

Flutter. (2026). *Build apps for any screen*. Consultado em 18 de fevereiro de 2026, de <https://flutter.dev>

Git. (2026). *Git documentation*. Consultado em 27 de maio de 2026, de <https://git-scm.com/doc>

Mapbox. (2026). *Maps SDK for Flutter*. Consultado em 7 de março de 2026, de <https://docs.mapbox.com/flutter/maps/guides>

MySQL. (2026a). *MySQL documentation*. Consultado em 7 de março de 2026, de <https://dev.mysql.com/doc/>

MySQL. (2026b). *MySQL Workbench*. Consultado em 7 de março de 2026, de <https://www.mysql.com/products/workbench/>

Open-Meteo. (2026). *Open-Meteo weather forecast API*. Consultado em 3 de maio de 2026, de <https://open-meteo.com>

Dio (2026a). *Dio package*. Consultado em 15 de março de 2026, de <https://pub.dev/packages/dio>

Flutter Secure Storage package (2026b). *Flutter Secure Storage package*. Consultado em 7 de março de 2026, de https://pub.dev/packages/flutter_secure_storage

Railway. (2026). *Deploy a Django app*. Consultado em 9 de abril de 2026, de <https://docs.railway.com/guides/django>

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The definitive guide to Scrum: The rules of the game*. Scrum.org. Consultado em 27 de maio de 2026, de <https://scrumguides.org/scrum-guide.html>

Soussi, A., Zero, E., Sacile, R., Trincherro, D., & Fossa, M. (2024). Smart sensors and smart data for precision agriculture: A review. *Sensors*, 24(8), 2647. <https://doi.org/10.3390/s24082647>

SQLite. (2025). *About SQLite*. Consultado em 27 de maio de 2026, de <https://www.sqlite.org/about.html>

Thunder Client. (2026). *Thunder Client*. Consultado em 5 de abril de 2026, de <https://www.thunderclient.com/>

Vision4You. (2023). *About*. Consultado em 20 de maio de 2026, de <https://vision4you.pt/about>

Visual Studio Code. (2026). *Visual Studio Code documentation*. Consultado em 20 de fevereiro de 2026, de <https://code.visualstudio.com/docs>

Wisecrop. (2026). *Wisecrop*. Consultado em 20 de fevereiro de 2026, de <https://www.wisecrop.com>

Anexo A – Planeamento de atividades

A tabela seguinte apresenta o planeamento geral do projeto, organizado por fases, semanas e resultados esperados.

Tabela A.1 - Planeamento de atividades do projeto

FASE	SEMANAS	PRINCIPAIS ATIVIDADES	RESULTADO ESPERADO
FASE 1	1 a 2	Análise da documentação, estudo de soluções concorrentes, análise de mercado, escolha das tecnologias e preparação do ambiente de desenvolvimento	Compreensão do projeto, definição do enquadramento e ambiente preparado
FASE 2	3 a 5	Adaptação da base de dados, desenvolvimento da API inicial funcional, API/ <i>backend</i> , autenticação, recuperação de palavra-passe, sistema de convites e primeiros <i>endpoints</i>	Base de dados adaptada, API inicial funcional, autenticação e controlo de acesso implementados
FASE 3	6 a 7	Estrutura base da aplicação móvel, navegação, gestão de explorações, integração com a API e implementação dos fluxos iniciais	Aplicação Flutter estruturada, navegação criada e comunicação inicial com a API

FASE 4	8 a 9	Desenvolvimento do <i>dashboard</i> e do mapa interativo com Mapbox, incluindo representação dos terrenos através de polígonos	<i>Dashboard</i> funcional e mapa com representação geográfica dos terrenos
FASE 5	10 a 12	Desenvolvimento dos módulos de terrenos, culturas, atividades, observações de campo, análises e criação de terrenos com polígonos	Módulos agrícolas principais implementados e integrados com a API
FASE 6	13 a 15	Desenvolvimento dos módulos de rega, agenda, sensores IoT, notificações, suporte <i>offline</i> , sincronização, testes, ajustes finais e preparação da APK	Funcionalidades complementares concluídas, suporte <i>offline</i> implementado, sincronização validada e APK preparada

Anexo B – Modelo de dados

A tabela seguinte apresenta uma síntese das principais tabelas utilizadas no projeto Vision4Farms Mobile e a respetiva finalidade no sistema.

Tabela B.1 - Tabelas da base de dados

TABELA	DESCRIÇÃO
USERS	Credenciais de acesso dos utilizadores
PROFILES	Dados pessoais e de contacto dos utilizadores
COMPANIES	Empresas agrícolas associadas ao sistema
FARMS	Explorações agrícolas
PROFILES_HAS_FARMS	Relação entre utilizadores e explorações, incluindo o papel do utilizador
LANDS	Terrenos pertencentes a uma exploração
CROPS	Catálogo de culturas disponíveis
CROPS_VARIETIES	Variedades associadas às culturas
YIELDS	Culturas plantadas ou parcelas de cultivo apresentadas na <i>app</i>
OBSERVATIONS	Observações de campo registadas na aplicação móvel
ACTIVITIES	Atividades agrícolas associadas à exploração, terreno ou cultura

ACTIVITIES_IMAGES	Imagens associadas a atividades e observações
AGENDA	Eventos e tarefas calendarizadas
NOTIFICATIONS	Notificações associadas ao utilizador e à exploração
WATER_SOURCE	Fontes de água da exploração
WATER_USAGE_LOG	Registos de consumo de água e histórico de rega
WATER_IRRIGATION_PLANNED	Regas planeadas
LANDS_SOIL_ANALYSIS	Análises de solo associadas aos terrenos
YIELDS_ANALYSIS	Análises foliares associadas às culturas
SENSOR_NODE_ASSOCIATIONS	Associações entre nós sensores físicos e terrenos agrícolas
FCM_TOKENS	<i>Tokens</i> Firebase Cloud Messaging associados aos dispositivos dos utilizadores
YIELDS_HARVESTS	Registos de colheitas associados às culturas, incluindo quantidade produzida, unidade de medida e custos associados

Anexo C – Endpoints da API

A tabela seguinte apresenta uma síntese dos principais *endpoints* disponibilizados pela API desenvolvida no âmbito do projeto Vision4Farms Mobile.

Tabela C.1 - Principais *endpoints* da API

MÉTODO	ENDPOINT	DESCRIÇÃO
GET	/api/status/	Verificação do estado da aplicação, modo de manutenção e versão mínima suportada
POST	/api/auth/login/	Login do utilizador
POST	/api/auth/register/	Registo por convite
POST	/api/auth/refresh/	Renovação do <i>token</i> de acesso
GET	/api/auth/me/	Obtenção dos dados do utilizador autenticado
POST	/api/auth/logout/	<i>Logout</i> do utilizador
POST	/api/auth/password-reset/	Pedido de recuperação de palavra-passe

POST	/api/auth/password-reset/confirm/	Confirmação de nova palavra-passe
PATCH	/api/auth/profile/	Atualização do perfil do utilizador
POST	/api/auth/password-change/	Alteração de palavra-passe
POST	/api/auth/profile/picture/	<i>Upload</i> da fotografia de perfil
GET	/api/farms/	Listagem de explorações
POST	/api/farms/create/	Criação de exploração
GET	/api/farms/{id}/	Detalhe de uma exploração
POST	/api/farms/{id}/invite/generate/	Geração de convite para exploração
POST	/api/farms/invite/join/	Entrada numa exploração por convite
GET	/api/dashboard/{farm_id}/	Obtenção dos dados do <i>dashboard</i>
GET	/api/dashboard/farms/	Listagem de explorações disponíveis para o <i>dashboard</i>
GET	/api/map/{farm_id}/	Obtenção dos dados do mapa
GET / POST	/api/lands/	Listagem e criação de terrenos
GET	/api/lands/{id}/	Obtenção do detalhe de um terreno

GET	/api/lands/{id}/irrigation/	Obtenção dos dados de rega associados a um terreno
GET	/api/lands/{id}/soil-analyses/	Listagem das análises de solo de um terreno
POST	/api/lands/{id}/soil-analyses/create/	Criação de análise de solo
GET	/api/lands/soil-analyses/{id}/	Obtenção do detalhe de uma análise de solo
GET	/api/lands/soil-analyses/{id}/pdf/	Geração ou descarregamento do PDF de uma análise de solo
GET / POST	/api/crops/	Listagem e criação de culturas do catálogo
GET	/api/crops/farm/{farm_id}/	Listagem de culturas associadas a uma exploração
GET	/api/crops/{id}/hub/	Obtenção da informação agregada de uma cultura
GET / POST	/api/yields/	Listagem e criação de culturas plantadas
GET	/api/yields/{id}/	Obtenção do detalhe de uma cultura plantada
GET	/api/yields/{id}/analyses/	Listagem das análises foliares de uma cultura
POST	/api/yields/{id}/analyses/create/	Criação de análise foliar

GET	/api/yields/analyses/{id}/pdf/	Geração ou descarregamento do PDF de uma análise foliar
GET / POST	/api/yields/{id}/harvests/	Listagem e criação de colheitas de uma cultura
POST	/api/yields/{id}/harvests/create/	Criação de registo de colheita
GET	/api/yields/harvests/{id}/pdf/	Geração ou descarregamento do PDF de uma colheita
GET / POST	/api/activities/	Listagem e criação de atividades
GET / POST	/api/activities/observations/	Listagem e criação de observações
POST	/api/activities/observations/{id}/upload/	<i>Upload</i> de imagem associada a uma observação
GET / POST	/api/agenda/	Listagem e criação de eventos
GET / PATCH	/api/agenda/{id}/	Consulta ou atualização de evento
DELETE	/api/agenda/{id}/delete/	Eliminação de evento
GET	/api/notifications/	Listagem de notificações
PATCH	/api/notifications/{id}/read/	Marcação de notificação como lida
PATCH	/api/notifications/{id}/toggle-read/	Alternância entre notificação lida e não lida

GET	/	/api/notifications/preferences/	Consulta e atualização das preferências de notificações
PATCH			
POST		/api/notifications/devices/	Registo de <i>token</i> FCM do dispositivo
DELETE		/api/notifications/devices/{token}/	Remoção de <i>token</i> FCM do dispositivo
POST		/api/notifications/sensor-alert/	Criação de notificação associada a alerta de sensor
GET		/api/water/types/	Listagem dos tipos de fontes de água
GET		/api/water/methods/	Listagem dos métodos de rega
GET	/	/api/water/sources/	Listagem e criação de fontes de água
POST			
GET	/	/api/water/usage-logs/	Listagem e criação de registos de consumo de água
POST			
GET	/	/api/water/planned/	Listagem e criação de regas planeadas
POST			
POST		/api/water/planned/{id}/execute/	Execução de uma rega planeada
GET		/api/sensors/associations/?farm_id={id}	Listagem das associações de sensores de uma exploração

POST	/api/sensors/associations/bulk/	Criação ou atualização em massa de associações de sensores
PATCH	/api/sensors/associations/{id}/	Atualização de uma associação de sensor
DELETE	/api/sensors/associations/{id}/	Remoção lógica de uma associação de sensor

Os *endpoints* apresentados correspondem aos principais recursos utilizados pela aplicação móvel. Alguns *endpoints* foram agrupados por módulo para simplificar a apresentação no anexo.

Anexo D – Capturas de ecrã da aplicação móvel

O presente anexo apresenta um conjunto de capturas de ecrã da aplicação Vision4Farms Mobile, organizadas por módulos funcionais. As imagens ilustram os principais fluxos de utilização implementados, desde a autenticação e gestão de explorações até aos módulos de *dashboard*, mapa, terrenos, atividades, sensores, rega, culturas, agenda e notificações.

D.1 Autenticação e acesso

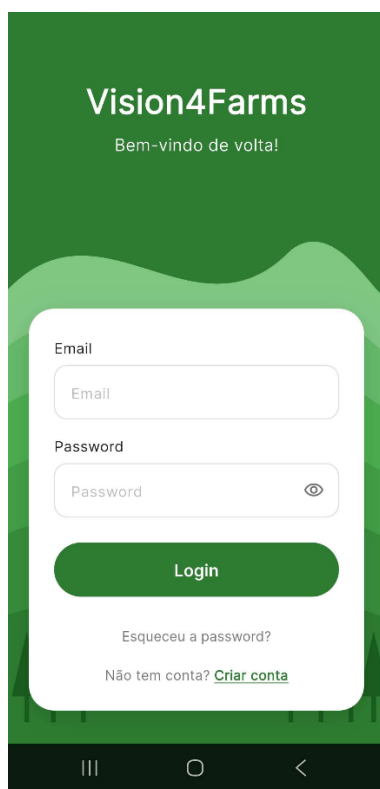


Figura D.1 - Ecrã de autenticação

Figura D.2 - Formulário de registo com código de convite

Figura D.3 - Ecrã de recuperação de palavra-passe

D.2 Gestão de explorações

← Nova Exploração

Identificação da exploração

Nome

Insira o nome

Data de fundação / aquisição

dd/mm/aaaa

Valor de aquisição

0,00€

Morada

Morada

Insira a morada da exploração

Código-Postal

Insira o código ...

Localidade

Insira a localid...

+ Adicionar

Figura D.4 - Formulário de criação de exploração

As Suas Explorações

Todas as suas explorações

Quinta do Vale Verde

Viseu, Viseu 9 ha

Gestor

Parcelas da Serra

Viseu, Viseu 0 ha

Gestor

12

Viseu, Viseu 0 ha

Gestor

Viseu

Viseu, Viseu 0 ha

Gestor

+ Adicionar nova exploração

Explorações

Juntar

Figura D.5 - Listagem de explorações do utilizador



Figura D.6 - Ecrã de entrada por código de convite

D.3 Dashboard, perfil e definições



Figura D.7 - Dashboard com área cultivada, meteorologia e sensores



Figura D.8 - Dashboard com atividades, agenda, rega e análises

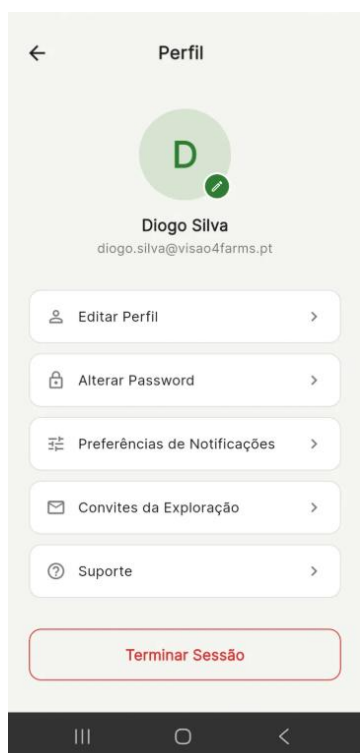


Figura D.9 - Ecrã de perfil com opções de gestão

← **Editar Perfil**

Nome
Diogo Silva

Email
diogo.silva@visao4farms.pt

Telemóvel
+351912345678

Telefone
+351232345678

NIF
234567891

NIFAP
123456789

Figura D.10 - Formulário de edição de perfil

← **Preferências de Notificações**

Notificações imediatas
Recebidas no momento em que são criadas

Atividades
Novas atividades ☒

Observações
Novas observações de campo ☒

Sistema
Alertas e atualizações do sistema ☒

Notificações com lembrete
Avisam-te com antecedência antes do evento

Agenda
Novos eventos e tarefas do calendário ☒

Rega
Planeamento e registos de rega ☒

Antecedência do lembrete
Quanto tempo antes queres ser avisado

Figura D.11 - Configuração de preferências de notificações

D.4 Mapa e terrenos



Figura D.12 - Mapa interativo com marcadores de terrenos e legenda de estado



Figura D.13 - *Bottom sheet* de pré-visualização do terreno no mapa



Figura D.14 - Listagem dos terrenos da exploração

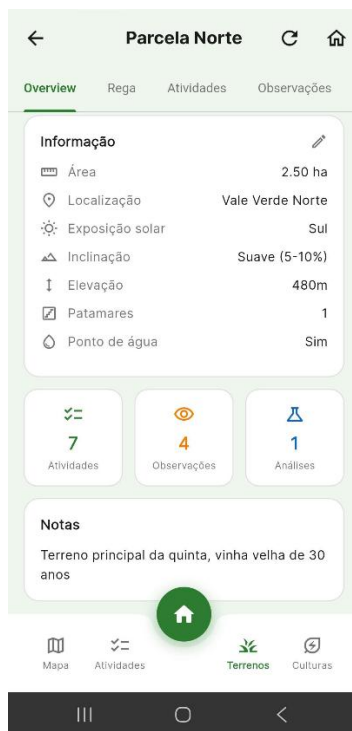


Figura D.15 - Ecrã de detalhe do terreno

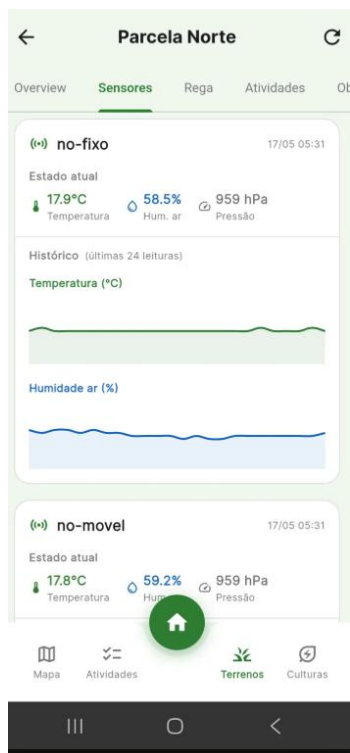


Figura D.16 - Dados de sensores com gráficos de histórico



Figura D.17 - Próximas regas e histórico de rega do terreno



Figura D.18 - Atividades associadas ao terreno



Figura D.19 - Observações de campo do terreno



Figura D.20 - Análise de solo com valores laboratoriais

D.5 Atividades, sensores, rega e culturas



Figura D.21 - Listagem de atividades agrícolas com indicador de atrasos e filtro por prioridade

Nova Atividade ✕

Tipo de atividade *

Tratamento Fertilização Poda

Rega Colheita Inspeção

Outra

Nome da atividade

Ex: Rega do pomar norte

Terreno *

Selecione um terreno ▼

Prioridade

Normal Alta Urgente

Data * Hora

dd/mm/aaaa 📅 --:-- 🕒

Mapa Atividades Terrenos Culturas

Figura D.22 - Formulário de criação de atividade agrícola com seleção de tipo e prioridade

Sensores 📶

Nós Alertas Decisões

☁ Meteorologia da exploração (Open-Meteo)

18.3°C 70% 8.4 km/h

Temperatura Humidade Vento

↔ Médias dos sensores (últimas 10 leituras)

21.1°C 62.4% 965 hPa

Temperatura Humidade ar Pressão

🕒 Última leitura: 21/05/2026 12:00

📶 no-fixo Online

Último contacto: 2026-05-21 11:41:32

📶 consenso Online

Último contacto: 2026-05-04 20:21:00

Mapa Atividades Terrenos Culturas

Figura D.23 - Módulo de sensores, nós, alertas e decisões de apoio à decisão

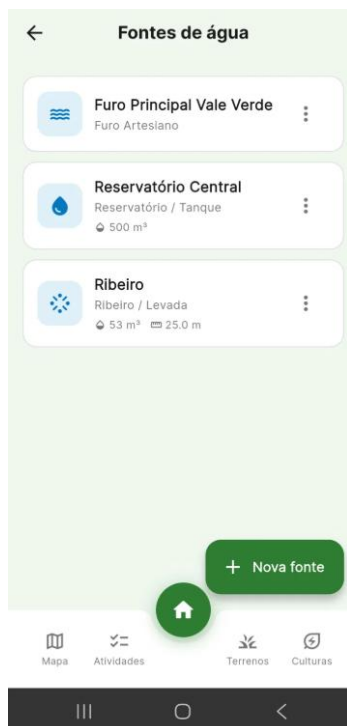


Figura D.24 - Gestão de fontes de água da exploração



Figura D.25 - Listagem de culturas com métricas resumidas



Figura D.26 - Detalhe de cultura com informação agronómica e variedades

D.6 Agenda e notificações



Figura D.27 - Módulo de agenda com calendário e filtro por terreno

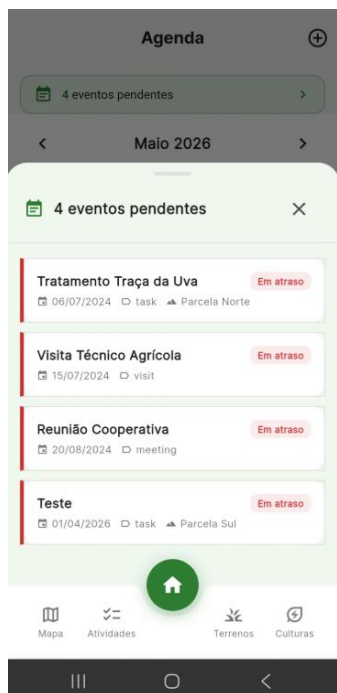


Figura D.28 - Eventos pendentes e em atraso

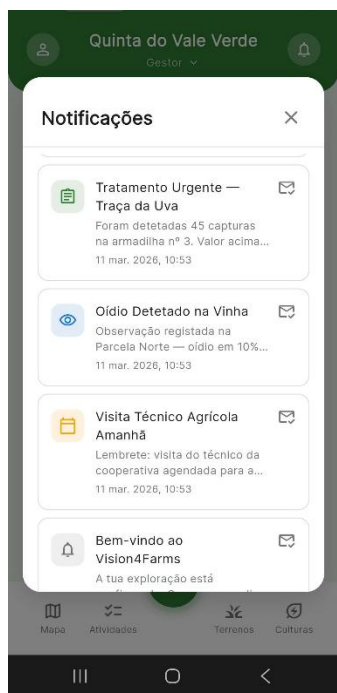


Figura D.29 - Ecrã de notificações com alertas de atividades e observações

Anexo E – Manual de Instalação e Execução

O presente anexo apresenta os requisitos e os principais passos necessários para executar o *backend*, executar a aplicação móvel em ambiente de desenvolvimento e instalar a versão APK da aplicação Vision4Farms Mobile.

Requisitos

Backend:

- Python 3.10 ou superior;
- MySQL 8.0 ou superior;
- pip;
- ficheiro .env configurado;
- base de dados MySQL criada/importada.

Aplicação móvel:

- Flutter SDK 3.x;
- Android Studio;
- dispositivo Android ou emulador;
- Android 8.0 ou superior.

Execução do *backend* localmente

Para executar o *backend* em ambiente local, deve ser criada e ativada uma virtual environment Python.

```
python -m venv venv
venv\Scripts\activate
```

De seguida, devem ser instaladas as dependências do projeto:

```
pip install -r requirements.txt
```

Após a instalação das dependências, deve ser configurado o ficheiro `.env` com as variáveis necessárias ao funcionamento da aplicação. Um exemplo de configuração é apresentado de seguida:

```
DB_NAME=vision4farms
DB_USER=root
DB_PASSWORD=...
DB_HOST=localhost
SECRET_KEY=...
EMAIL_HOST_USER=...
EMAIL_HOST_PASSWORD=...
```

Depois de configuradas as variáveis de ambiente, deve ser importado o esquema SQL para a base de dados MySQL. Esta operação pode ser realizada através do MySQL Workbench ou de outra ferramenta compatível com MySQL.

Por fim, o servidor Django pode ser iniciado com o seguinte comando:

```
python manage.py runserver
```

Após este comando, a API fica disponível localmente, por defeito, no endereço:

```
http://127.0.0.1:8000/
```

Execução da aplicação móvel em ambiente de desenvolvimento

Antes de executar a aplicação móvel, devem ser instaladas as dependências Flutter:

```
flutter pub get
```

Caso a aplicação seja testada num dispositivo Android físico ligado por USB e o *backend* esteja a correr localmente no computador, pode ser necessário redirecionar a porta 8000 através do ADB:

```
adb reverse tcp:8000 tcp:8000
```

De seguida, a aplicação pode ser executada com o comando:

```
flutter run
```

A aplicação móvel comunica com a API através do endereço configurado no ficheiro `app_constants.dart`. Em ambiente local, este endereço deve apontar para o servidor Django em execução. Em ambiente de teste com API online, deve apontar para o URL público da API.

Instalação da APK

Para instalar a aplicação em formato APK num dispositivo Android, devem ser seguidos os seguintes passos:

1. Copiar o ficheiro `app-release.apk` para o dispositivo Android.
2. Ativar a opção de instalação a partir de fontes desconhecidas nas definições do dispositivo, caso seja necessário.
3. Abrir o ficheiro APK no dispositivo.
4. Confirmar a instalação da aplicação.
5. Abrir a aplicação após a instalação.

A aplicação requer ligação à API para autenticação, carregamento de dados principais e sincronização. Por esse motivo, antes de gerar ou instalar a APK, deve ser confirmado que o endereço da API se encontra corretamente configurado no ficheiro `app_constants.dart`.

Utilização da API alojada no Railway

Para permitir o teste da aplicação em formato APK fora do ambiente local, a API/*backend* pode ser disponibilizada online através do Railway. Neste cenário, a aplicação móvel não deve apontar para *localhost* ou 127.0.0.1, mas sim para o URL público da API alojada.

Assim, no ficheiro `app_constants.dart`, o endereço base da API deve ser configurado com o URL disponibilizado pelo Railway. Esta configuração permite que a aplicação

instalada no dispositivo Android comunique com o *backend* mesmo quando não está ligada ao computador de desenvolvimento.

Esta abordagem foi utilizada para aproximar os testes da aplicação de um cenário real de utilização, permitindo validar a comunicação entre a APK, a API/*backend* e a base de dados fora do ambiente local.